

DevOps Lab Report

Naman Singh

SAP ID: 500123185

<https://pariharnaman.github.io/containerization-devops/>

containerization-devops

Containerization and DevOps

Academic Repository

Student Details

- **Name:** Naman Singh
 - **SAP ID:** 500123185
 - **Batch:** 2
 - **Specialisation:** CCVT
-

About This Repository

This repository contains theoretical concepts and practical lab experiments related to the subject **Containerization and DevOps**.

It is structured to support:

- Conceptual understanding
 - Practical implementation
 - Industry-relevant DevOps workflows
-

Learning Objectives

- Understand the fundamentals of Containerization
- Learn core DevOps principles and lifecycle
- Work with modern tools such as Docker and Git
- Automate build, deployment, and integration workflows

- Gain hands-on experience through structured lab experiments

Topics Covered

- Introduction to Containerization
 - Virtual Machines vs Containers
 - Docker Architecture
 - Docker Commands and Images
 - DevOps Lifecycle
 - CI/CD Basics
 - Version Control with Git
-

Lab Experiments

Experiment 1

[Experiment 1](#)

Experiment 2

[Experiment 2](#)

Experiment 3

[Experiment 3](#)

Lab task

[Labtask](#)

Experiment 4

[Experiment 4](#)

Experiment 5

[Experiment 5](#)

Experiment 6

Experiment 6

Experiment 7

Experiment 7

Experiment 9

Experiment 9

Experiment 10

Experiment 10

Experiment 11

Experiment 11

Experiment 12

Experiment 12

Tools and Technologies Used

- Docker
 - Git and GitHub
 - Visual Studio Code
 - Linux Commands
-

[https://pariharnaman.github.io/containerization-devops/
Lab/Experiment1/](https://pariharnaman.github.io/containerization-devops/Lab/Experiment1/)

containerization-devops

Experiment 1: Comparison of Virtual Machines (VMs) and Containers using Ubuntu and Nginx

Objective

- Install Vagrant and run nginx on VM
- Containerize Nginx on docker
- Compare image size

Procedure

Step 1: Installing Vagrant

Installation of Vagrant, using homebrew for mac.

```
namansinghparihar — zsh — 80x28
namansinghparihar@NAMANs-MacBook-Air-3 ~ % brew tap hashicorp/tap

==> Tapping hashicorp/tap
Cloning into '/opt/homebrew/Library/Taps/hashicorp/homebrew-tap'...
remote: Enumerating objects: 6300, done.
remote: Counting objects: 100% (1063/1063), done.
remote: Compressing objects: 100% (309/309), done.
remote: Total 6300 (delta 927), reused 759 (delta 754), pack-reused 5237 (from 2)
Receiving objects: 100% (6300/6300), 1.10 MiB | 5.65 MiB/s, done.
Resolving deltas: 100% (4527/4527), done.
Tapped 2 casks and 32 formulae (99 files, 1.5MB).
namansinghparihar@NAMANs-MacBook-Air-3 ~ % brew install hashicorp/tap/hashicorp-vagrant

==> Downloading https://releases.hashicorp.com/vagrant/2.4.9/vagrant_2.4.9_darwi
##### 100.0%
==> Installing Cask hashicorp-vagrant
==> Running installer for hashicorp-vagrant with `sudo` (which may request your
Password:
installer: Package name is Vagrant
installer: Installing at base path /
installer: The install was successful.
📦 hashicorp-vagrant was successfully installed!
namansinghparihar@NAMANs-MacBook-Air-3 ~ % vagrant --version

Vagrant 2.4.9
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

Step 2: Add Vagrant box image of ubuntu

Initializing ubuntu image over the Vagrant box.

```
vm-lab — curl - vagrant up — 80x28

Installing the 'vagrant_utm' plugin. This can take a few minutes...
Fetching vagrant_utm-0.1.5.gem
Installed the plugin 'vagrant_utm (0.1.5)'!
namansinghparihar@NAMANS-MacBook-Air-3 ~ % mkdir vm-lab
[ cd vm-lab ]
namansinghparihar@NAMANS-MacBook-Air-3 vm-lab % vagrant init utm/ubuntu-24.04
A `Vagrantfile` has been placed in this directory. You are now
ready to `vagrant up` your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
`vagrantup.com` for more information on using Vagrant.
namansinghparihar@NAMANS-MacBook-Air-3 vm-lab % vagrant box add utm/ubuntu-24.04
--provider=utm
==> box: Loading metadata for box 'utm/ubuntu-24.04'
box: URL: https://vagrantcloud.com/api/v2/vagrant/utm/ubuntu-24.04
==> box: Adding box 'utm/ubuntu-24.04' (v0.0.1) for provider: utm (arm64)
box: Downloading: https://vagrantcloud.com/utm/boxes/ubuntu-24.04/versions/0
.0.1/providers/utm/arm64/vagrant.box
==> box: Successfully added box 'utm/ubuntu-24.04' (v0.0.1) for 'utm (arm64)'!
namansinghparihar@NAMANS-MacBook-Air-3 vm-lab % vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'utm/ubuntu-24.04' could not be found. Attempting to find and i
nstall...
default: Box Provider: virtualbox
default: Box Version: >= 0
==> default: Loading metadata for box 'utm/ubuntu-24.04'
default: URL: https://vagrantcloud.com/api/v2/vagrant/utm/ubuntu-24.04
```

Step 3: VM Creation

VM created successfully, “vagrant up --provider=utm” command directly makes VM of the image on UTM


```
vm-lab -- -zsh -- 80x28
Vagrant exited after cleanup due to external interrupt.
[namansinghparihar@NAMANs-MacBook-Air-3 vm-lab % vagrant up --provider=utm
Bringing machine 'default' up with 'utm' provider...
==> default: Importing base box 'utm/ubuntu-24.04'...
==> default: Generating MAC address for NAT networking...
==> default: Checking if box 'utm/ubuntu-24.04' version '0.0.1' is up to date...
==> default: Setting the name of the VM: vm-lab_default_1770433834577_29873
==> default: Forwarding ports...
default: 22 (guest) => 2222 (host) (adapter 1)
==> default: Running 'pre-boot' VM customizations...
==> default: Booting VM...
==> default: Waiting for machine to boot. This may take a few minutes...
default: SSH address: 127.0.0.1:2222
default: SSH username: vagrant
default: SSH auth method: private key
default:
default: Vagrant insecure key detected. Vagrant will automatically replace
default: this with a newly generated keypair for better security.
default:
default: Inserting generated public key within guest...
default: Removing insecure key from the guest if it's present...
default: Key inserted! Disconnecting and reconnecting using new SSH key...
==> default: Machine booted and ready!
==> default: Checking for guest additions in VM...
default: Guest additions detected
==> default: Mounting shared folders...
default: /Users/namansinghparihar/vm-lab => /vagrant
namansinghparihar@NAMANs-MacBook-Air-3 vm-lab %
```

Step 4: Install and verify Nginx

Insatllation of nginx.

```
vm-lab — vagrant@vagrant: ~ — ssh - vagrant ssh — 80x28

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
vagrant@vagrant:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
vagrant@vagrant:~$
```

Step 5: Check Vagrant stats

```
vm-lab — vagrant@vagrant: ~ — ssh - vagrant ssh — 80x13

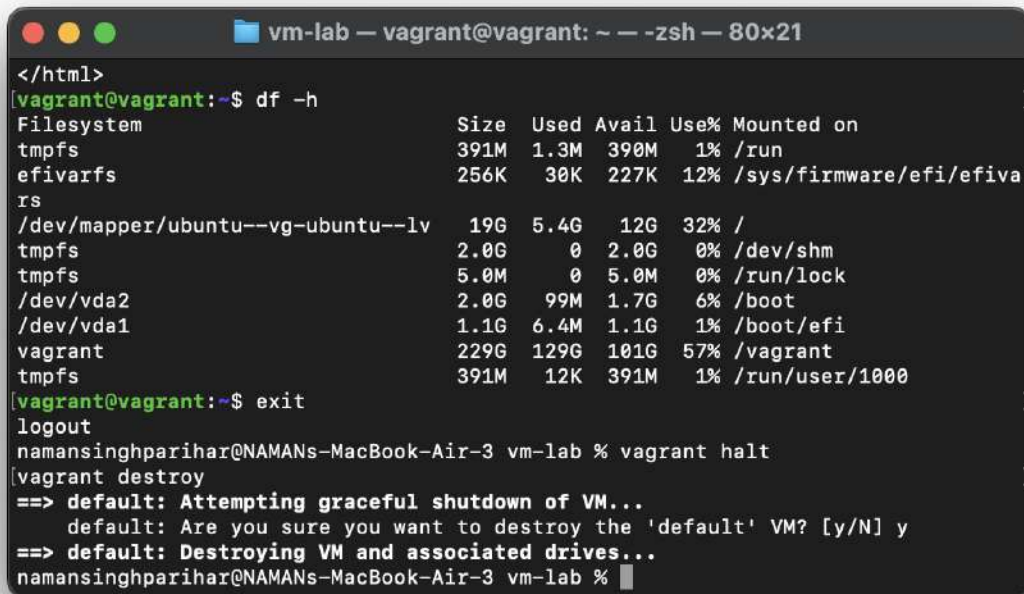
[vagrant@vagrant:~$ df -h]

```

Filesystem	Size	Used	Avail	Use%	Mounted on
tmpfs	391M	1.3M	390M	1%	/run
efivarfs	256K	30K	227K	12%	/sys/firmware/efi/efivars
/dev/mapper/ubuntu--vg-ubuntu--lv	19G	5.4G	12G	32%	/
tmpfs	2.0G	0	2.0G	0%	/dev/shm
tmpfs	5.0M	0	5.0M	0%	/run/lock
/dev/vda2	2.0G	99M	1.7G	6%	/boot
/dev/vda1	1.1G	6.4M	1.1G	1%	/boot/efi
vagrant	229G	129G	101G	57%	/vagrant
tmpfs	391M	12K	391M	1%	/run/user/1000

```
vagrant@vagrant:~$
```

Step 6: Halt and Destroy VM

A terminal window titled "vm-lab — vagrant@vagrant: ~ — zsh — 80x21" showing the process of halting and destroying a VM. The user runs 'df -h' to show disk space usage, then 'exit' to log out. Finally, they run 'vagrant halt' and 'vagrant destroy'. The destroy command prompts for confirmation to destroy the VM and its drives, which is confirmed with 'y'.

```
</html>
[vagrant@vagrant:~$ df -h
Filesystem                Size      Used Avail Use% Mounted on
tmpfs                      391M    1.3M   390M   1% /run
efivarfs                   256K     30K   227K  12% /sys/firmware/efi/efivars
/dev/mapper/ubuntu--vg-ubuntu--lv 19G   5.4G   12G   32% /
tmpfs                      2.0G     0    2.0G   0% /dev/shm
tmpfs                      5.0M     0    5.0M   0% /run/lock
/dev/vda2                  2.0G    99M   1.7G   6% /boot
/dev/vda1                  1.1G    6.4M   1.1G   1% /boot/efi
vagrant                   229G   129G   101G   57% /vagrant
tmpfs                      391M    12K   391M   1% /run/user/1000
[vagrant@vagrant:~$ exit
logout
namansinghparihar@NAMANs-MacBook-Air-3 vm-lab % vagrant halt
[vagrant destroy
==> default: Attempting graceful shutdown of VM...
      default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...
namansinghparihar@NAMANs-MacBook-Air-3 vm-lab %
```

Step 7: Pull Ubuntu Image over Docker and run Nginx

Using pull command we will pull ubuntu and run Nginx over it.

```
vm-lab — vagrant@vagrant: ~ — zsh — 80x27
namansinghparihar@NAMANs-MacBook-Air-3 vm-lab % docker pull ubuntu

docker run -d -p 8080:80 --name nginx-container nginx

Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest
7de22f0203413b3774bb8f7b5e9ac201d1a623d6536345752597fa83aa9f2d07
namansinghparihar@NAMANs-MacBook-Air-3 vm-lab % curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
```

Step 8: Check stats and compare with Vagrant

```
vm-lab — vagrant@vagrant: ~ — docker stats — 80x14
CONTAINER ID   NAME          CPU %      MEM USAGE / LIMIT   MEM %      NET I
/O            BLOCK I/O   PIDS
7de22f020341   nginx-container  0.00%      20.79MiB / 7.653GiB  0.27%      2.33k
B / 1.4kB     13.6MB / 12.3kB    11
```

Conclusion

This experiment shows a clear difference between Vagrant virtual machines and Docker containers when running Nginx.

Vagrant (Ubuntu VM) runs a full operating system, so it uses more disk space and system resources.

Docker (Ubuntu container) shares the host kernel and only runs the required processes, resulting in very low CPU and memory usage.

Because of this architectural difference, the system statistics varied significantly.
Conclusion: Docker is lightweight and resource-efficient for applications like Nginx, while Vagrant is better for full OS-level testing and isolation.

[https://pariharnaman.github.io/containerization-devops/
Lab/Experiment2/](https://pariharnaman.github.io/containerization-devops/Lab/Experiment2/)

containerization-devops

Experiment 2: Docker Image and Container Management

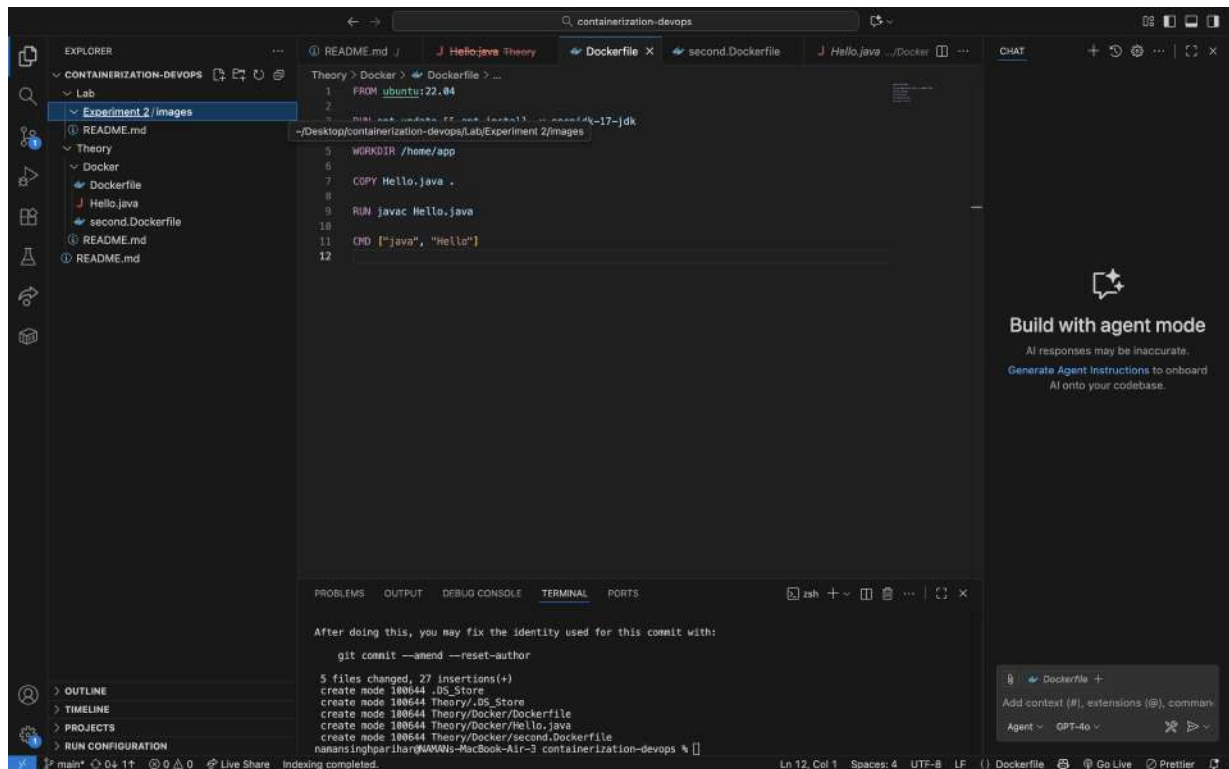
Objective

- Pull Docker images
- Run Docker containers
- Manage Docker container lifecycle

Procedure

Step 1: Create Experiment File

A file was created to document Experiment 2 for the Containerization and DevOps subject.



Step 2: Pull Docker Image

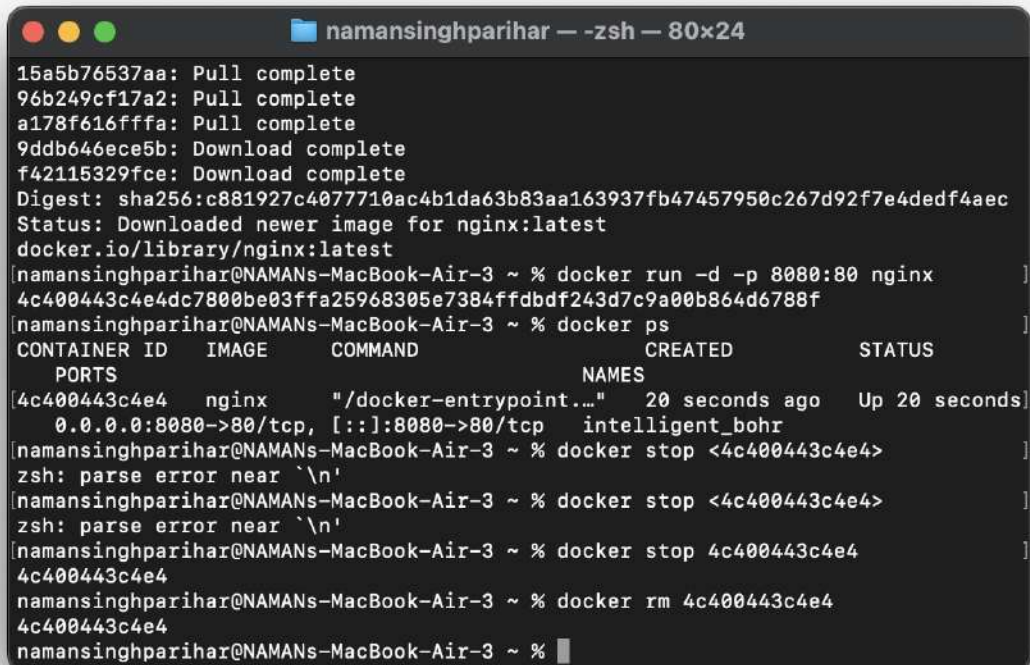
The required Docker image was pulled from Docker Hub to the local system.

```
namansinghparihar — zsh — 80x24

Last login: Fri Jan 30 12:28:27 on ttys002
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
f54f17cce8f7: Pull complete
f43c327f6738: Pull complete
d637807aba98: Pull complete
1d5a66120144: Pull complete
15a5b76537aa: Pull complete
96b249cf17a2: Pull complete
a178f616fffa: Pull complete
9ddb646ece5b: Download complete
f42115329fce: Download complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```


Step 3: Run Docker Container

A Docker container was created and executed from the pulled image with proper port mapping to allow external access.

A terminal window titled 'namansinghparihar — zsh — 80x24' showing a series of Docker commands and their outputs. The terminal text is as follows:

```
15a5b76537aa: Pull complete
96b249cf17a2: Pull complete
a178f616fffa: Pull complete
9ddb646ece5b: Download complete
f42115329fce: Download complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d -p 8080:80 nginx ]
4c400443c4e4dc7800be03ffa25968305e7384ffdbdf243d7c9a00b864d6788f
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker ps ]
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS
NAMES
4c400443c4e4   nginx     "/docker-entrypoint...." 20 seconds ago Up 20 seconds
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   intelligent_bohr
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop <4c400443c4e4> ]
zsh: parse error near `\'
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop <4c400443c4e4> ]
zsh: parse error near `\'
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop 4c400443c4e4 ]
4c400443c4e4
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rm 4c400443c4e4 ]
4c400443c4e4
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % ]
```

Step 4: Verify Running Container

The running container was verified to ensure that it was active and functioning correctly.


```
namansinghparihar — -zsh — 80x24
a178f616fffa: Pull complete
9ddb646ece5b: Download complete
f42115329fce: Download complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d -p 8080:80 nginx
4c400443c4e4dc7800be03ffa25968305e7384ffdbdf243d7c9a00b864d6788f
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS         NAMES
4c400443c4e4   nginx    "/docker-entrypoint..." 20 seconds ago Up 20 seconds
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp intelligent_bohr
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop <4c400443c4e4>
zsh: parse error near `\\n'
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop <4c400443c4e4>
zsh: parse error near `\\n'
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop 4c400443c4e4
4c400443c4e4
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rm 4c400443c4e4
4c400443c4e4
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS         NAMES
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 5: Stop Docker Container

The running container was stopped as part of container lifecycle management.

```
namansinghparihar — -zsh — 80x24
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d -p 8080:80 nginx
4c400443c4e4dc7800be03ffa25968305e7384ffdbdf243d7c9a00b864d6788f
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS         NAMES
4c400443c4e4   nginx     "/docker-entrypoint.    20 seconds ago Up 20 seconds]
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   intelligent_bohr
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop <4c400443c4e4>
zsh: parse error near `\\n'
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop <4c400443c4e4>
zsh: parse error near `\\n'
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop 4c400443c4e4
4c400443c4e4
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rm 4c400443c4e4
4c400443c4e4
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS         NAMES
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 6: Remove Docker Container

The stopped container was removed from the system to free resources.

```
namansinghparihar — -zsh — 80x24
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d -p 8080:80 nginx
4c400443c4e4dc7800be03ffa25968305e7384ffdbdf243d7c9a00b864d6788f
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS
PORTS         NAMES
4c400443c4e4   nginx    "/docker-entrypoint.…"   20 seconds ago Up 20 seconds
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   intelligent_bohr
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop <4c400443c4e4>
zsh: parse error near `\\n'
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop <4c400443c4e4>
zsh: parse error near `\\n'
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop 4c400443c4e4
4c400443c4e4
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rm 4c400443c4e4
4c400443c4e4
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS         NAMES
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Result

Docker images were successfully pulled, containers were executed, verified, stopped, and removed as part of proper container lifecycle management.

Conclusion

This experiment successfully demonstrated Docker image management, container execution, verification, and lifecycle operations, providing hands-on understanding of containerization concepts.

[https://pariharnaman.github.io/containerization-devops/
Lab/Experiment3/](https://pariharnaman.github.io/containerization-devops/Lab/Experiment3/)

containerization-devops

Experiment 3: Comparison of Nginx Servers (Latest, Ubuntu, Alpine)

Objective

- Pull Nginx Docker image
 - Build custom Docker images
 - Run containers from different base images
 - Compare image sizes and efficiency
-

Procedure

Step 1: Pull Nginx Latest Image

The official Nginx latest image was pulled from Docker Hub to the local system.

```
docker pull nginx:latest
```

```
namansinghparihar — zsh — 80x24
Last login: Fri Feb 20 13:55:19 on ttys000
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % sudo docker images
Password:

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nemon28/java-app:1.0	c92aa3032511	1.11GB	329MB	
nginx:latest	6c9ab46b34b8	258MB	64.1MB	U

```

[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker pull nginx:latest
latest: Pulling from library/nginx
58072fb4fd7c: Pull complete
fb8853dab2da: Pull complete
1063666c8b8e: Pull complete
5380ffdd215e: Pull complete
0e9ba714bdf1: Pull complete
efc7865cf13e: Pull complete
ee66459a851f: Download complete
875ddf4f8f5c: Download complete
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 2: Run Nginx Server (Latest)

A container was created and executed from the nginx:latest image with port mapping.

```
docker run -d -p 8080:80 --name nginx-latest nginx:latest
```

```
namansinghparihar — zsh — 103x27
[namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d --name nginx-official -p 8080:80 nginx
c987eaab712f3efcabcf2a530315f2865ef5dcb5209557c49f859feb62d8d68f
[namansinghparihar@NAMANs-MacBook-Air-3 ~ % curl http://localhost:8080
[!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

Step 3: Check Image Statistics for Comparison

The image details were checked to observe the size and other properties for comparison.

```
docker images nginx
```

```
namansinghparihar — zsh — 103x6
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker images nginx

IMAGE          ID              DISK USAGE  CONTENT SIZE  EXTRA
nginx:latest    341bf0f3ce6c    258MB       64.1MB        U
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

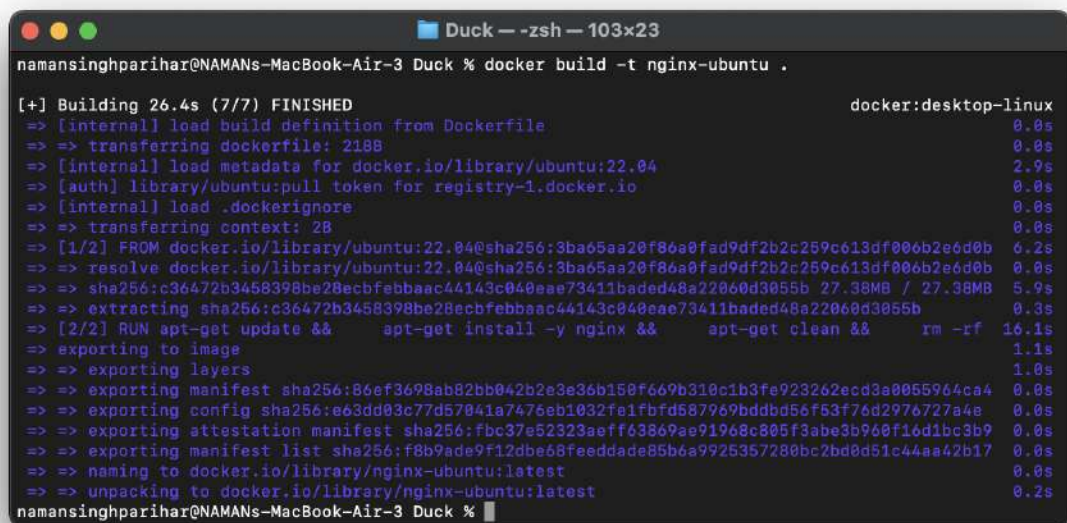
Step 4: Build Dockerfile (Ubuntu-Based Image)

A Dockerfile was created using Ubuntu as the base image and Nginx was installed.


```
FROM ubuntu:latest
RUN apt update && apt install -y nginx
CMD ["nginx", "-g", "daemon off;"]
```

The image was built using:

```
docker build -t nginx-ubuntu .
```

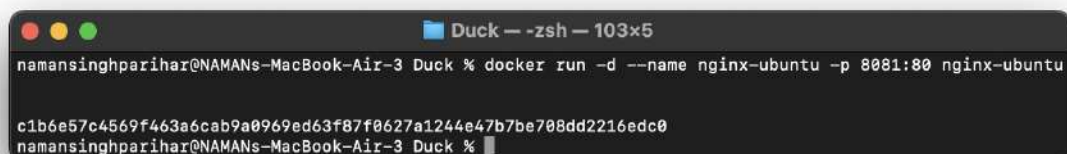
A terminal window titled "Duck --zsh-- 103x23" showing the output of the command "docker build -t nginx-ubuntu .". The output indicates a successful build of the "nginx-ubuntu" image. It shows the process of pulling the "ubuntu:22.04" base image, installing "nginx", and exporting the final image layers. The build completes with the message "[+] Building 26.4s (7/7) FINISHED".

```
namansinghparihar@NAMANs-MacBook-Air-3 Duck % docker build -t nginx-ubuntu .
[+] Building 26.4s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 218B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:22.04    2.9s
=> [auth] library/ubuntu:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 28                                       0.0s
=> [1/2] FROM docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0b 6.2s
=> => resolve docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0b 0.0s
=> => sha256:c36472b3458398be28ecbfebbaac44143c040eae73411bade48a22060d3055b 27.38MB / 27.38MB 5.9s
=> => extracting sha256:c36472b3458398be28ecbfebbaac44143c040eae73411bade48a22060d3055b 0.3s
=> [2/2] RUN apt-get update && apt-get install -y nginx && apt-get clean && rm -rf 16.1s
=> exporting to image                                              1.1s
=> => exporting layers                                              1.0s
=> => exporting manifest sha256:86ef3698ab82bb042b2e3e36b150f669b310c1b3fe923262ecd3a0055964ca4 0.0s
=> => exporting config sha256:e63dd03c77d57041a7476eb1032fe1fbfd587969bddbd56f53f76d2976727a4e 0.0s
=> => exporting attestation manifest sha256:fb37e52323a0ff63869ae91968c805f3abe3b960f16d1bc3b9 0.0s
=> => exporting manifest list sha256:f8b9ade9f12dbe68feeddade85b6a9925357280bc2bd0d51c44aa42b17 0.0s
=> => naming to docker.io/library/nginx-ubuntu:latest             0.0s
=> => unpacking to docker.io/library/nginx-ubuntu:latest          0.2s
namansinghparihar@NAMANs-MacBook-Air-3 Duck %
```

Step 5: Run Ubuntu-Based Nginx Container

The container was executed from the newly built nginx-ubuntu image.

```
docker run -d -p 8081:80 --name nginx-ubuntu-container nginx-ubuntu
```

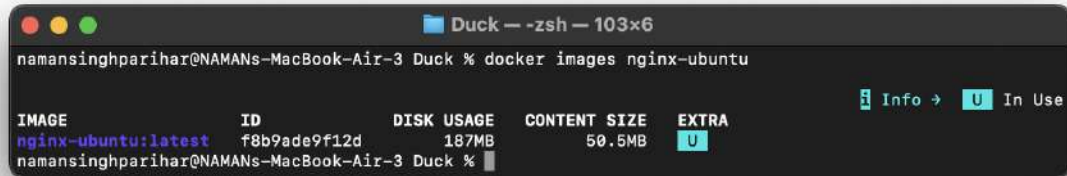
A terminal window titled "Duck --zsh-- 103x5" showing the output of the command "docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu". The output shows the container ID "c1b6e57c4569f463acab9a0969ed63f87f0627a1244e47b7be708dd2216edc0" and confirms the container is running.

```
namansinghparihar@NAMANs-MacBook-Air-3 Duck % docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
c1b6e57c4569f463acab9a0969ed63f87f0627a1244e47b7be708dd2216edc0
namansinghparihar@NAMANs-MacBook-Air-3 Duck %
```

Step 6: Observe Image Details (nginx-ubuntu)

The image size and details were observed for comparison.

```
docker images nginx-ubuntu
```



Step 7: Build Dockerfile (Alpine-Based Image)

A new Dockerfile was created using Alpine as the base image and Nginx was installed.

```
FROM alpine:latest
RUN apk add --no-cache nginx
CMD ["nginx", "-g", "daemon off;"]
```

The image was built using:

```
docker build -t nginx-alpine .
```



```
new --zsh -- 113x43
[namansinghparihar@NAMANS-MacBook-Air-3 new % docker build -t nginx-alpine .

[+] Building 5.5s (7/7) FINISHED
=> [internal] load build definition from Dockerfile                                docker:desktop-linux
=> => transferring dockerfile: 133B                                              0.0s
=> [internal] load metadata for docker.io/library/alpine:latest                  0.0s
=> [auth] library/alpine:pull token for registry-1.docker.io                    2.8s
=> [internal] load .dockerignore                                                 0.0s
=> => transferring context: 2B                                                  0.0s
=> [1/2] FROM docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659 0.0s
=> => resolve docker.io/library/alpine:latest@sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659 1.2s
=> => sha256:d8ad8cd7260f46cc068e16c39046ebc76526e41051f43a8c249884b200936c0 4.20MB / 4.20MB 0.0s
=> => extracting sha256:d8ad8cd7260f46cc068e16c39046ebc76526e41051f43a8c249884b200936c0 1.1s
=> [2/2] RUN apk add --no-cache nginx                                           0.1s
=> exporting to image                                                           1.3s
=> => exporting layers                                                         0.1s
=> => exporting manifest sha256:04f39de24da5db8b698422118664cc1c7580fd7f5392519ee225fbc44df94dd0 0.1s
=> => exporting config sha256:e3e0a67430d2e2032013e6b1b8938d11b91fae5b42bf5068ed2a05049d592049 0.0s
=> => exporting attestation manifest sha256:ccf2672edcd7967d0fd4982ffd7f1aeeb5f07679b23d22cber588972275fca0 0.0s
=> => exporting manifest list sha256:2d64c819dd75b70f5fc458148e8a4453cfee134ac73a497b20a6258ae124a267 0.0s
=> => naming to docker.io/library/nginx-alpine:latest                         0.0s
=> => unpacking to docker.io/library/nginx-alpine:latest                      0.0s
namansinghparihar@NAMANS-MacBook-Air-3 new %
```

Step 8: Run Alpine-Based Nginx Container

The container was executed from the nginx-alpine image.

```
docker run -d -p 8082:80 --name nginx-alpine-container nginx-alpine
```

```
new --zsh -- 113x5
namansinghparihar@NAMANS-MacBook-Air-3 new % docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
94c07e80b4b2e98811474706bf97c4f0b9c667c9fb758c90bd00e9a50476dd33
namansinghparihar@NAMANS-MacBook-Air-3 new %
```

Step 9: Observe Image Details (nginx-alpine)

The image size and details were checked for final comparison.

```
docker images nginx-alpine
```

Observation

- The nginx:latest image had moderate size.
- The nginx-ubuntu image was larger because it includes the full Ubuntu base OS.
- The nginx-alpine image was the smallest and most lightweight.

Result

Different Nginx Docker images were successfully built, executed, and compared based on their base image and size.

Conclusion

This experiment demonstrated the impact of base images on Docker image size and efficiency.

The Alpine-based image was the most lightweight and suitable for optimized deployments, while Ubuntu-based images were comparatively larger due to additional system packages.

[https://pariharnaman.github.io/containerization-devops/
Lab/Experiment4/](https://pariharnaman.github.io/containerization-devops/Lab/Experiment4/)

containerization-devops

Experiment 4: Docker Essentials

Task 1: Single-Stage Build of Flask Application

Objective

To containerize a Python Flask application using Docker with a **single-stage build**, and perform image and container management operations.

Technologies Used

- Python 3
 - Flask
 - Docker
-

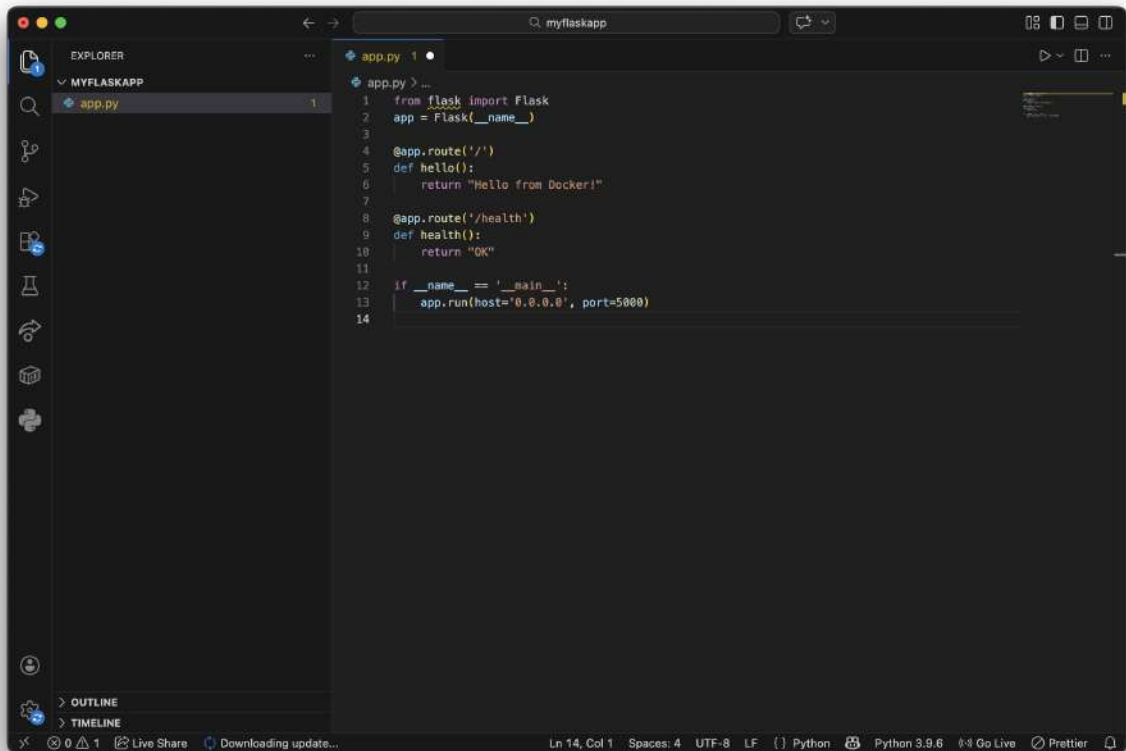
Files Created

1. **app.py** – Flask application
 2. **requirements.txt** – Dependencies
 3. **Dockerfile** – Instructions to build Docker image
 4. **.dockerignore** – Excluded unnecessary files
-

Steps Performed

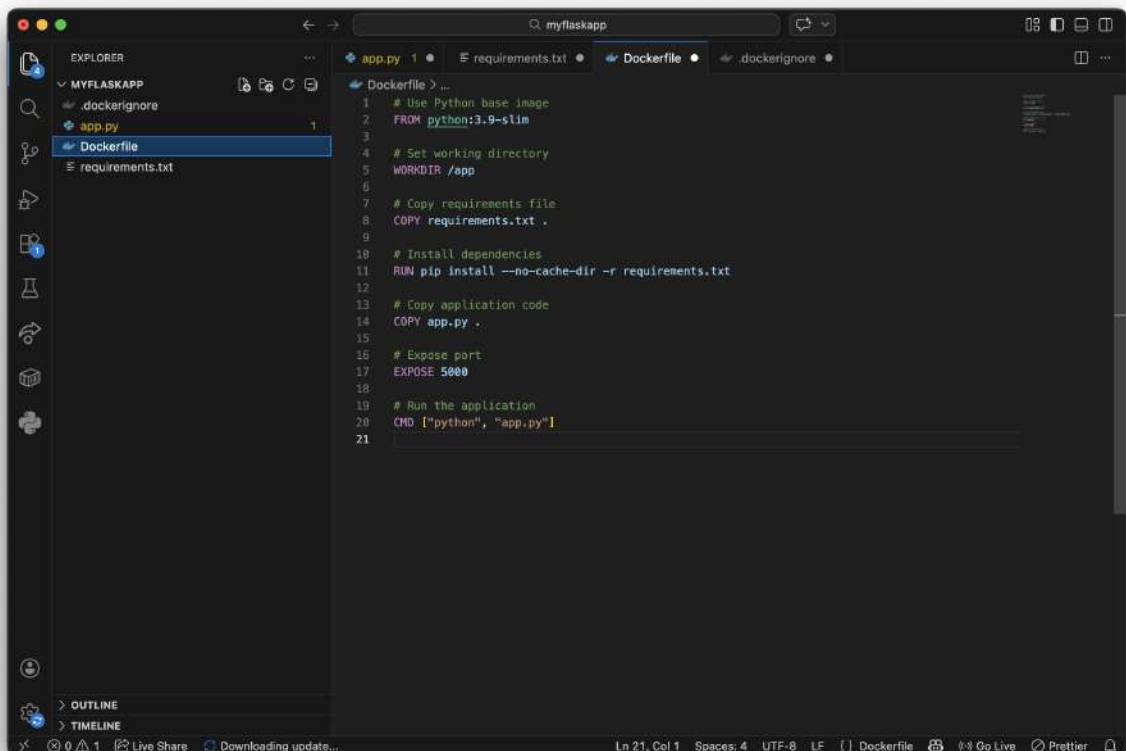
1: Created Flask Application (app.py)

A simple Flask app was created to display a message on the browser.



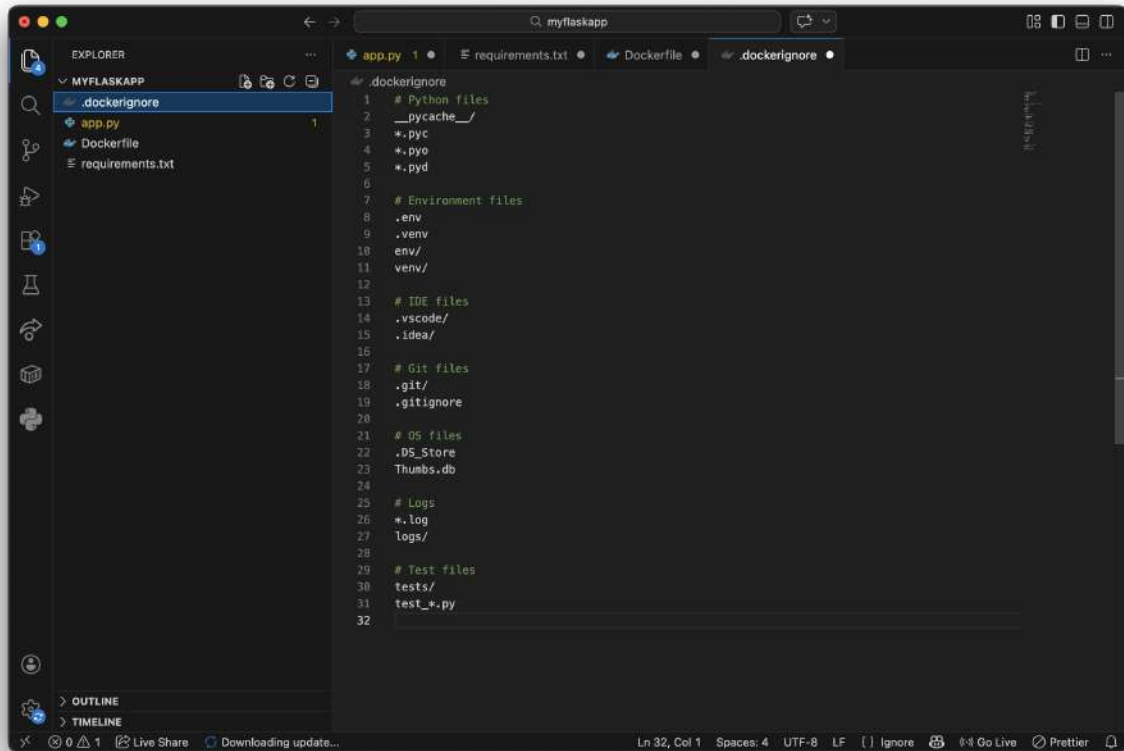
2. Created Dockerfile

Added Flask dependency required to run the application.



3. Created .dockerignore

Ignored unnecessary files such as cache, environment files, and Git data to reduce image size.



4. Built Docker Image

```
docker build -t my-flask-app:latest .
```

```
myflaskapp — -zsh — 80x24
[namansinghparihar@NAMANs-MacBook-Air-3 ~ % cd desktop ]
[namansinghparihar@NAMANs-MacBook-Air-3 desktop % cd myflaskapp ]
[namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker build -t my-flask-app]
.
[+] Building 16.0s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 361B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 3.6s
=> [auth] library/python:pull token for registry-1.docker.io      0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 273B                                     0.0s
=> [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338 7.2s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338 0.0s
=> => sha256:ebfe7d4fa0c501a81a5ba6d1e1e2958e4b005d3ce 13.83MB / 13.83MB 3.1s
=> => sha256:c23f4b50347300e01a1a1da6dd0266adcf8e44671002ad2 251B / 251B 0.6s
=> => sha256:479b8ad8bcc3edbeba82d4959dfbdc65226d5b55df3 1.27MB / 1.27MB 1.9s
=> => sha256:a16e551192670581ec8359c70ab9eebf8f2af5468 30.14MB / 30.14MB 6.6s
=> => extracting sha256:a16e551192670581ec8359c70ab9eebf8f2af5468ffc79b3 0.3s
=> => extracting sha256:479b8ad8bcc3edbeba82d4959dfbdc65226d5b55df3f3691 0.0s
=> => extracting sha256:ebfe7d4fa0c501a81a5ba6d1e1e2958e4b005d3ce3827b0a 0.1s
=> => extracting sha256:c23f4b50347300e01a1a1da6dd0266adcf8e44671002ad28 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 321B                                     0.0s
=> [2/5] WORKDIR /app                                           0.1s
```

5. Tagged Docker Image

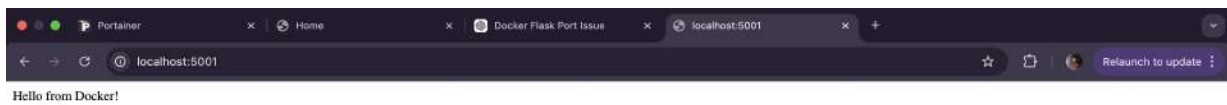
```
myflaskapp — -zsh — 80x10
[namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker images]

```

IMAGE	ID	DISK USAGE	Info → CONTENT SIZE	U In Use	EXTRA
ghcr.io/k3d-io/k3d-proxy:5.8.3	d43e6fffd27c2	92.6MB	23.9MB		
ghcr.io/k3d-io/k3d-tools:5.8.3	c5ceb2b9e8ff	35MB	10.5MB		
my-flask-app:1.0	f2b3634b297c	226MB	49.1MB		
my-flask-app:latest	f2b3634b297c	226MB	49.1MB		
my-flask-app:v1.0	f2b3634b297c	226MB	49.1MB		

6. Running Container over local host

```
myflaskapp -- zsh -- 117x14
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % curl http://localhost:5000
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker run --name flask-container -p 5001:5000 my-flask-app
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.3:5000
Press CTRL+C to quit
192.168.65.1 - - [27/Mar/2026 17:23:35] "GET / HTTP/1.1" 200 -
192.168.65.1 - - [27/Mar/2026 17:23:35] "GET /favicon.ico HTTP/1.1" 404 -
^C
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp %
```



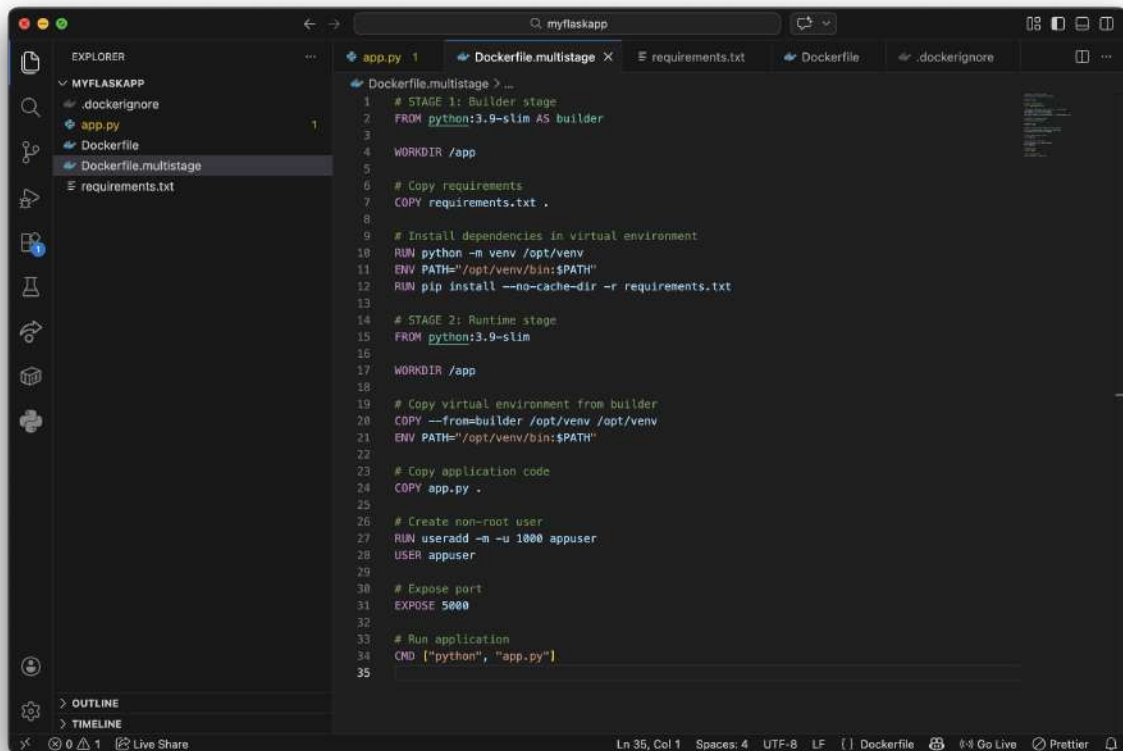
7. Removing Flask Container

```
myflaskapp -- zsh -- 98x5
^C
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker rm -f flask-container
flask-container
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp %
```


Task 2: Multi-stage Build

Steps Performed

1. Creating Docker.multistage file



```
1 # STAGE 1: Builder stage
2 FROM python:3.9-slim AS builder
3
4 WORKDIR /app
5
6 # Copy requirements
7 COPY requirements.txt .
8
9 # Install dependencies in virtual environment
10 RUN python -m venv /opt/venv
11 ENV PATH="/opt/venv/bin:$PATH"
12 RUN pip install --no-cache-dir -r requirements.txt
13
14 # STAGE 2: Runtime stage
15 FROM python:3.9-slim
16
17 WORKDIR /app
18
19 # Copy virtual environment from builder
20 COPY --from=builder /opt/venv /opt/venv
21 ENV PATH="/opt/venv/bin:$PATH"
22
23 # Copy application code
24 COPY app.py .
25
26 # Create non-root user
27 RUN useradd -m -u 1000 appuser
28 USER appuser
29
30 # Expose port
31 EXPOSE 5000
32
33 # Run application
34 CMD ["python", "app.py"]
35
```

2. Performing Multi-stage Build

```
myflaskapp -- -zsh -- 98x25
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker build -f Dockerfile.multistage -t flask-multistage .
[+] Building 7.5s (13/13) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile.multistage    0.0s
=> => transferring dockerfile: 675B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 1.5s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 273B                                    0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 63B                                      0.0s
=> [builder 1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f14496 0.0s
=> CACHED [builder 2/5] WORKDIR /app                             0.0s
=> CACHED [builder 3/5] COPY requirements.txt .                   0.0s
=> [builder 4/5] RUN python -m venv /opt/venv                     1.8s
=> [builder 5/5] RUN pip install --no-cache-dir -r requirements.txt 3.3s
=> [stage-1 3/5] COPY --from=builder /opt/venv /opt/venv         0.1s
=> [stage-1 4/5] COPY app.py .                                    0.0s
=> [stage-1 5/5] RUN useradd -m -u 1000 appuser                   0.1s
=> exporting to image                                             0.4s
=> => exporting layers                                           0.5s
=> => exporting manifest sha256:5715f9106ecd20e4cf95deec361ae0a5de2038ade57c54ac9ecb57e411 0.0s
=> => exporting config sha256:99a9615bbc935172051a79ad10c5f0d832cc99e3d829efaecaafccc927862 0.0s
=> => exporting attestation manifest sha256:2c4c680feebfb21483d1d83deff663c0091c3b6fdfe259 0.0s
=> => exporting manifest list sha256:e1268cd813c3dd7b65397c41ccce31a1ea58303699288e17dec1b 0.0s
```

3. Comparing Build sizes

```
myflaskapp -- -zsh -- 98x10
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker images | grep flask-
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
flask-multistage:latest      e1268cd813c3      245MB      52.4MB
flask-regular:latest        2532b44d0968      226MB      49.1MB
my-flask-app:1.0            f2b3634b297c      226MB      49.1MB
my-flask-app:latest         f2b3634b297c      226MB      49.1MB
my-flask-app:v1.0           f2b3634b297c      226MB      49.1MB
username/my-flask-app:1.0   93d2b3388461      226MB      49.1MB
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp %
```

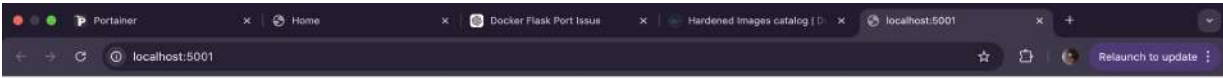
4. Pushing tagged images to Docker Hub

```
myflaskapp --zsh -- 98x31
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker tag my-flask-app:latest username/my-flask-app:1.0
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker tag my-flask-app:latest username/my-flask-app:latest
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker push username/my-flask-app:1.0
The push refers to repository [docker.io/username/my-flask-app]
a16e55119267: Waiting
3e88322d1393: Waiting
f16b7da44720: Waiting
479b8ad8bcc3: Waiting
ebfe7d4fa0c5: Waiting
03f3aaacd142: Waiting
c23f4b503473: Waiting
6a28e962801a: Waiting
bc0482d9370a: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker push username/my-flask-app:latest
The push refers to repository [docker.io/username/my-flask-app]
c23f4b503473: Waiting
479b8ad8bcc3: Waiting
bc0482d9370a: Waiting
3e88322d1393: Waiting
a16e55119267: Waiting
03f3aaacd142: Waiting
f16b7da44720: Waiting
ebfe7d4fa0c5: Waiting
6a28e962801a: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp %
```

5. Pull and run docker image (my-flask-app)

```
myflaskapp --zsh -- 132x7
Run 'docker run --help' for more information
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp % docker run -d -p 5001:5000 username/my-flask-app:latest
f022a49ae4f16c42622e7363abbcfd7d2310137b90319877c5e6638d1e04c744
namansinghparihar@NAMANs-MacBook-Air-3 myflaskapp %
```

Localhost:5001

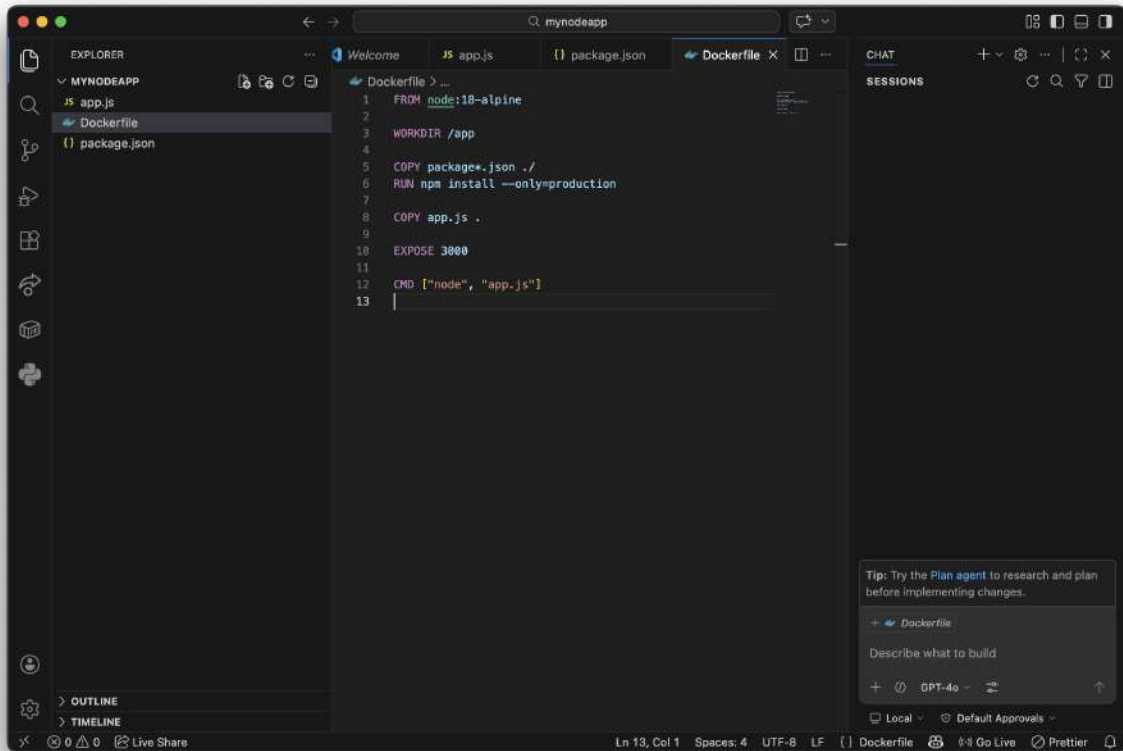


Hello from Docker!

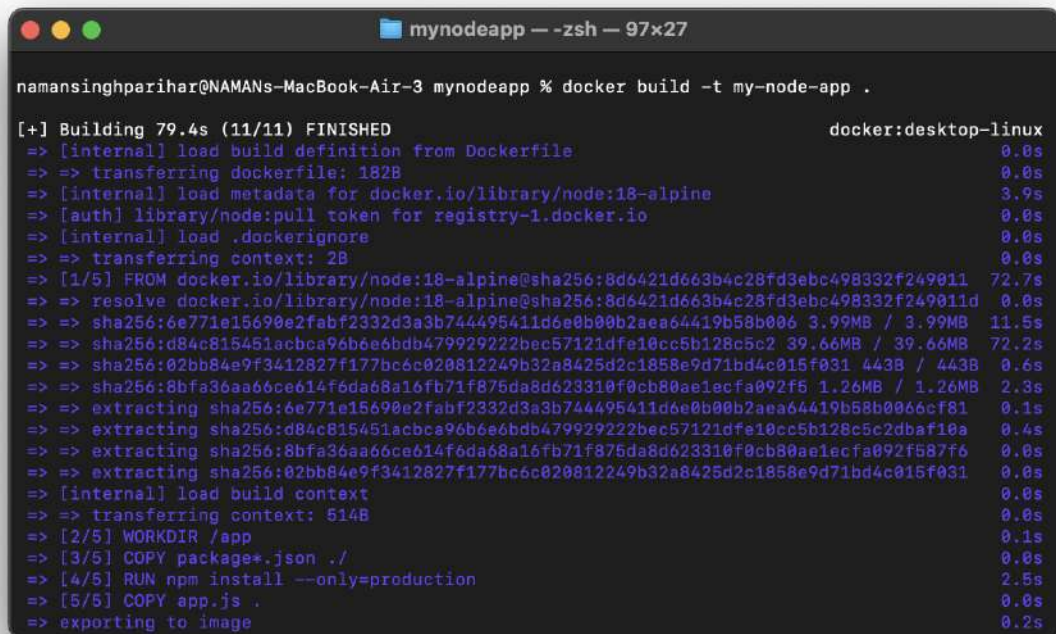
Task 3: Node.js

Steps Performed

1. Create Dockerfile, App.js ,and json package file



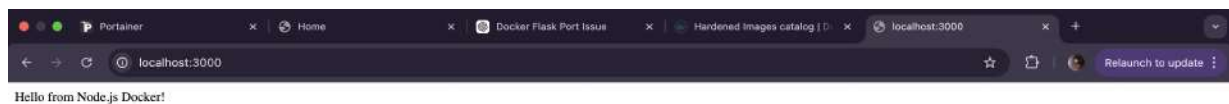
2. Build docker image and Run docker container

A terminal window titled 'mynodeapp --zsh -- 97x27' showing the output of a 'docker build -t my-node-app .' command. The output shows the build process for a Docker image, including downloading the Dockerfile, resolving the base image (node:18-alpine), and copying files. The build is completed successfully.

```
namansinghparihar@NAMANs-MacBook-Air-3 mynodeapp % docker build -t my-node-app .

[+] Building 79.4s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 182B                                0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine  3.9s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011 72.7s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d 0.0s
=> => sha256:6e771e15690e2fabf2332d3a3b744495411d6e0b00b2aea64419b58b006 3.99MB / 3.99MB 11.5s
=> => sha256:d84c815451acbc96b6e6bdb479929222bec57121dfe10cc5b128c5c2 39.66MB / 39.66MB 72.2s
=> => sha256:02bb84e9f3412827f177bc6c020812249b32a8425d2c1858e9d71bd4c015f031 443B / 443B 0.6s
=> => sha256:8bfa36aa66ce614f6da68a16fb71f875da8d623310f0cb80ae1ecfa092f5 1.26MB / 1.26MB 2.3s
=> => extracting sha256:6e771e15690e2fabf2332d3a3b744495411d6e0b00b2aea64419b58b0066cf81 0.1s
=> => extracting sha256:d84c815451acbc96b6e6bdb479929222bec57121dfe10cc5b128c5c2dbaf10a 0.4s
=> => extracting sha256:8bfa36aa66ce614f6da68a16fb71f875da8d623310f0cb80ae1ecfa092f587f6 0.0s
=> => extracting sha256:02bb84e9f3412827f177bc6c020812249b32a8425d2c1858e9d71bd4c015f031 0.0s
=> [internal] load build context                                   0.0s
=> => transferring context: 514B                                       0.0s
=> [2/5] WORKDIR /app                                              0.1s
=> [3/5] COPY package*.json ./                                     0.0s
=> [4/5] RUN npm install --only=production                        2.5s
=> [5/5] COPY app.js .                                             0.0s
=> exporting to image                                              0.2s
```

3. Check localhost



[https://pariharnaman.github.io/containerization-devops/
Lab/Experiment5/](https://pariharnaman.github.io/containerization-devops/Lab/Experiment5/)

containerization-devops

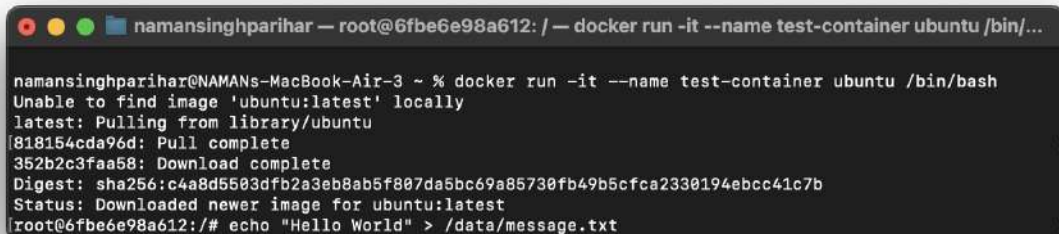
Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

Step 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

- Data inside a container is temporary
- When the container stops or is removed, data is lost
- Containers do not store data permanently

➡ We need a way to persist data outside the container



```
namansinghparihar — root@6fbc6e98a612: / — docker run -it --name test-container ubuntu /bin/...  
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -it --name test-container ubuntu /bin/bash  
Unable to find image 'ubuntu:latest' locally  
latest: Pulling from library/ubuntu  
818154cda96d: Pull complete  
352b2c3faa58: Download complete  
Digest: sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b  
Status: Downloaded newer image for ubuntu:latest  
root@6fbc6e98a612:/# echo "Hello World" > /data/message.txt
```



```
exit
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop test-container
test-container
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker start test-container
test-container
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker exec test-container cat /data/message.txt
Hello World
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop test-container
test-container
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rm test-container
test-container
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker exec test-container cat /data/message.txt
Error response from daemon: No such container: test-container
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 2: Volume Types

Docker provides different types of volumes for data persistence:

a) Named Volumes

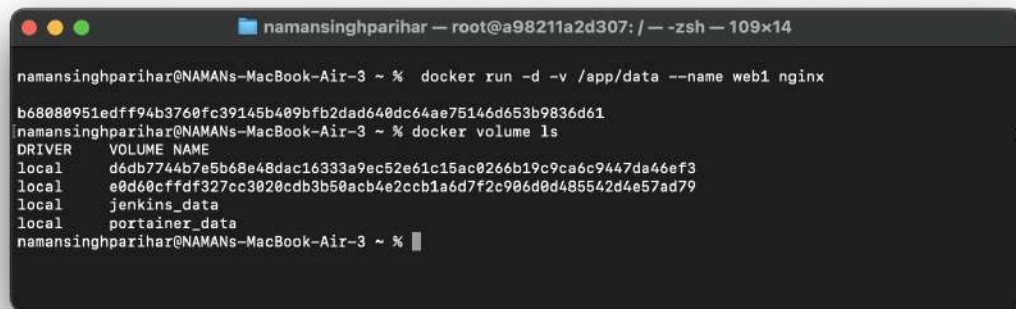
- Managed by Docker
- Easy to use and reusable
- Stored in Docker's default location

```
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker volume create mydata
mydata
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d -v mydata:/app/data --name web2 nginx
1fced9d8a8cb44f3c71df5403966359612b6bc90fee7857120676de63afc16a5
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker volume ls
DRIVER      VOLUME NAME
local       d6db7744b7e5b68e48dac16333a9ec52e61c15ac0266b19c9ca6c9447da46ef3
local       e0d60cfffdf327cc3020cdb3b50acb4e2ccb1a6d7f2c906d0d485542d4e57ad79
local       jenkins_data
local       mydata
local       portainer_data
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

b) Anonymous Volumes

- Created automatically
- No specific name assigned

- Hard to manage and reuse

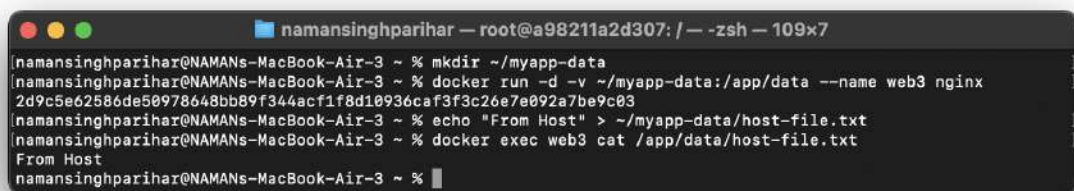


```
namansinghparihar — root@a98211a2d307: / — zsh — 109x14

namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d -v /app/data --name web1 nginx
b68080951edff94b3760fc39145b409bfb2dad640dc64ae75146d653b9836d61
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker volume ls
DRIVER      VOLUME NAME
local       d6db7744b7e5b68e48dac16333a9ec52e61c15ac0266b19c9ca6c9447da46ef3
local       e0d60cffdf327cc3020cdb3b50acb4e2ccb1a6d7f2c906d0d485542d4e57ad79
local       jenkins_data
local       portainer_data
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

c) Bind Mounts

- Links a container to a specific path on the host
- Full control over files
- Useful for development



```
namansinghparihar — root@a98211a2d307: / — zsh — 109x7

namansinghparihar@NAMANs-MacBook-Air-3 ~ % mkdir ~/myapp-data
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d -v ~/myapp-data:/app/data --name web3 nginx
2d9c5e62586de50978648bb89f344acf1f8d10936caf3f3c26e7e092a7be9c03
namansinghparihar@NAMANs-MacBook-Air-3 ~ % echo "From Host" > ~/myapp-data/host-file.txt
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker exec web3 cat /app/data/host-file.txt
From Host
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

Step 3: Practical Volume Examples

Example 1: Database with Persistent Storage

- Use volumes to store database data
- Data remains safe even if the container is removed

```
namansinghparihar — root@a98211a2d307: / — zsh — 109x24
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
9db918e56439: Pull complete
1fc9ca300ffc: Pull complete
57625cf440a6: Pull complete
2ab1b3740ab5: Pull complete
0b96495ec72e: Pull complete
e896781c011d: Pull complete
5faa84208659: Pull complete
44694e56b891: Pull complete
7f772b69f429: Pull complete
810c9f27d822: Pull complete
a1a79c866863: Pull complete
1216a0cbaf7e: Download complete
3711edbe1f5b: Download complete
Digest: sha256:529b4afa138686871cf698617ef6e347a857179fa2c58e6811b6c20862a1cd6a
Status: Downloaded newer image for mysql:8.0
0b6f541016c251e2ada490f1577110e0bea1e98cf6b7cc412e652ec5c18659b7
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

```
namansinghparihar — root@a98211a2d307: / — zsh — 109x11
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop mysql-db
mysql-db
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rm mysql-db
mysql-db
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
b4a5c1100ff05056108f59c6b5528f5b7e4223320f0b95e34427e3929ae7e7a9
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Example 2: Web App with Configuration Files

- Use bind mounts to manage config files
- Easily update configurations without rebuilding the container

```
namansinghparihar — root@a98211a2d307: / — zsh — 109x9
namansinghparihar@NAMANS-MacBook-Air-3 ~ % mkdir ~/nginx-config
namansinghparihar@NAMANS-MacBook-Air-3 ~ % echo 'server {
  listen 80;
  server_name localhost;
  location / {
    return 200 "Hello from mounted config!";
  }
}' > ~/nginx-config/nginx.conf
```

```
namansinghparihar — root@a98211a2d307: / — zsh — 109x9
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d \
--name nginx-custom \
-p 3000:80 \
-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx
2f54a59613fa02f924ec26762f368f7cb8199fd7fd6766178d11d946bf88e5e
namansinghparihar@NAMANS-MacBook-Air-3 ~ % curl http://localhost:3000
Hello from mounted config!
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 4: Setting Environment Variables in Application

- Used to pass configuration into containers
- Helps manage sensitive data (e.g., passwords, API keys)
- Keeps applications flexible and reusable

```
namansinghparihar — root@a98211a2d307: / — zsh — 87x22
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker exec flask-app env
[docker exec flask-app printenv DATABASE_HOST
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=aed1d56d05df
DATABASE_HOST=prod-db.example.com
DEBUG=true
PORT=2000
LANG=C.UTF-8
GPG_KEY=E3FF2839C048B25C084DEBE9B26995E310250568
PYTHON_VERSION=3.9.25
PYTHON_SHA256=00e07d7c0f2f0cc002432d1ee84d2a40dae404a99303e3f97701c10966c91834
PYTHONUNBUFFERED=1
PYTHONDONTWRITEBYTECODE=1
HOME=/root
prod-db.example.com
namansinghparihar@NAMANS-MacBook-Air-3 ~ % curl http://localhost:2341/config
{
  "db_host": "prod-db.example.com",
  "debug": true,
  "has_api_key": false
}
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

```
namansinghparihar — root@a98211a2d307: / — zsh — 87x12
[namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d --name flask-app -p 2341:2000 ]
-e DATABASE_HOST="prod-db.example.com" -e DEBUG="true" -e PORT="2000" flask-app
aed1d56d05dfbdafe6ed29158920d6a2912e93916f19283ba9a70f97c953e5954
[namansinghparihar@NAMANs-MacBook-Air-3 ~ % http://localhost:2341/config ]
zsh: no such file or directory: http://localhost:2341/config
[namansinghparihar@NAMANs-MacBook-Air-3 ~ % curl http://localhost:2341/config ]
{
  "db_host": "prod-db.example.com",
  "debug": true,
  "has_api_key": false
}
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

Step 5: Docker Monitoring

Basic Monitoring Commands

a) `docker stats`

- Shows real-time CPU, memory, and network usage

```
namansinghparihar — root@a98211a2d307: / — zsh — 87x22
[CONTAINER ID   NAME          CPU %       MEM USAGE / LIMIT   MEM %      NET I/O]
[  BLOCK I/O      PIDS
aed1d56d05df   flask-app    0.22%      45.45MiB / 7.653GiB  0.58%      2.19kB / 1.46
kB      856kB / 0B      3
2f54a59613fa   nginx-custom 0.00%      8.867MiB / 7.653GiB  0.11%      1.9kB / 670B
0B / 4.1kB    11
b4a5c1100ff0   new-mysql    0.41%      366.3MiB / 7.653GiB  4.67%      1.54kB / 126B
65.5kB / 15.1MB 37
2d9c5e62586d   web3         0.00%      8.883MiB / 7.653GiB  0.11%      1.8kB / 126B
69.6kB / 12.3kB 11
1fced9d8a8cb   web2         0.00%      8.785MiB / 7.653GiB  0.11%      1.92kB / 126B
0B / 12.3kB    11
b68080951edf   web1         0.00%      8.871MiB / 7.653GiB  0.11%      2.05kB / 126B
0B / 12.3kB    11
ec71e10eb23e   jenkins-server 0.16%      767.5MiB / 7.653GiB  9.79%      71.3MB / 6.5M
B      0B / 212MB      52
e52359ff4cd8   portainer    0.04%      82.51MiB / 7.653GiB  1.05%      6.26kB / 126B
62.8MB / 13.4MB 12

^C
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

```
namansinghparihar — root@a98211a2d307: / — zsh — 87x22
[CONTAINER ID   NAME     CPU %     MEM USAGE / LIMIT     MEM %     NET I/O     BLOCK I/O
b68080951edf   web1     0.00%     8.871MiB / 7.653GiB     0.11%     2.05kB / 126B  0B / 12.3kB
1fced9d8a8cb   web2     0.00%     8.785MiB / 7.653GiB     0.11%     1.92kB / 126B  0B / 12.3kB

^C%
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

- Specific format output

```
namansinghparihar — root@a98211a2d307: / — zsh — 87x22
NAME           CPU %     MEM USAGE / LIMIT     NET I/O
flask-app      0.25%     45.45MiB / 7.653GiB   2.19kB / 1.46kB
nginx-custom   0.00%     8.867MiB / 7.653GiB   1.9kB / 670B
new-mysql      0.37%     366.3MiB / 7.653GiB   1.54kB / 126B
web3           0.00%     8.883MiB / 7.653GiB   1.8kB / 126B
web2           0.00%     8.785MiB / 7.653GiB   1.92kB / 126B
web1           0.00%     8.871MiB / 7.653GiB   2.05kB / 126B
jenkins-server 0.18%     767.8MiB / 7.653GiB   71.3MB / 6.5MB
portainer      0.05%     82.51MiB / 7.653GiB   6.26kB / 126B

^C%
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

- No-stream (single snapshot)

```

namansinghparihar — root@a98211a2d307: / — zsh — 87x22
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker stats --no-stream
CONTAINER ID   NAME          CPU %     MEM USAGE / LIMIT   MEM %     NET I/O
a6d1d56d05df   flask-app     0.28%    45.45MiB / 7.653GiB  0.58%    2.19kB / 1.46
kB 856kB / 0B    3
2f54a59613fa   nginx-custom  0.00%    8.867MiB / 7.653GiB  0.11%    1.9kB / 670B
0B / 4.1kB    11
b4a5c1100ff0   new-mysql     0.38%    366.3MiB / 7.653GiB  4.67%    1.54kB / 126B
65.5kB / 15.1MB 37
2d9c5e62586d   web3          0.00%    8.883MiB / 7.653GiB  0.11%    1.8kB / 126B
69.6kB / 12.3kB 11
1fced9d8a8cb   web2          0.00%    8.785MiB / 7.653GiB  0.11%    1.92kB / 126B
0B / 12.3kB    11
b68080951edf   web1          0.00%    8.871MiB / 7.653GiB  0.11%    2.05kB / 126B
0B / 12.3kB    11
ec71e10eb23e   jenkins-server 0.12%    767.5MiB / 7.653GiB  9.79%    71.3MB / 6.5M
B 0B / 212MB    52
e52359ff4cd8   portainer     0.00%    82.92MiB / 7.653GiB  1.06%    6.26kB / 126B
62.8MB / 13.5MB 12
namansinghparihar@NAMANs-MacBook-Air-3 ~ %

```

- container name format

```

namansinghparihar — root@a98211a2d307: / — docker stats --format Container:...
Container: flask-app | CPU: 0.27% | Memory: 0.58%
Container: nginx-custom | CPU: 0.00% | Memory: 0.11%
Container: new-mysql | CPU: 0.33% | Memory: 4.67%
Container: web3 | CPU: 0.00% | Memory: 0.11%
Container: web2 | CPU: 0.00% | Memory: 0.11%
Container: web1 | CPU: 0.00% | Memory: 0.11%
Container: jenkins-server | CPU: 0.12% | Memory: 9.79%
Container: portainer | CPU: 0.00% | Memory: 1.05%

```


- JSON output format

```

namansinghparihar — root@a98211a2d307: / — zsh — 86x26

namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stats --format json --no-stream
{"BlockIO":"856kB / 0B","CPUPerc":"0.26%","Container":"aed1d56d05dfbda6ed29158920d6a2912e93916f19283ba9a70f97c953e5954","ID":"aed1d56d05df","MemPerc":"0.58%","MemUsage":"45.45MiB / 7.653GiB","Name":"flask-app","NetIO":"2.19kB / 1.46kB","PIDs":"3"}
{"BlockIO":"0B / 4.1kB","CPUPerc":"0.00%","Container":"2f54a59613fa02f924ec26762f368f7cb8199fd7fd6766178d111d946bf88e5e","ID":"2f54a59613fa","MemPerc":"0.11%","MemUsage":"8.867MiB / 7.653GiB","Name":"nginx-custom","NetIO":"1.9kB / 670B","PIDs":"11"}
{"BlockIO":"65.5kB / 15.1MB","CPUPerc":"0.37%","Container":"b4a5c1100ff05056108f59c6b5528f5b7e4223320f0b95e34427e3929ae7e7a9","ID":"b4a5c1100ff0","MemPerc":"4.67%","MemUsage":"366.3MiB / 7.653GiB","Name":"new-mysql","NetIO":"1.54kB / 126B","PIDs":"37"}
{"BlockIO":"69.6kB / 12.3kB","CPUPerc":"0.00%","Container":"2d9c5e62586de50978648bb89f344acf1f8d10936caf3f3c26e7e092a7be9c03","ID":"2d9c5e62586d","MemPerc":"0.11%","MemUsage":"8.883MiB / 7.653GiB","Name":"web3","NetIO":"1.8kB / 126B","PIDs":"11"}
{"BlockIO":"0B / 12.3kB","CPUPerc":"0.00%","Container":"1fced9d8a8cb44f3c71df5403966359612b6bc90fee7857120676de63afc16a5","ID":"1fced9d8a8cb","MemPerc":"0.11%","MemUsage":"8.785MiB / 7.653GiB","Name":"web2","NetIO":"1.92kB / 126B","PIDs":"11"}
{"BlockIO":"0B / 12.3kB","CPUPerc":"0.00%","Container":"b68080951edff94b3760fc39145b409bfb2dad640dc64ae75146d653b9836d61","ID":"b68080951edf","MemPerc":"0.11%","MemUsage":"8.871MiB / 7.653GiB","Name":"web1","NetIO":"2.05kB / 126B","PIDs":"11"}
{"BlockIO":"0B / 212MB","CPUPerc":"0.16%","Container":"ec71e10eb23e8eba659c37c87e14f3b72708120d6f4bbef3810e4277b8281bde","ID":"ec71e10eb23e","MemPerc":"9.79%","MemUsage":"767.5MiB / 7.653GiB","Name":"jenkins-server","NetIO":"71.3MB / 6.5MB","PIDs":"52"}
{"BlockIO":"62.8MB / 13.5MB","CPUPerc":"0.00%","Container":"e52359ff4cd8577d8e6c37c9d7232d6201b9de195b9aebc09681419accd1ff9a","ID":"e52359ff4cd8","MemPerc":"1.05%","MemUsage":"82.45MiB / 7.653GiB","Name":"portainer","NetIO":"6.26kB / 126B","PIDs":"12"}
namansinghparihar@NAMANS-MacBook-Air-3 ~ %

```

- Wide output format

```

namansinghparihar — root@a98211a2d307: / — zsh — 91x26

namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stats --no-stream --no-trunc
CONTAINER ID        MEM USAGE / LIMIT     MEM %      NET I/O       BLOCK I/O      NAME           PIDS        CPU %
aed1d56d05dfbda6ed29158920d6a2912e93916f19283ba9a70f97c953e5954    45.7MiB / 7.653GiB     0.58%      2.19kB / 1.46kB    856kB / 0B     flask-app      3           0.25%
2f54a59613fa02f924ec26762f368f7cb8199fd7fd6766178d111d946bf88e5e    8.867MiB / 7.653GiB     0.11%      1.9kB / 670B      0B / 4.1kB     nginx-custom   11          0.00%
b4a5c1100ff05056108f59c6b5528f5b7e4223320f0b95e34427e3929ae7e7a9    366.3MiB / 7.653GiB     4.67%      1.54kB / 126B     65.5kB / 15.1MB    new-mysql      37          0.36%
2d9c5e62586de50978648bb89f344acf1f8d10936caf3f3c26e7e092a7be9c03    8.883MiB / 7.653GiB     0.11%      1.8kB / 126B      69.6kB / 12.3kB    web3           11          0.00%
1fced9d8a8cb44f3c71df5403966359612b6bc90fee7857120676de63afc16a5    8.785MiB / 7.653GiB     0.11%      1.92kB / 126B     0B / 12.3kB       web2           11          0.00%
b68080951edff94b3760fc39145b409bfb2dad640dc64ae75146d653b9836d61    8.871MiB / 7.653GiB     0.11%      2.05kB / 126B     0B / 12.3kB       web1           11          0.00%
ec71e10eb23e8eba659c37c87e14f3b72708120d6f4bbef3810e4277b8281bde    767.5MiB / 7.653GiB     9.79%      71.3MB / 6.5MB    0B / 212MB        jenkins-server 52          0.18%
e52359ff4cd8577d8e6c37c9d7232d6201b9de195b9aebc09681419accd1ff9a    82.45MiB / 7.653GiB     1.05%      6.26kB / 126B     62.8MB / 13.5MB    portainer      12          0.00%
namansinghparihar@NAMANS-MacBook-Air-3 ~ %

```

b) docker top

- Displays running processes inside a container

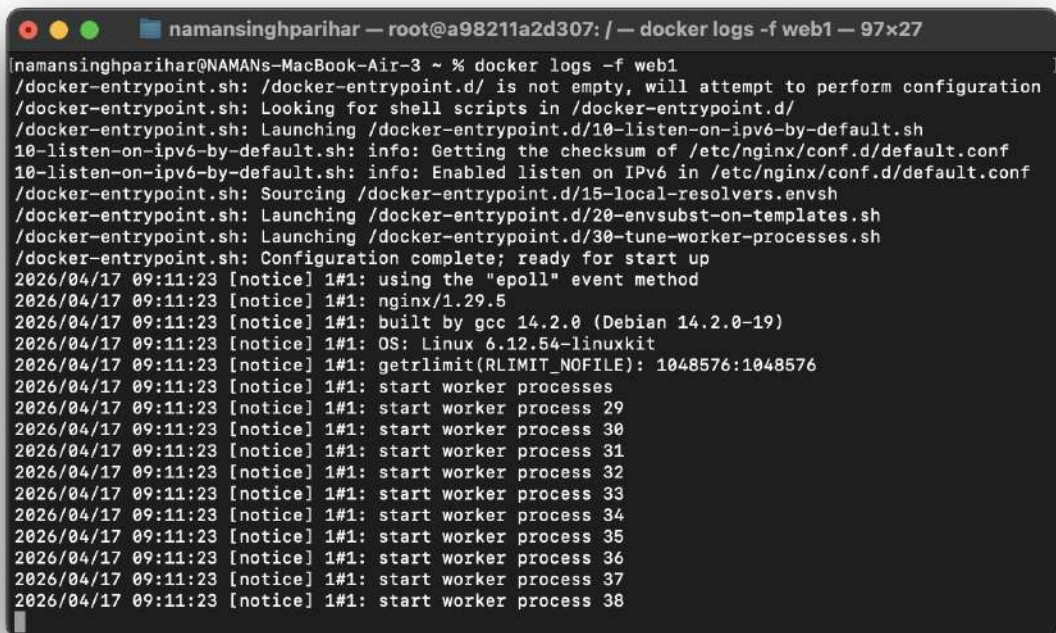

```

nanansinghparihar@NAMANS-MacBook-Air-3 ~ % ps aux | grep docker
nanansinghparihar 6996  0.3  1.0 413611856 159536  ?? S   Sat09AM 17:09:48 /Applications/Docker.app/Contents/MacOS/com.docker.backend services
nanansinghparihar 37482 0.0  0.3 1882019216 51248  ?? S   Sat09AM 0:00:25 /Applications/Visual Studio Code.app/Contents/Frameworks/Code Helper (Plugin).app/Contents/MacOS/Code Helper (Plugin) /Users/n
amansinghparihar/.vscode/extensions/ms-azurertools.vscode-containers-2.4.1/dist/dockerfile-language-server-nodejs/lib/server.js --node-ipc --node-ipc --clientProcessId=37217
nanansinghparihar 7124  0.0  0.2 412102168 27264  ?? S   Sat09AM 0:07:59 /Applications/Docker.app/Contents/Resources/bin/cagent api --listen unix:///Users/nanansinghparihar/Library/Containers/com.dock
er.docker/Data/docker-cagent.sock --model=gateway https://ai-backend-service.docker.com/proxy?origin=desktop&desktopVersion=4.57.0 --pull-interval 15 docker/gordon
nanansinghparihar 7837  0.0  0.1 412611200 14352  ?? S   Sat09AM 0:02:15 /Applications/Docker.app/Contents/MacOS/com.docker.virtualization --kernel /Applications/Docker.app/Contents/Resources/linuxkit
/kernel --cmdline init/initrd loglevel=1 root=/dev/vdb rootfstype=erofs ro vsyscall=emulate panic=0 eth0.dhcp eth1.dhcp linuxkit.unified_cgroup_hierarchy=1 console=hvc0 virtio.net.disable_csnum=1 vpnkit.conn
ect-connect://2/10999 --boot /Applications/Docker.app/Contents/Resources/linuxkit/desktop.img --disk /Users/nanansinghparihar/Library/Containers/com.docker.docker/Data/vm/0/data/Docker.raw --networktype gviso
r --cpus 10 --memoryMiB 8092 --console=1mg /Users/nanansinghparihar/Library/Containers/com.docker.docker/Data/log/vm/console.log --watchdog --virtiofs /Users --virtiofs /Volumes --virtiofs /private --virtiofs
/tmp --virtiofs /var/folders --rosetta
nanansinghparihar 7833  0.0  0.2 412127664 28768  ?? S   Sat09AM 0:05:21 /Applications/Docker.app/Contents/MacOS/com.docker.build --log-no-timestamp
nanansinghparihar 7800  0.0  0.2 412134624 27248  ?? S   Sat09AM 0:11:40 /Applications/Docker.app/Contents/MacOS/com.docker.backend fork
nanansinghparihar 4998  0.0  0.2 412136468 27360  ?? S   Sat09AM 0:37:52 /Applications/Docker.app/Contents/MacOS/com.docker.backend
root 567  0.0  0.0 41918992 592  ?? Ss  Sat08AM 0:00:12 /Library/PrivilegedHelperTools/com.docker.vmmtd
nanansinghparihar 38484 0.0  0.0 418735216 1616 s000 S+  12:33AM 0:00:01 grep docker
nanansinghparihar 38481 0.0  0.1 411857432 24384  ?? S   12:33AM 0:00:02 /Applications/Docker.app/Contents/Resources/bin/docker stats --all --no-trunc --no-stream --format {{ json .}}
nanansinghparihar 38480 0.0  0.2 411868096 25584  ?? S   12:33AM 0:00:03 /Applications/Docker.app/Contents/Resources/bin/docker stats --all --no-trunc --no-stream --format {{ json .}}
nanansinghparihar 38399 0.0  0.2 411859536 25648  ?? S   12:33AM 0:00:03 /Applications/Docker.app/Contents/Resources/bin/docker stats --all --no-trunc --no-stream --format {{ json .}}
nanansinghparihar@NAMANS-MacBook-Air-3 ~ %

```

c) docker logs

- Shows logs generated by the container



```

nanansinghparihar — root@a98211a2d307: / — docker logs -f web1 — 97x27
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker logs -f web1]
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/17 09:11:23 [notice] 1#1: using the "epoll" event method
2026/04/17 09:11:23 [notice] 1#1: nginx/1.29.5
2026/04/17 09:11:23 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/17 09:11:23 [notice] 1#1: OS: Linux 6.12.54-linuxkit
2026/04/17 09:11:23 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/17 09:11:23 [notice] 1#1: start worker processes
2026/04/17 09:11:23 [notice] 1#1: start worker process 29
2026/04/17 09:11:23 [notice] 1#1: start worker process 30
2026/04/17 09:11:23 [notice] 1#1: start worker process 31
2026/04/17 09:11:23 [notice] 1#1: start worker process 32
2026/04/17 09:11:23 [notice] 1#1: start worker process 33
2026/04/17 09:11:23 [notice] 1#1: start worker process 34
2026/04/17 09:11:23 [notice] 1#1: start worker process 35
2026/04/17 09:11:23 [notice] 1#1: start worker process 36
2026/04/17 09:11:23 [notice] 1#1: start worker process 37
2026/04/17 09:11:23 [notice] 1#1: start worker process 38

```

- Last N lines

```
namansinghparihar — root@a98211a2d307: / — zsh — 97x27
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker logs --tail 100 web1
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/04/17 09:11:23 [notice] 1#1: using the "epoll" event method
2026/04/17 09:11:23 [notice] 1#1: nginx/1.29.5
2026/04/17 09:11:23 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/04/17 09:11:23 [notice] 1#1: OS: Linux 6.12.54-linuxkit
2026/04/17 09:11:23 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/04/17 09:11:23 [notice] 1#1: start worker processes
2026/04/17 09:11:23 [notice] 1#1: start worker process 29
2026/04/17 09:11:23 [notice] 1#1: start worker process 30
2026/04/17 09:11:23 [notice] 1#1: start worker process 31
2026/04/17 09:11:23 [notice] 1#1: start worker process 32
2026/04/17 09:11:23 [notice] 1#1: start worker process 33
2026/04/17 09:11:23 [notice] 1#1: start worker process 34
2026/04/17 09:11:23 [notice] 1#1: start worker process 35
2026/04/17 09:11:23 [notice] 1#1: start worker process 36
2026/04/17 09:11:23 [notice] 1#1: start worker process 37
2026/04/17 09:11:23 [notice] 1#1: start worker process 38
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

- Combine options

```
namansinghparihar — root@a98211a2d307: / — docker logs -f --tail 50 -t web1 — 97x37
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker logs -f --tail 50 -t web1
2026-04-17T09:11:23.144484138Z /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will at
tempt to perform configuration
2026-04-17T09:11:23.144636305Z /docker-entrypoint.sh: Looking for shell scripts in /docker-entryp
oint.d/
2026-04-17T09:11:23.145136430Z /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on
-ipv6-by-default.sh
2026-04-17T09:11:23.151402847Z 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /et
c/nginx/conf.d/default.conf
2026-04-17T09:11:23.155190055Z 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /
etc/nginx/conf.d/default.conf
2026-04-17T09:11:23.155275347Z /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-reso
lvers.envsh
2026-04-17T09:11:23.155278388Z /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst
on-templates.sh
2026-04-17T09:11:23.158297180Z /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-work
er-processes.sh
2026-04-17T09:11:23.159564847Z /docker-entrypoint.sh: Configuration complete; ready for start up
2026-04-17T09:11:23.165895347Z 2026/04/17 09:11:23 [notice] 1#1: using the "epoll" event method
2026-04-17T09:11:23.165905638Z 2026/04/17 09:11:23 [notice] 1#1: nginx/1.29.5
2026-04-17T09:11:23.165906805Z 2026/04/17 09:11:23 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2
.0-19)
2026-04-17T09:11:23.165907555Z 2026/04/17 09:11:23 [notice] 1#1: OS: Linux 6.12.54-linuxkit
2026-04-17T09:11:23.165908222Z 2026/04/17 09:11:23 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 104857
6:1048576
2026-04-17T09:11:23.166302097Z 2026/04/17 09:11:23 [notice] 1#1: start worker processes
2026-04-17T09:11:23.166430680Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 29
2026-04-17T09:11:23.166439597Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 30
2026-04-17T09:11:23.166600555Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 31
2026-04-17T09:11:23.166655972Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 32
2026-04-17T09:11:23.166671680Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 33
2026-04-17T09:11:23.166723805Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 34
2026-04-17T09:11:23.166802847Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 35
2026-04-17T09:11:23.166944638Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 36
2026-04-17T09:11:23.166950347Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 37
2026-04-17T09:11:23.166955138Z 2026/04/17 09:11:23 [notice] 1#1: start worker process 38
```

Step 6: Network Types

a) Bridge Network

- Default network type in Docker
- Containers can communicate with each other

```
namansinghparihar — root@a98211a2d307: / — zsh — 97x34
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d --name web1 --network my-network nginx
b8b7c7f6dba0f89e9c3ac6f1d5c338c0f9ddba20c4981eb315ed645b85b4e2d3
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rm -f web2
web2
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d --name web2 --network my-network nginx
52fa753512680cb31dc4bf14ea4c15ceb28a9de20c700578413c8ca30886afff
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker exec web1 curl http://web2
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total   Spent    Left   Speed
100   615    100   615    0     0   265k      0 --:--:-- --:--:-- --:--:--   300k
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

b) None Network

- No network access for the container
- Fully isolated


```
namansinghparihar — root@a98211a2d307: / — zsh — 97x23

[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d --name isolated-app --network none alpine]
e sleep 3600
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
d17f077ada11: Pull complete
4c52853c1057: Download complete
2bf6ea167e4a: Download complete
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest
053ba10d449768b4e3d058910b93dfc230c42b79af9475409edc8d3cc816914d
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker exec isolated-app ifconfig]
lo
  Link encap:Local Loopback
  inet addr:127.0.0.1  Mask:255.0.0.0
  inet6 addr: ::1/128  Scope:Host
  UP LOOPBACK RUNNING  MTU:65536  Metric:1
  RX packets:0 errors:0 dropped:0 overruns:0 frame:0
  TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
  collisions:0 txqueuelen:1000
  RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

c) Host Network

- Shares host's network directly
- No network isolation

```
namansinghparihar — root@a98211a2d307: / — zsh — 97x34
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d \
  --name host-app \
  -p 1234:80 \
  nginx
6d3f0101377cdec3552b7bd05ba215567c9e703b9ae2bbe850e7989ea889856c
namansinghparihar@NAMANS-MacBook-Air-3 ~ % curl http://localhost:1234
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

[https://pariharnaman.github.io/containerization-devops/
Lab/Experiment6/](https://pariharnaman.github.io/containerization-devops/Lab/Experiment6/)

containerization-devops

Experiment 6: Comparison of Docker Run and Docker Compose

Task 1: Single Container Comparison

Step 1: Run Nginx Using Docker Run

- Starts an Nginx container using a single command
- Quick way to deploy a container without configuration files
- Useful for testing and basic setups

Illustration

- Running Alpine

```
namansinghparihar — zsh — 80x33
Last login: Fri Apr 24 17:06:54 on ttys002
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
  nginx:alpine

Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
a96b658a00fe: Pull complete
10cbc192f783: Pull complete
910c2a6cad6d: Pull complete
662c8d6f6620: Pull complete
83fbf849ee89: Pull complete
634f1d1ce0f7: Pull complete
a89d14ef5abe: Pull complete
6fe7ecffd0b2: Download complete
59993c59bb89: Download complete
Digest: sha256:5616878291a2eed594aee8db4dade5878cf7edcb475e59193904b198d9b830de
Status: Downloaded newer image for nginx:alpine
26c2f907ff9321d923953a8809378bc0bba175fed53986b82b206a1d0d67c8ac
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED
26c2f907ff93	nginx:alpine	"/docker-entrypoint..."	16 seconds ago
	lab-nginx	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	
e52359ff4cd8	portainer/portainer-ce:lts	"/portainer"	6 weeks ago
	portainer	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp	

```
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

- Localhost port



- Removing Alpine

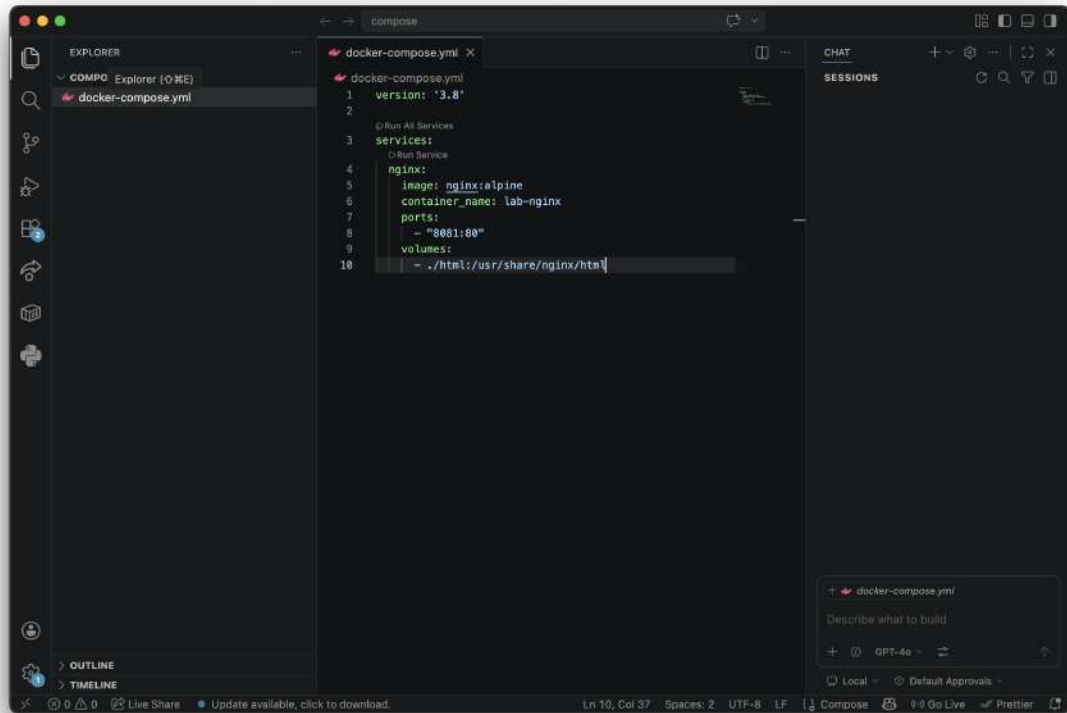
```
namansinghparihar — zsh — 80x5
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker stop lab-nginx
lab-nginx
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker rm lab-nginx
lab-nginx
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 2: Run Same Setup Using Docker Compose

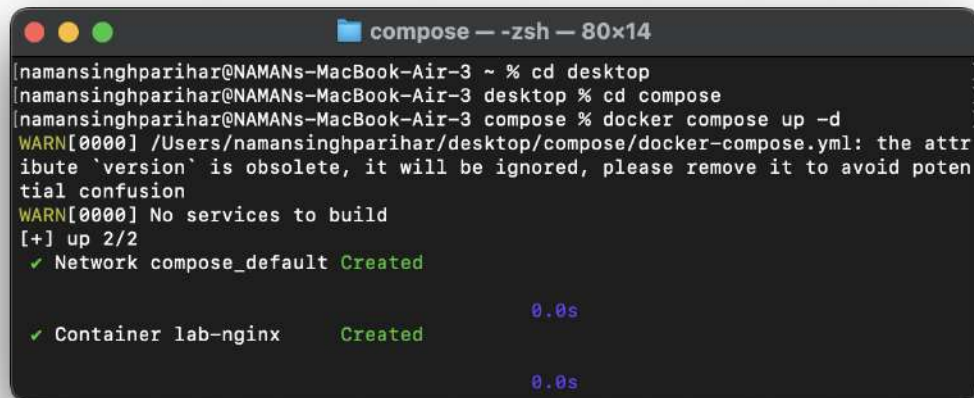
- Uses a YAML file to define and run services
- Easier to manage compared to single commands
- Ideal for repeatable and structured setups

Illustration

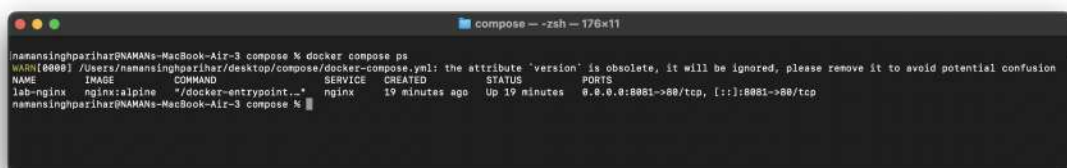
- Compose File



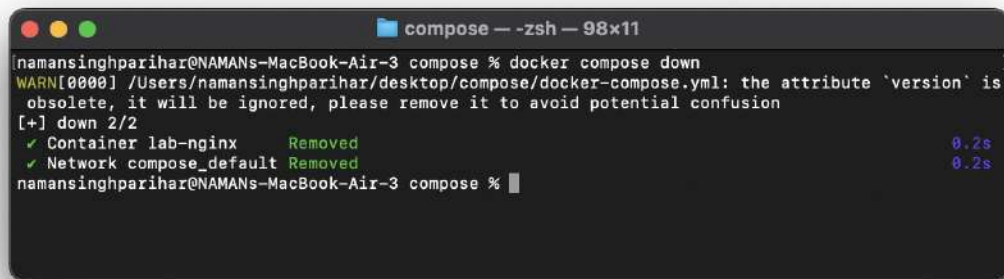
- Compose Up



- Compose verify using ps



- Compose Down



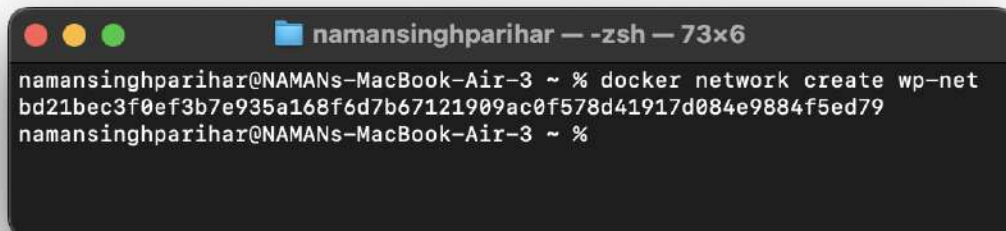
```
compose — -zsh — 98x11
[namansinghparihar@NAMANS-MacBook-Air-3 compose % docker compose down
WARN[0000] /Users/namansinghparihar/desktop/compose/docker-compose.yml: the attribute `version` is
obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 2/2
✓ Container lab-nginx      Removed      0.2s
✓ Network compose_default Removed      0.2s
namansinghparihar@NAMANS-MacBook-Air-3 compose %
```

Task 2: Multi-Container Application

Step 1: Using Docker Run

a) Create Network

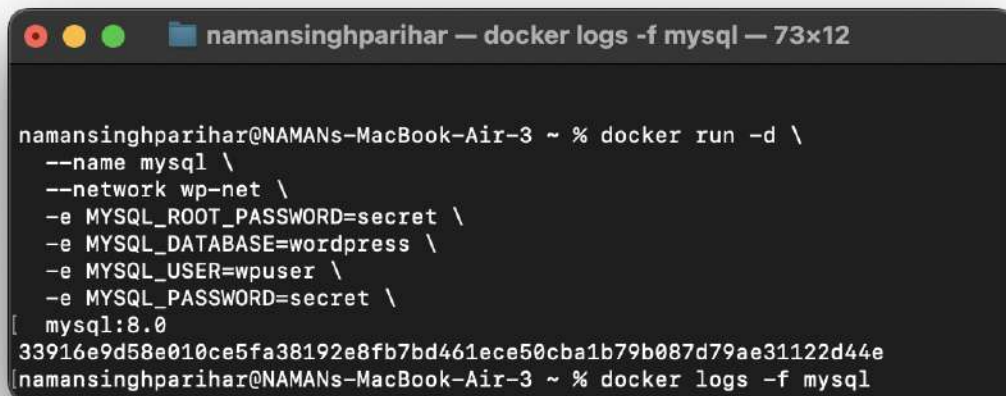
- Creates a custom network for container communication



```
namansinghparihar — -zsh — 73x6
namansinghparihar@NAMANS-MacBook-Air-3 ~ % docker network create wp-net
bd21bec3f0ef3b7e935a168f6d7b67121909ac0f578d41917d084e9884f5ed79
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

b) Run MySQL

- Starts MySQL container as database service

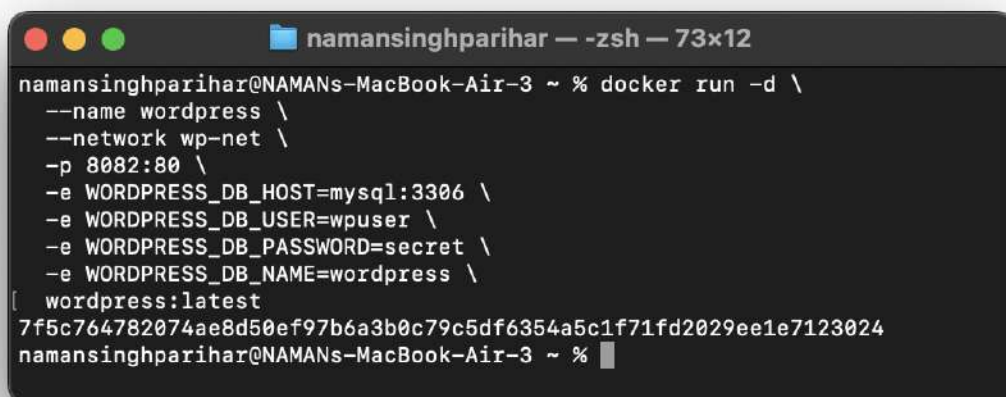


```
namansinghparihar — docker logs -f mysql — 73x12

namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d \
  --name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  -e MYSQL_USER=wpuser \
  -e MYSQL_PASSWORD=secret \
  mysql:8.0
33916e9d58e010ce5fa38192e8fb7bd461ece50cba1b79b087d79ae31122d44e
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker logs -f mysql
```

c) Run WordPress

- Starts WordPress container connected to MySQL

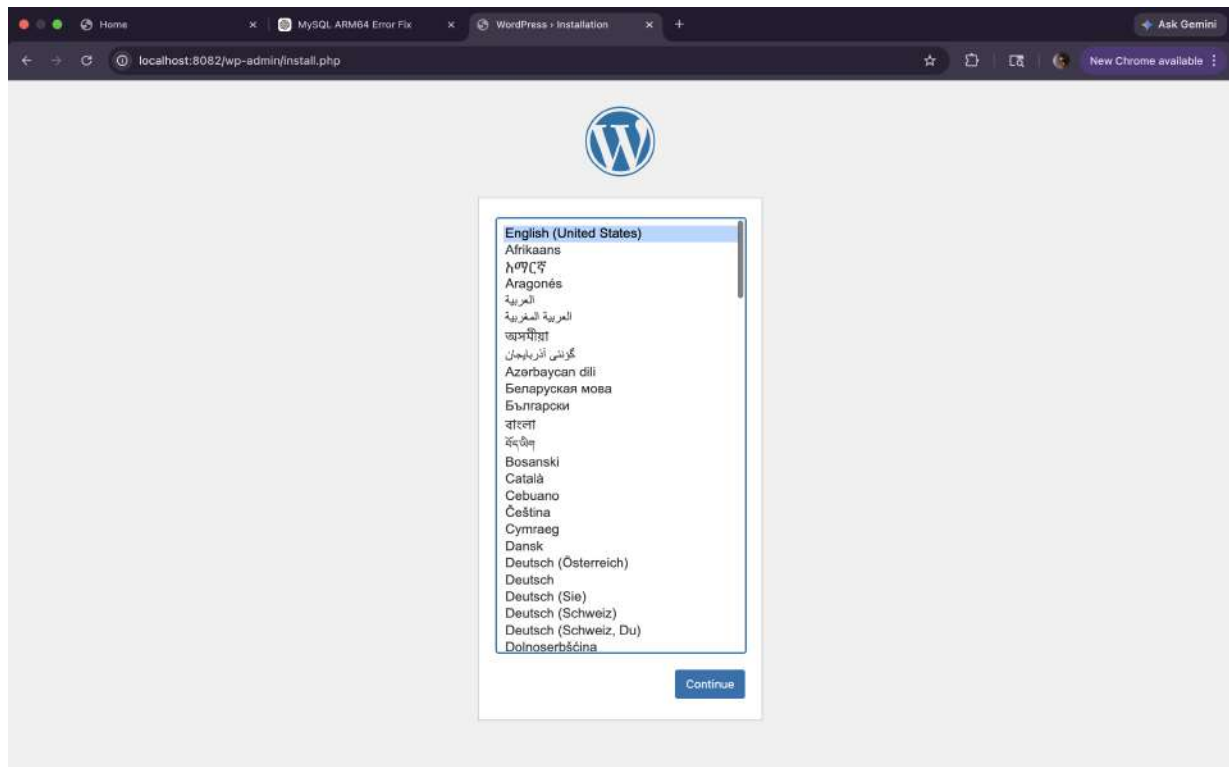


```
namansinghparihar — -zsh — 73x12

namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d \
  --name wordpress \
  --network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql:3306 \
  -e WORDPRESS_DB_USER=wpuser \
  -e WORDPRESS_DB_PASSWORD=secret \
  -e WORDPRESS_DB_NAME=wordpress \
  wordpress:latest
7f5c764782074ae8d50ef97b6a3b0c79c5df6354a5c1f71fd2029ee1e7123024
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

d) Localhost Run Check

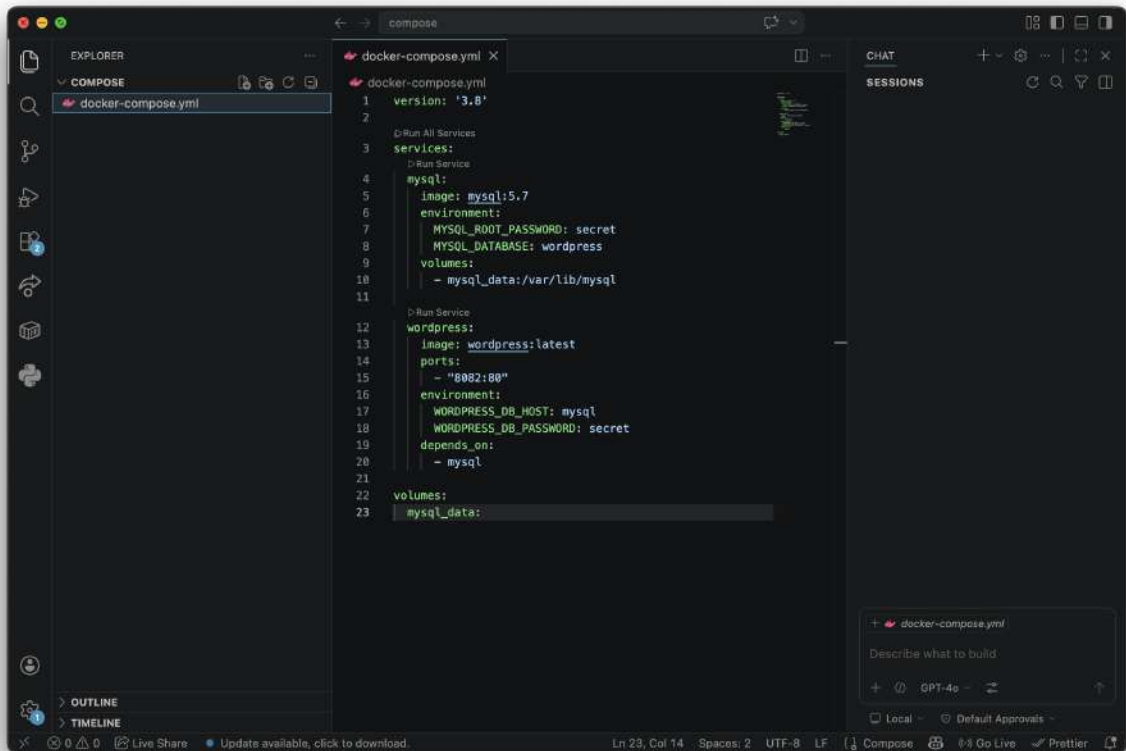
- Verify application is running in browser



Step 2: Using Docker Compose

a) Create Compose File

- Define services in a YAML file



b) Docker Compose Up

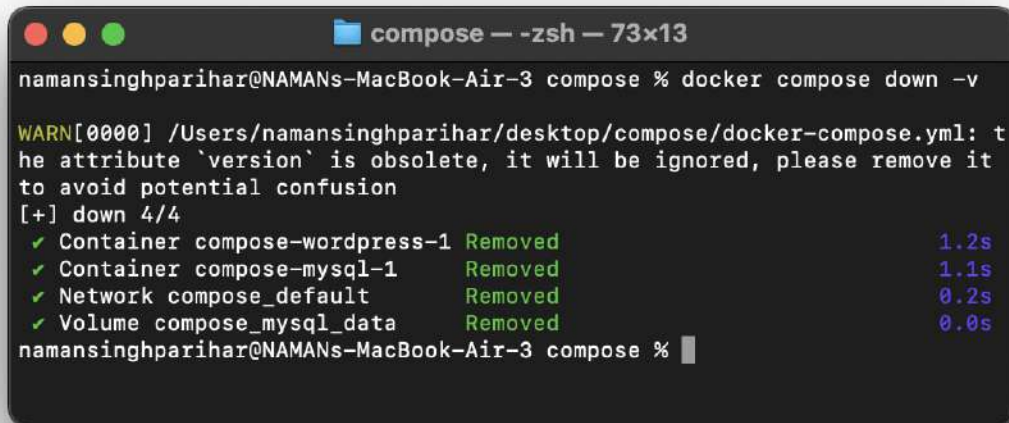
- Starts all services together

```
compose — zsh — 73x13

[namansinghparihar@NAMANS-MacBook-Air-3 compose % docker compose up -d ]
WARN[0000] /Users/namansinghparihar/desktop/compose/docker-compose.yml: t
he attribute 'version' is obsolete, it will be ignored, please remove it
to avoid potential confusion
WARN[0000] No services to build
[+] up 3/3
  ✓ Network compose_default          Created      0.0s
  ✓ Container compose-mysql-1        Created      0.0s
  ✓ Container compose-wordpress-1    Created      0.0s
namansinghparihar@NAMANS-MacBook-Air-3 compose %
```

c) Docker Compose Down

- Stops and removes all services



```
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker compose down -v

WARN[0000] /Users/namansinghparihar/desktop/compose/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] down 4/4
✓ Container compose-wordpress-1 Removed 1.2s
✓ Container compose-mysql-1 Removed 1.1s
✓ Network compose_default Removed 0.2s
✓ Volume compose_mysql_data Removed 0.0s
namansinghparihar@NAMANs-MacBook-Air-3 compose %
```

Task 3: Convert Docker Run to Docker Compose

1: Create Docker Compose

- Convert `docker run` commands into a compose file
- Defines services, networks, and configurations in one place



```
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker compose up -d
WARN[0000] No services to build
[+] up 2/2
✓ Network compose_default Created 0.0s
✓ Container webapp Created 0.0s
namansinghparihar@NAMANs-MacBook-Air-3 compose %
```

2: Verify Compose

- Check if services are running correctly
- Ensures setup is working as expected

```
compose — -zsh — 109x12
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker compose up -d
WARN[0000] No services to build
[+] up 2/2
✓ Network compose_default Created 0.0s
✓ Container webapp Created 0.0s
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS
PORTS
webapp node:18-alpine "docker-entrypoint.s..." webapp 31 seconds ago Restarting (0) 4 seconds ago
namansinghparihar@NAMANs-MacBook-Air-3 compose %
```

3: Docker Compose Down

- Stops and removes all running services
- Cleans up the environment

```
compose — -zsh — 109x7
✓ Container webapp Created 0.0s
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker compose down
[+] down 2/2
✓ Container webapp Removed 0.0s
✓ Network compose_default Removed 0.2s
namansinghparihar@NAMANs-MacBook-Air-3 compose % clear
```

Problem 2: Volume + Network Configuration

1: Create Network

- Creates a custom network for container communication

```
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker network create app-net
16b02e044431eb37af75d4925797bb3d5f177f5ae896abe66e70c5c6530d21de
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d \
  --name postgres-db \
  --network app-net \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15

Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
100d1dae010f: Pull complete
705ba9f7ca7e: Pull complete
1b4af8a0a0e7: Pull complete
7809f62fa1ea: Pull complete
e4fb5f1cd4d4: Pull complete
fb60363eacea: Pull complete
424e2ef71176: Pull complete
17378f5e5b97: Pull complete
24f274ec7f84: Pull complete
9d4f5cc97771: Pull complete
8f880f1adddd: Pull complete
21fbaba40a21: Pull complete
69bf3bcc46b5: Pull complete
7e81c455a81b: Pull complete
5c2511632614: Download complete
3fd84edda450: Download complete
Digest: sha256:29342cb52157b098821961d2c14eec3c019071f56e5d559e990cf07cf541ea9b
Status: Downloaded newer image for postgres:15
6fda8163ad8ecbe99869e9547bf9071f2ce02f125466c0f8fa4a8531d14a379b
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

2: Run PostgreSQL DB

- Starts PostgreSQL container with persistent storage
- Stores data using volumes

```
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker network create app-net
16b02e04431eb37af75d4925797bb3d5f177f5ae896abe66e70c5c6530d21de
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d \
  --name postgres-db \
  --network app-net \
  -e POSTGRES_USER=admin \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:15

Unable to find image 'postgres:15' locally
15: Pulling from library/postgres
100d1dae010f: Pull complete
705ba9f7ca7e: Pull complete
1b4af8a0a0e7: Pull complete
7809f62fa1ea: Pull complete
e4fb5f1cd4d4: Pull complete
fb60363eeca: Pull complete
424e2ef71176: Pull complete
17378f5e5b97: Pull complete
24f274ec7f84: Pull complete
9d4f5cc97771: Pull complete
8f880f1adddd: Pull complete
21fbeb40a21: Pull complete
69bf3bcc46b5: Pull complete
7e81c455a81b: Pull complete
5c2511632614: Download complete
3fd84edda450: Download complete
Digest: sha256:29342cb52157b098821961d2c14eec3c019071f56a5d559e990cf07cf541ea9b
Status: Downloaded newer image for postgres:15
6fda8163ad8ecbe99869e9547bf9071f2ce02f125466c0f8fa4a8531d14a379b
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

3: Run Python Slim (3.11)

- Runs a lightweight Python container
- Connects to the PostgreSQL service

```
namansinghparihar@NAMANs-MacBook-Air-3 ~ % docker run -d \
  --name backend \
  --network app-net \
  -p 7001:7000 \
  -e DB_HOST=postgres-db \
  -e DB_USER=admin \
  -e DB_PASS=secret \
  python:3.11-slim
ee96b6c40bb86b5211c45e3b152e60d2d0439a3812d8fb56401a8962b6957aaf
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

Problem 2: Volume + Network Configuration

1: Docker Compose Up

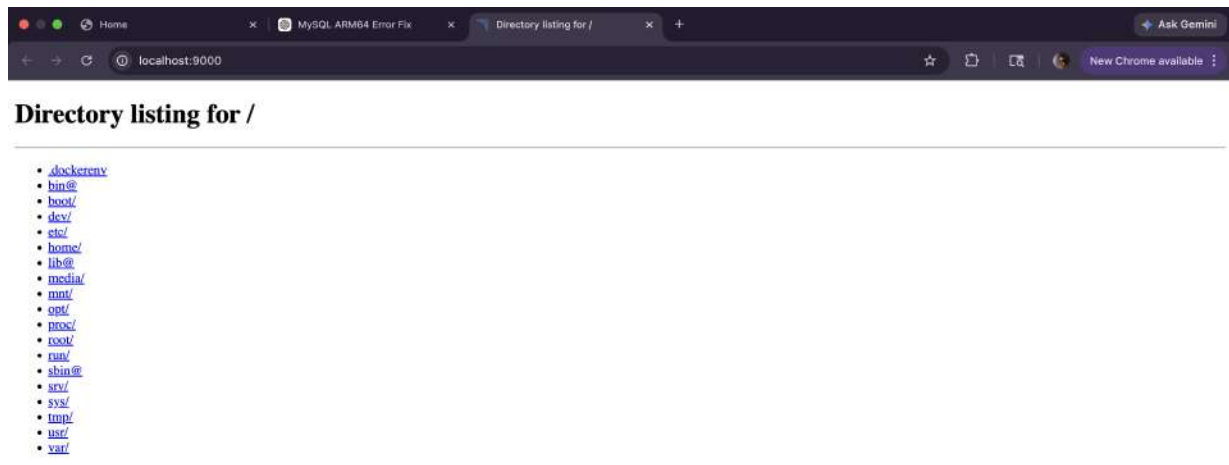
- Starts all services defined in the compose file
- Automatically sets up network and volumes

```
compose — zsh — 109x15

[namansinghparihar@NAMANs-MacBook-Air-3 compose % docker compose up -d
WARN[0000] No services to build
[+] up 3/3
✓ Network compose_app-net Created 0.0s
✓ Container postgres-db Created 0.0s
✓ Container backend Created 0.0s
[namansinghparihar@NAMANs-MacBook-Air-3 compose % docker compose ps
NAME                IMAGE             COMMAND                  SERVICE    CREATED         STATUS          PORTS
S
backend             python:3.11-slim  "python -m http.serv..." backend    49 seconds ago  Up 49 seconds   0.0.
0.0:9000->8000/tcp, [::]:9000->8000/tcp
postgres-db         postgres:15       "docker-entrypoint.s..." postgres-db 49 seconds ago  Up 49 seconds   5432
/tcp
namansinghparihar@NAMANs-MacBook-Air-3 compose %
```

2: Localhost Check

- Verify services are running in the browser
- Confirms successful setup



Experiment 6 B

Multi-Container Application using Docker Compose (WordPress + Database)

Step 1: Create `docker-compose.yml` & Run

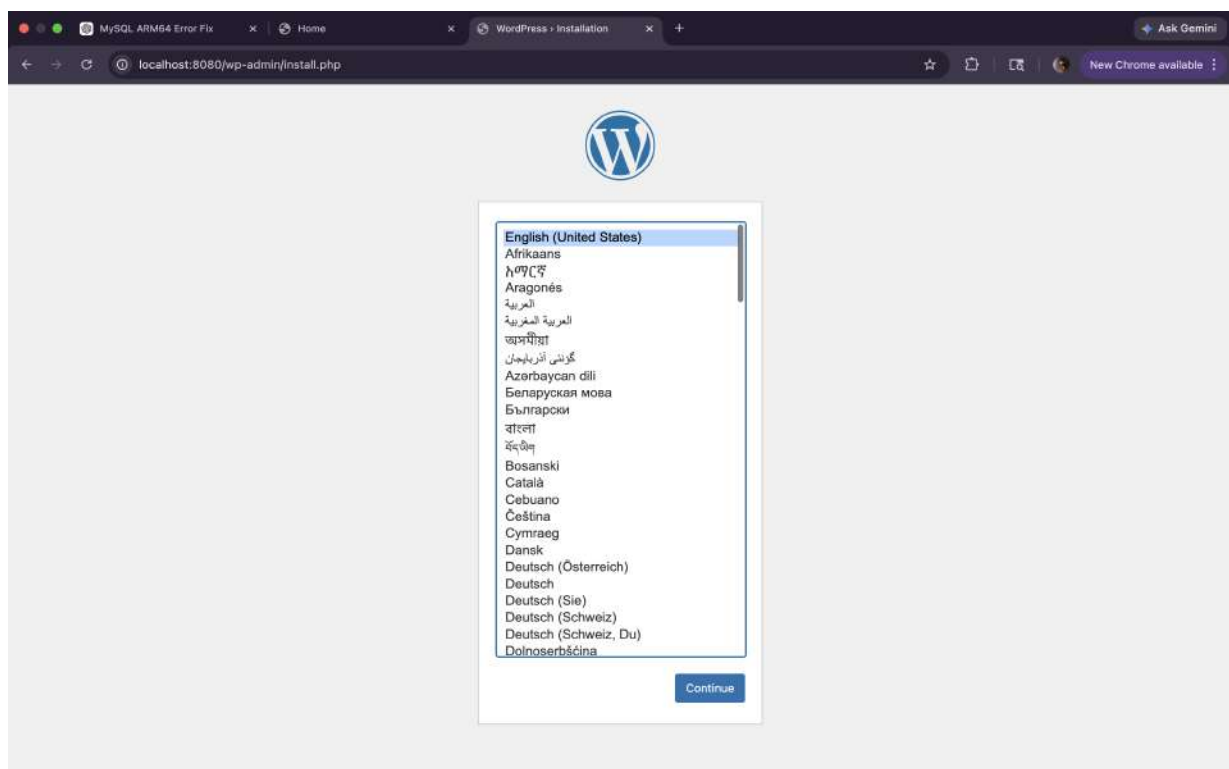
- Define WordPress and database services

- Start both containers together using compose

```
compose — zsh — 109x15
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker-compose up -d
WARN[0000] /Users/namansinghparihar/desktop/compose/docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion
WARN[0000] No services to build
[+] up 5/5
✓ Network compose_default Created 0.0s
✓ Volume compose_db_data Created 0.0s
✓ Volume compose_wp_data Created 0.0s
✓ Container wordpress_db Created 0.0s
✓ Container wordpress_app Created 0.0s
namansinghparihar@NAMANs-MacBook-Air-3 compose %
```

Step 2: Check Localhost

- Open browser to verify WordPress setup
- Confirms application is running



Step 3: Docker Compose Down

- Stops and removes all services
- Cleans up the environment

```
compose — zsh — 109x15
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker-compose down
WARN[0000] /Users/namansinghparihar/desktop/compose/docker-compose.yml: the attribute `version` is obsolete,
it will be ignored, please remove it to avoid potential confusion
[+] down 3/3
  ✓ Container wordpress_app Removed 1.2s
  ✓ Container wordpress_db Removed 1.0s
  ✓ Network compose_default Removed 0.2s
namansinghparihar@NAMANs-MacBook-Air-3 compose %
```

Running Same Setup with Docker Swarm

Step 1: Initialize Swarm

- Converts Docker into swarm mode
- Enables orchestration features

```
compose — zsh — 109x15
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker swarm init
Swarm initialized: current node (xu7ifwlaxifgccctwpp7xs5pa) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3xh9z40pjjpg1k4bw7r3sx0xvqwftklfaghxe0675xf4axzxm8-8gfhghi5fk59oc25lgr
    hn56xw 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
namansinghparihar@NAMANs-MacBook-Air-3 compose %
```

Step 2: Scale Service

- Increases or decreases number of service replicas
- Helps handle load and improve availability


```
compose -- -zsh -- 109x22

namansinghparihar@NAMANs-MacBook-Air-3 compose % docker stack deploy -c docker-compose.yml wpstack
Ignoring unsupported options: restart
Ignoring deprecated options:
container_name: Setting the container name is not supported.
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_wordpress
Creating service wpstack_db
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker service scale wpstack_wordpress=3
wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
```

Step 3: Remove Stack

- Stops and removes all services in the stack
- Cleans up swarm resources

```
compose -- -zsh -- 109x6

namansinghparihar@NAMANs-MacBook-Air-3 compose % docker service rm wpstack_wordpress
wpstack_wordpress
namansinghparihar@NAMANs-MacBook-Air-3 compose % docker stack rm wpstack
Removing service wpstack_db
Removing network wpstack_default
namansinghparihar@NAMANs-MacBook-Air-3 compose %
```

<https://pariharnaman.github.io/containerization-devops/Lab/Experiment7/>

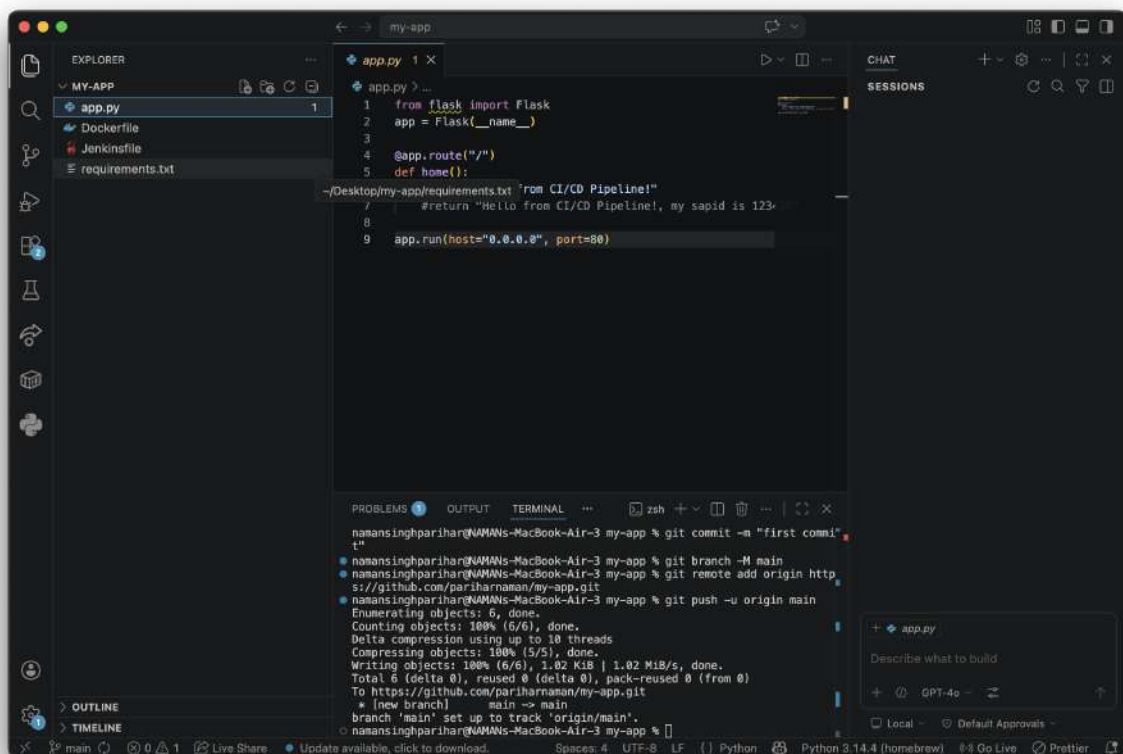
containerization-devops

Experiment 7: CI/CD using Jenkins, GitHub and Docker Hub

Part A: GitHub Repository Setup (Source Code + Build Definition)

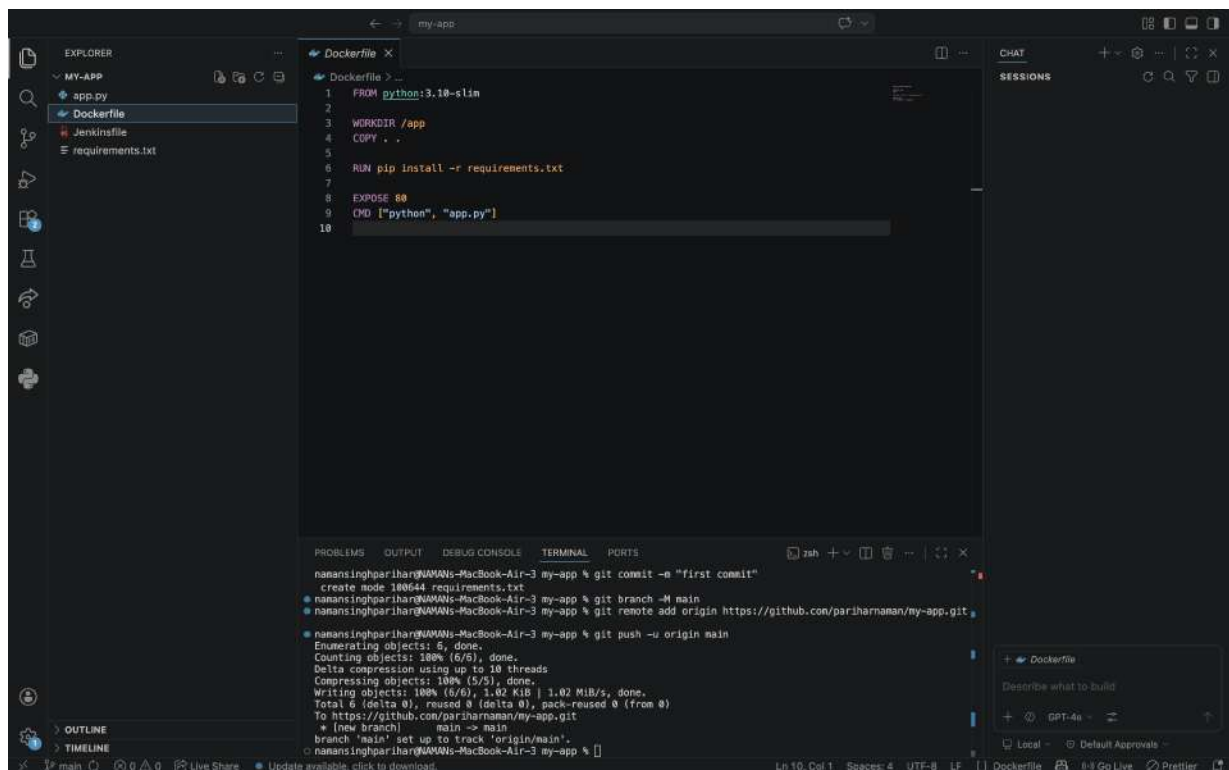
1: app.py

- Contains the main application code
- Defines the core functionality of the project



2: Dockerfile

- Defines how the application image is built
- Includes dependencies and runtime configuration



The screenshot shows the Visual Studio Code interface. The Explorer on the left shows a project named 'MY-APP' with files 'app.py', 'Dockerfile', 'Jenkinsfile', and 'requirements.txt'. The Dockerfile is open in the editor, showing the following content:

```
1 FROM python:3.10-slim
2
3 WORKDIR /app
4 COPY . .
5
6 RUN pip install -r requirements.txt
7
8 EXPOSE 80
9 CMD ["python", "app.py"]
10
```

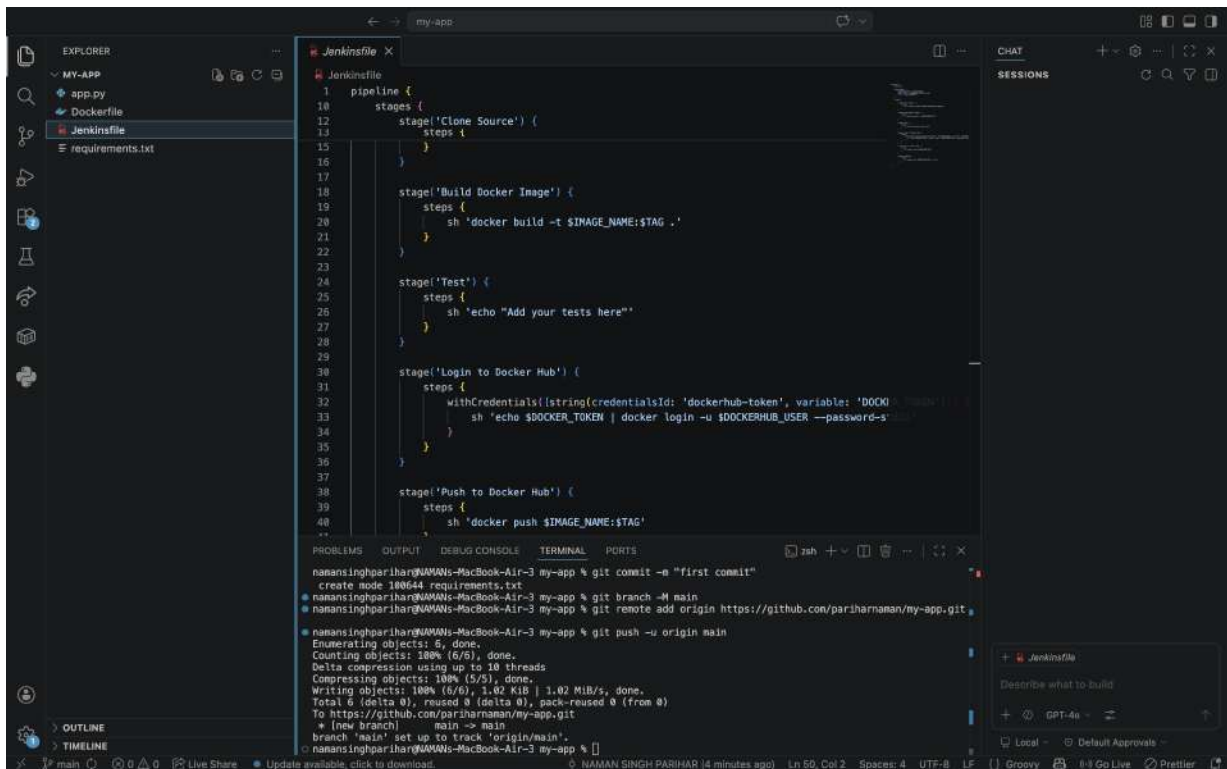
The Terminal at the bottom shows the following commands and output:

```
nanansingparihang@NANANS-MacBook-Air-3 my-app % git commit -m "first commit"
create mode 100644 requirements.txt
nanansingparihang@NANANS-MacBook-Air-3 my-app % git branch -M main
nanansingparihang@NANANS-MacBook-Air-3 my-app % git remote add origin https://github.com/pariharnaman/my-app.git
nanansingparihang@NANANS-MacBook-Air-3 my-app % git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 10 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 1.02 KiB | 1.02 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/pariharnaman/my-app.git
 * (new branch) main -> main
branch 'main' set up to track 'origin/main'.
nanansingparihang@NANANS-MacBook-Air-3 my-app %
```

The Chat panel on the right shows a 'Dockerfile' chat session with the prompt 'Describe what to build' and a response from GPT-4o.

3: Jenkinsfile

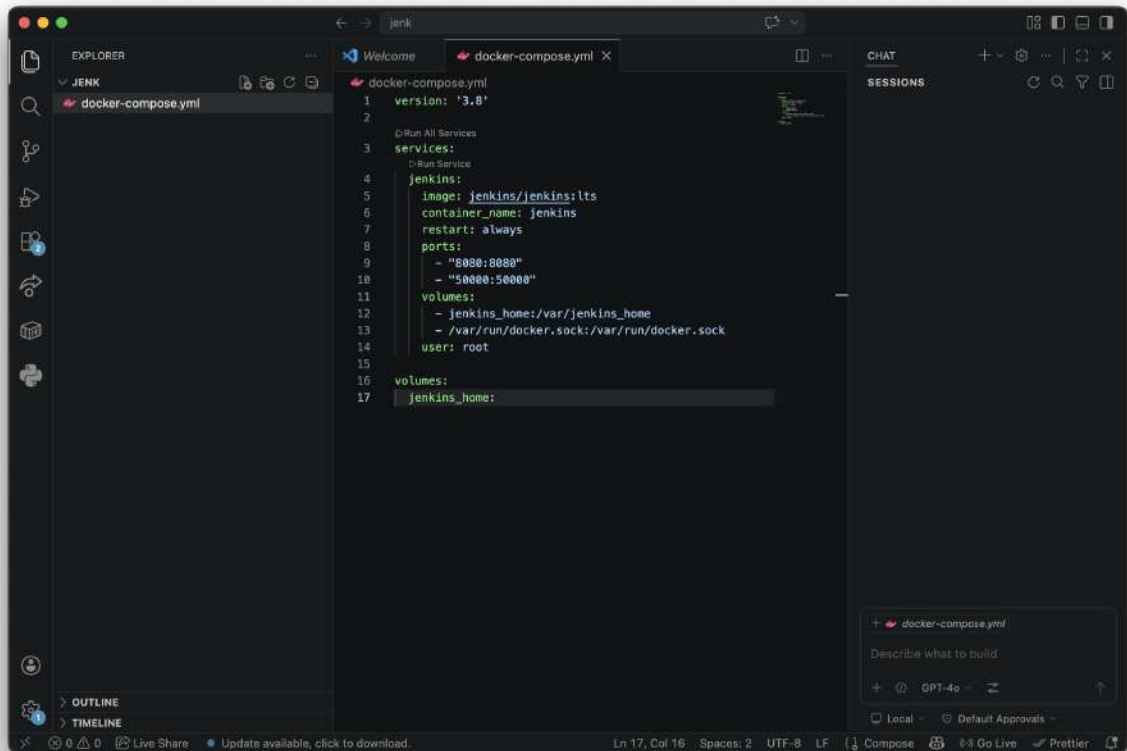
- Defines CI/CD pipeline stages
- Automates build, test, and deployment process



Part B: Jenkins Setup using Docker (Persistent Configuration)

1: Create Docker Compose File

- Defines Jenkins service with persistent storage
- Ensures data is retained across restarts



2: Start Jenkins

- Launch Jenkins container using Docker Compose
- Initializes the CI/CD environment

```
jenk — -zsh — 80x24
Last login: Fri May 1 22:26:17 on ttys000
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % cd desktop
[namansinghparihar@NAMANS-MacBook-Air-3 desktop % cd jenk
[namansinghparihar@NAMANS-MacBook-Air-3 jenk % docker compose up -d
WARN[0000] /Users/namansinghparihar/desktop/jenk/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] No services to build
[+] up 3/3
  ✓ Network jenk_default      Created          0.0s
  ✓ Volume jenk_jenkins_home  Created          0.0s
  ✓ Container jenkins         Created          0.0s
namansinghparihar@NAMANS-MacBook-Air-3 jenk %
```

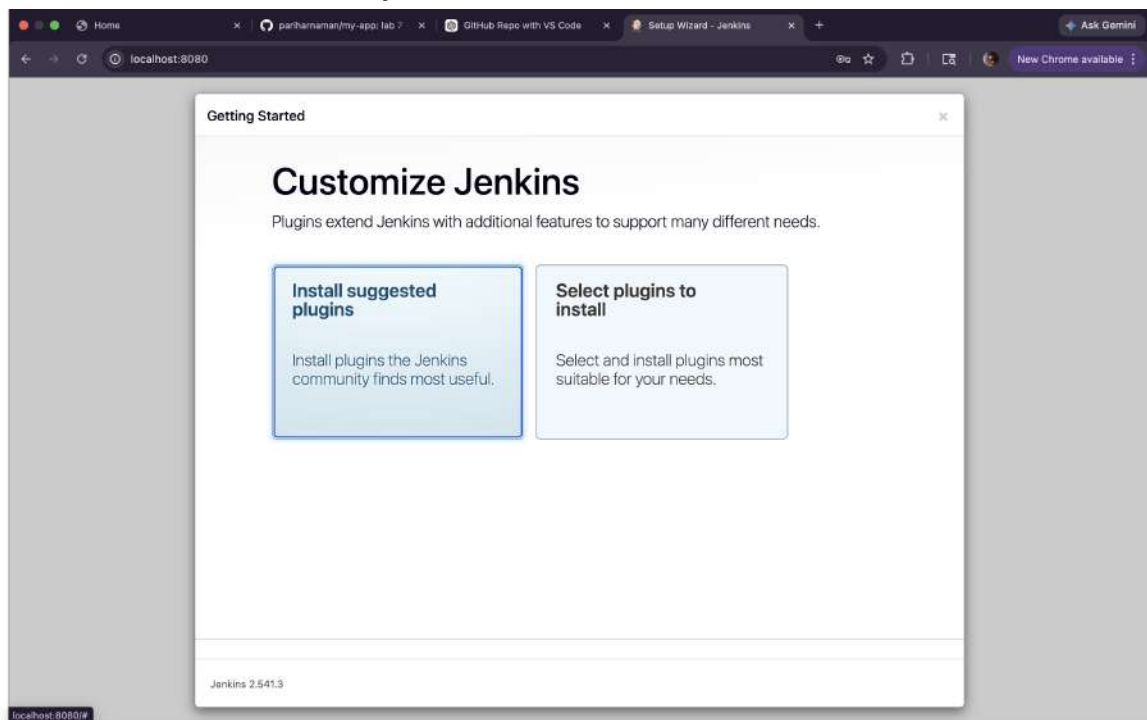
3: Unlock Jenkins

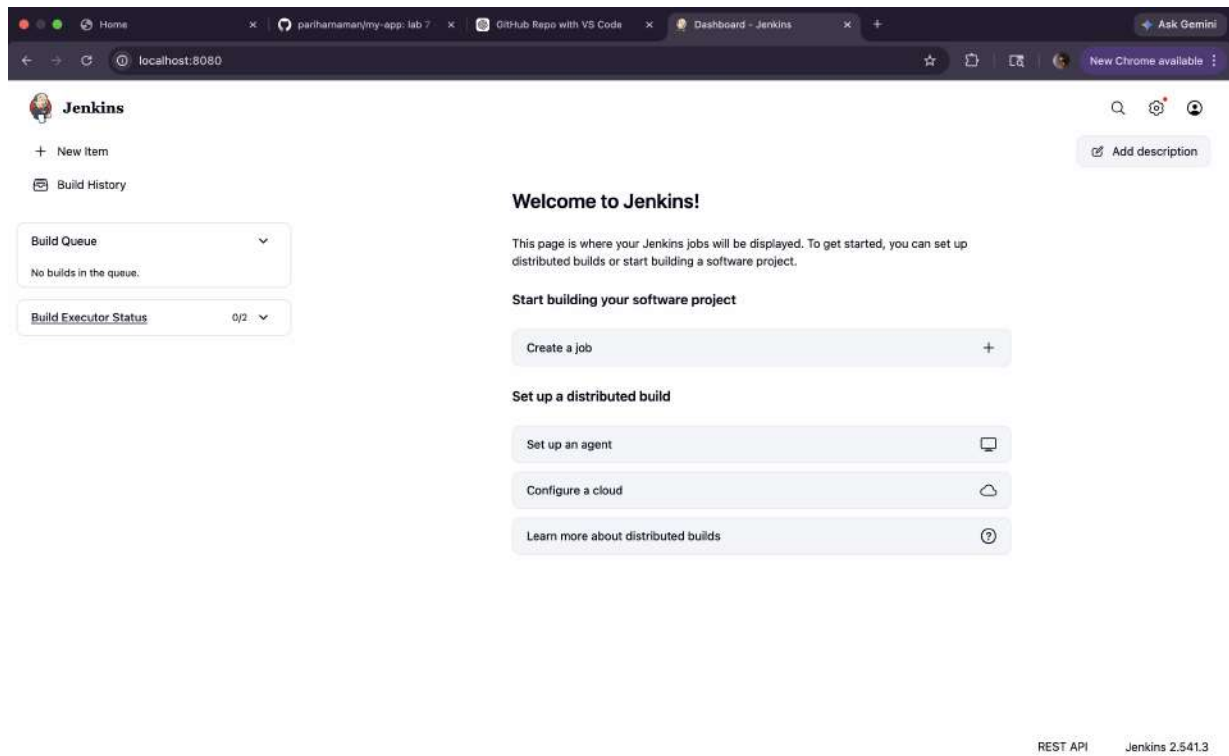
- Access Jenkins via browser
- Enter initial admin password to unlock

```
jenk — -zsh — 80x24
namansinghparihar@NAMANs-MacBook-Air-3 jenk % docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
b3190423d2604199b9f48a8180974d4d
namansinghparihar@NAMANs-MacBook-Air-3 jenk %
```

4: Jenkins on localhost

- Created first user
- Jenkins console accessibility

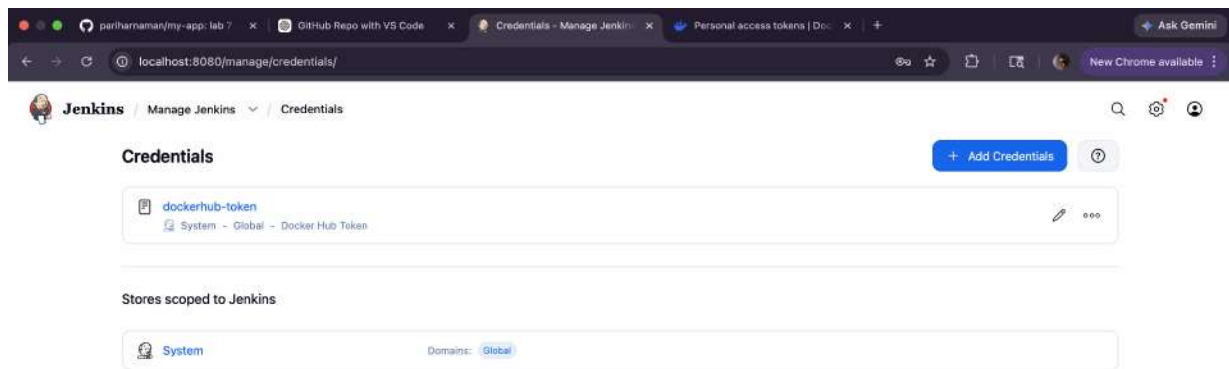
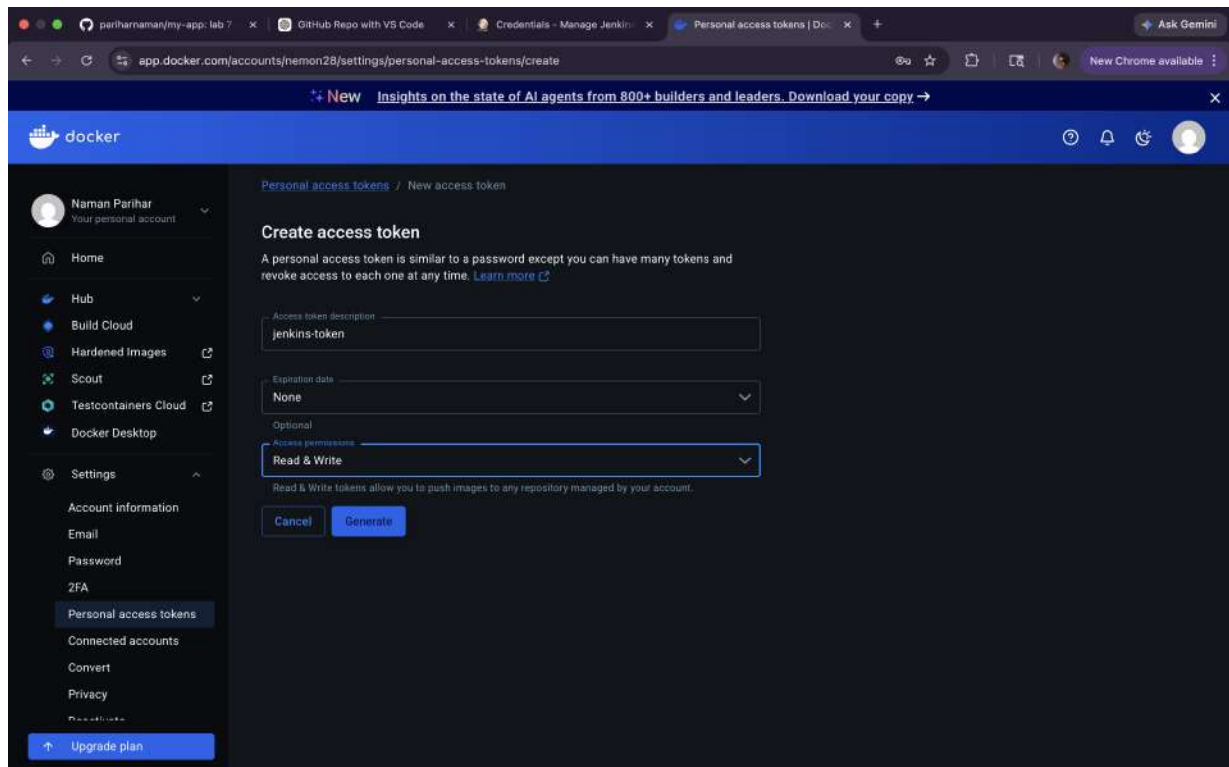




Part C: Jenkins Configuration

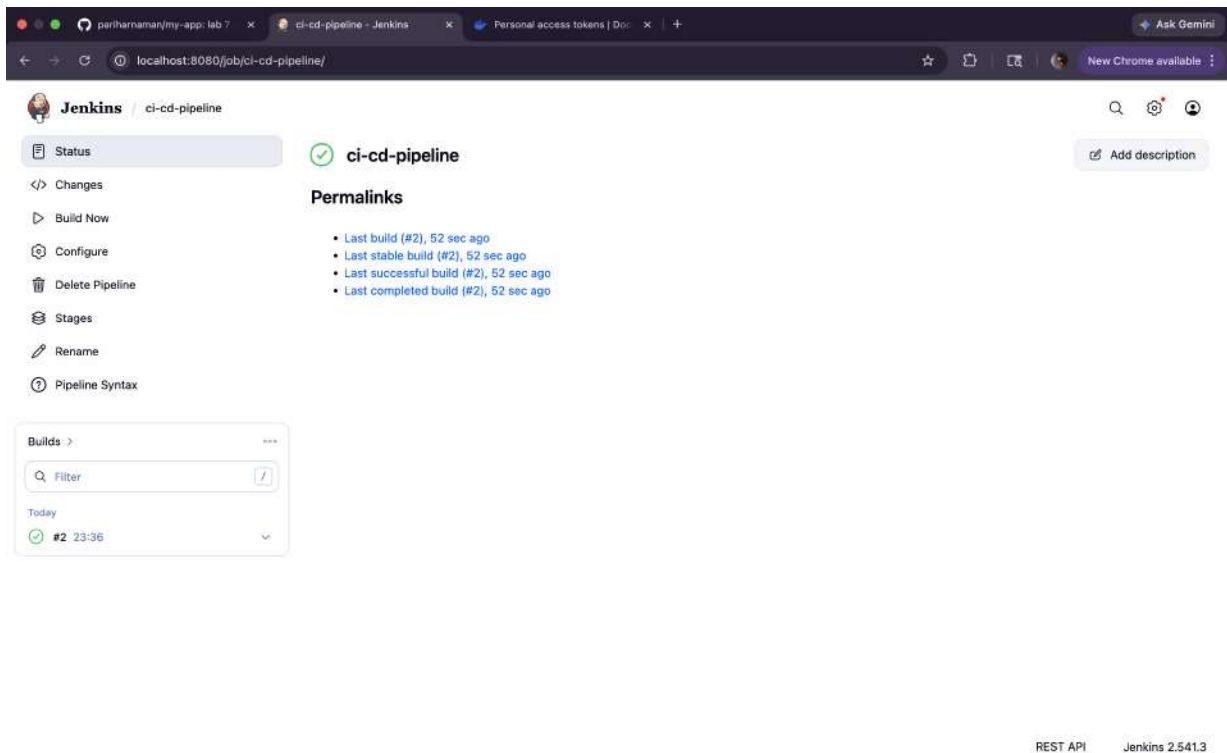
1: Add Docker Hub Credentials

- Configure credentials for Docker Hub access
- Enables pushing images from pipeline



2: Create Pipeline Job

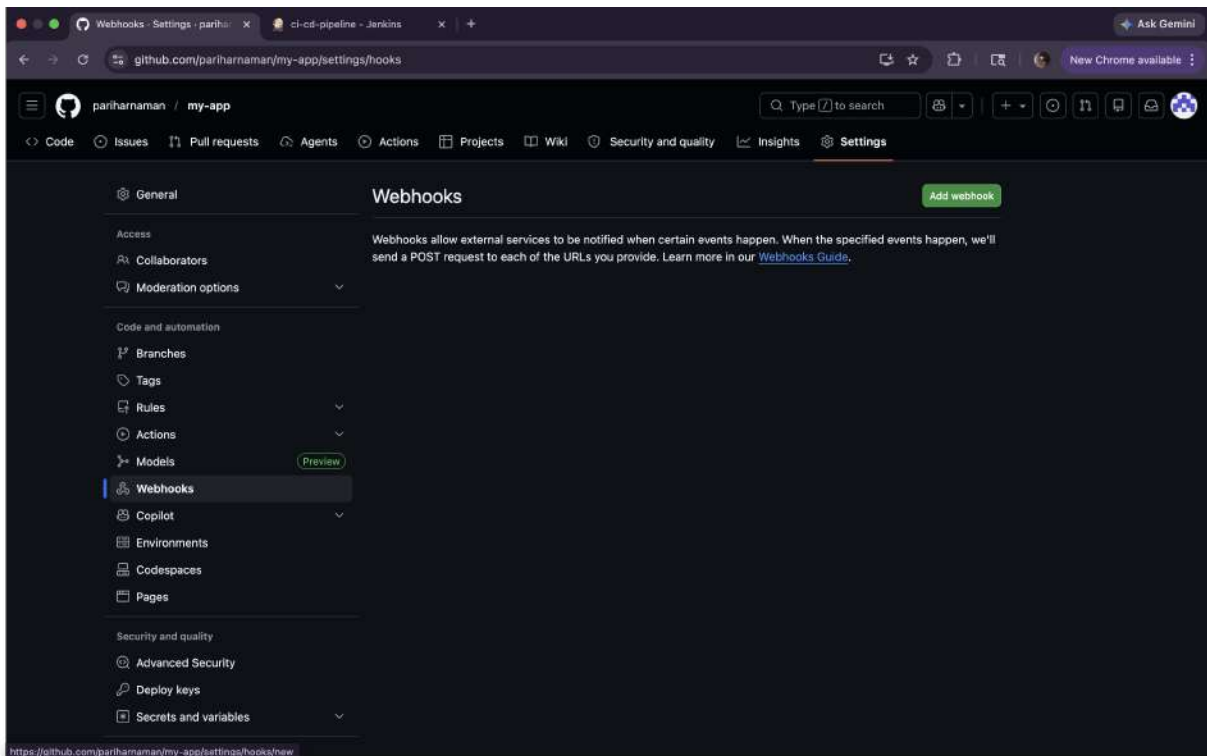
- Set up a Jenkins pipeline job
- Connects repository and automates build process



Part D: GitHub Webhook Integration

1: Configure Webhook

- Connect GitHub repository with Jenkins
- Triggers pipeline on code changes



2: Setup Ngrok for Communication

- Exposes local Jenkins server to the internet
- Enables GitHub to send webhook events

```
jenk — ngrok http 8080 — 80x24

ngrok (Ctrl+C to quit)

Take our ngrok in production survey! https://forms.gle/aXiBFWzEA36DudFn6

Session Status      online
Account             pariarnaman (Plan: Free)
Version             3.39.1
Region              India (in)
Latency              36ms
Web Interface        http://127.0.0.1:4040
Forwarding           https://untoasted-landmark-imaging.ngrok-free.dev

Connections          ttl    opn    rt1    rt5    p50    p90
                    0      0      0.00   0.00   0.00   0.00
```

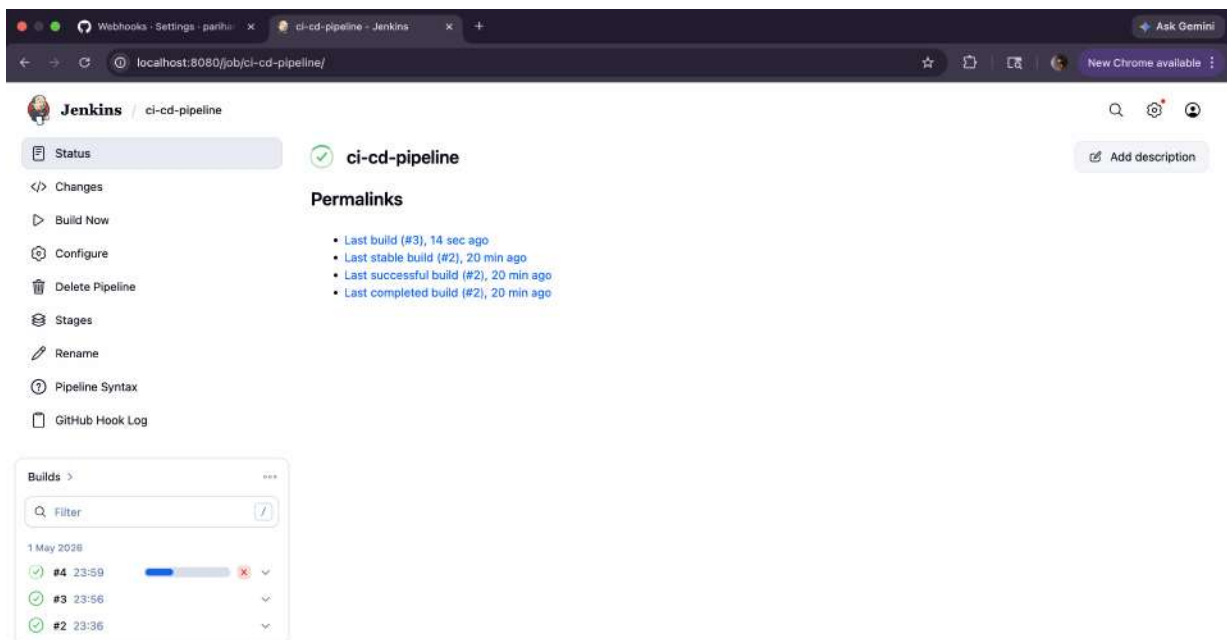
3: Test Your CI/CD Pipeline

- Push changes to trigger the pipeline
- Verify automated build and deployment

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
namansinghparihar@NAMANS-MacBook-Air-3 my-app % echo "webhook test 2" >> testt.txt
namansinghparihar@NAMANS-MacBook-Air-3 my-app % git add .
namansinghparihar@NAMANS-MacBook-Air-3 my-app % git commit -m " test cisd"
[main 9a10b9f] test cisd
Committer: NAMAN SINGH PARIHAR <namansinghparihar@NAMANS-MacBook-Air-3.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:
```



<https://pariharnaman.github.io/containerization-devops/Lab/Experiment9/>

containerization-devops

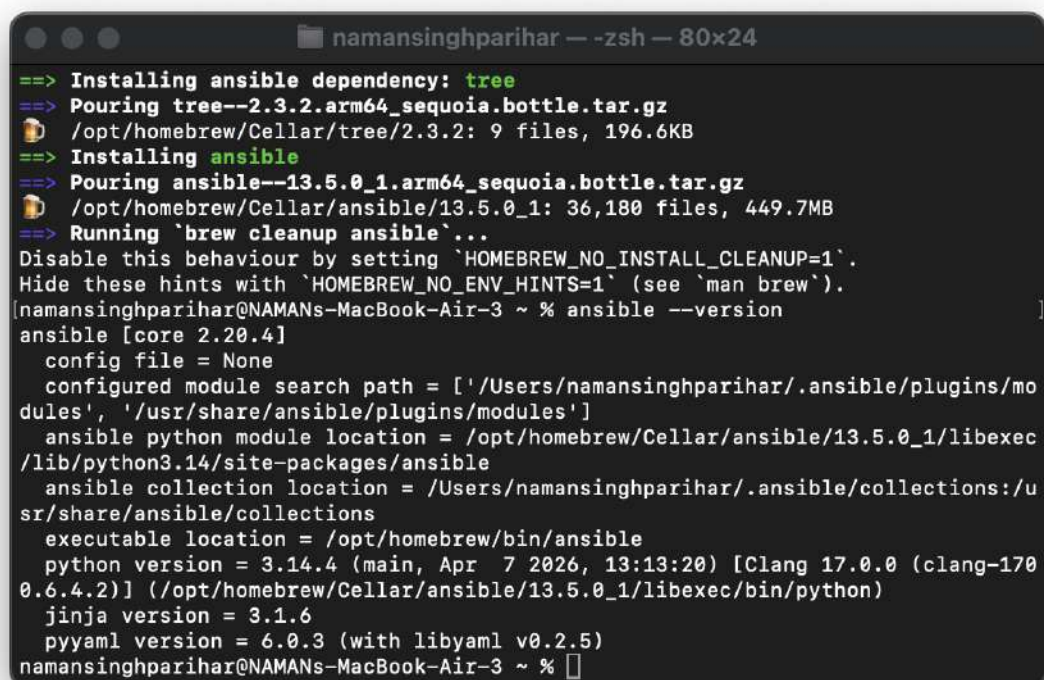
Experiment 9: Ansible

Step 1: Ansible Installation (macOS using Homebrew)

Installation

- Install Ansible using Homebrew:

```
brew install ansible
```

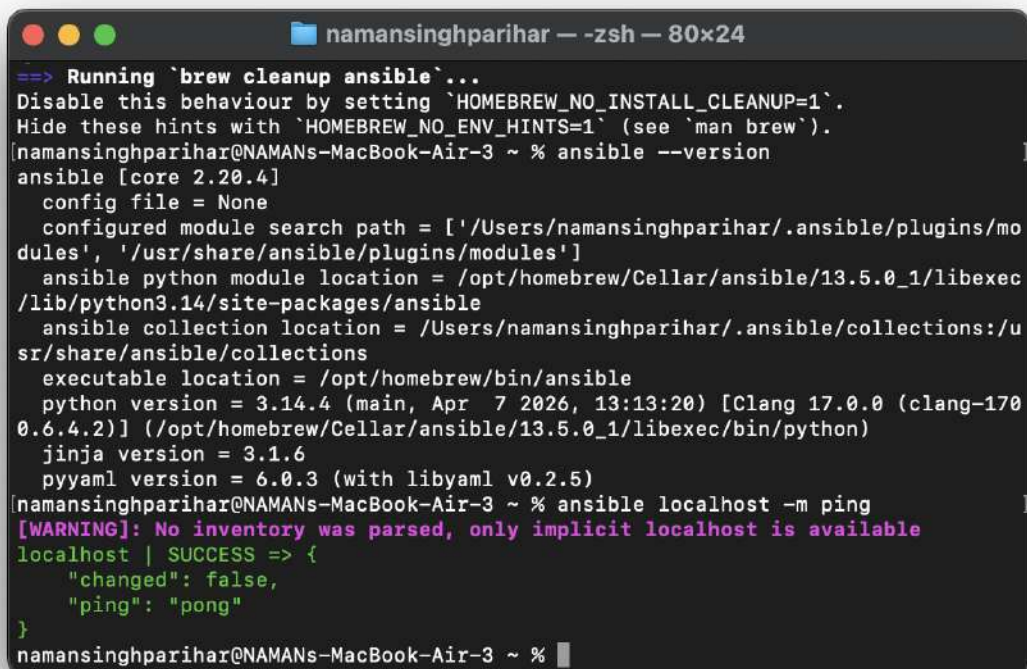


```
namansinghparihar — -zsh — 80x24
==> Installing ansible dependency: tree
==> Pouring tree--2.3.2.arm64_sequoia.bottle.tar.gz
🍺 /opt/homebrew/Cellar/tree/2.3.2: 9 files, 196.6KB
==> Installing ansible
==> Pouring ansible--13.5.0_1.arm64_sequoia.bottle.tar.gz
🍺 /opt/homebrew/Cellar/ansible/13.5.0_1: 36,180 files, 449.7MB
==> Running 'brew cleanup ansible'...
Disable this behaviour by setting 'HOMEBREW_NO_INSTALL_CLEANUP=1'.
Hide these hints with 'HOMEBREW_NO_ENV_HINTS=1' (see 'man brew').
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % ansible --version
ansible [core 2.20.4]
  config file = None
  configured module search path = ['/Users/namansinghparihar/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /opt/homebrew/Cellar/ansible/13.5.0_1/libexec/lib/python3.14/site-packages/ansible
  ansible collection location = /Users/namansinghparihar/.ansible/collections:/usr/share/ansible/collections
  executable location = /opt/homebrew/bin/ansible
  python version = 3.14.4 (main, Apr  7 2026, 13:13:20) [Clang 17.0.0 (clang-170.0.6.4.2)] (/opt/homebrew/Cellar/ansible/13.5.0_1/libexec/bin/python)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 2: Post-Installation Check

Verify that Ansible is installed correctly:

```
ansible --version
```



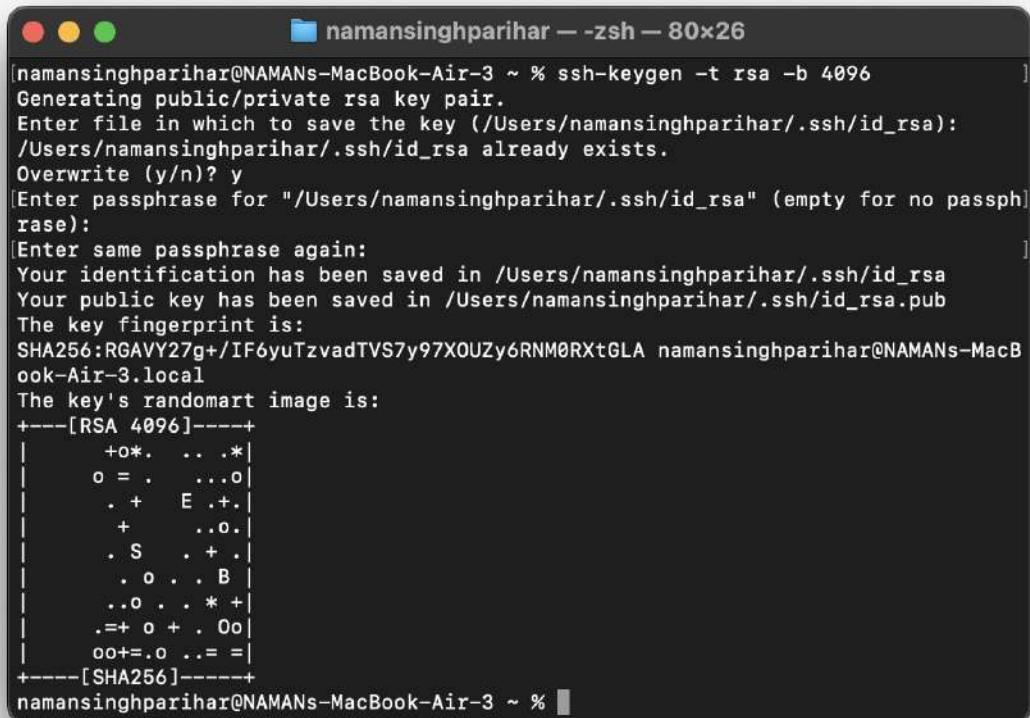
```
namansinghparihar — zsh — 80x24
==> Running `brew cleanup ansible`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % ansible --version
ansible [core 2.20.4]
  config file = None
  configured module search path = ['/Users/namansinghparihar/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /opt/homebrew/Cellar/ansible/13.5.0_1/libexec/lib/python3.14/site-packages/ansible
  ansible collection location = /Users/namansinghparihar/.ansible/collections:/usr/share/ansible/collections
  executable location = /opt/homebrew/bin/ansible
  python version = 3.14.4 (main, Apr  7 2026, 13:13:20) [Clang 17.0.0 (clang-170.0.6.4.2)] (/opt/homebrew/Cellar/ansible/13.5.0_1/libexec/bin/python)
  jinja version = 3.1.6
  pyyaml version = 6.0.3 (with libyaml v0.2.5)
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % ansible localhost -m ping
[WARNING]: No inventory was parsed, only implicit localhost is available
localhost | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 3: Create SSH Key Pair

An SSH key pair is used for secure authentication between systems.

- It consists of a **public key** and a **private key**
- The public key is shared with the server
- The private key remains secure on your system
- Enables password-less and secure login

Illustration



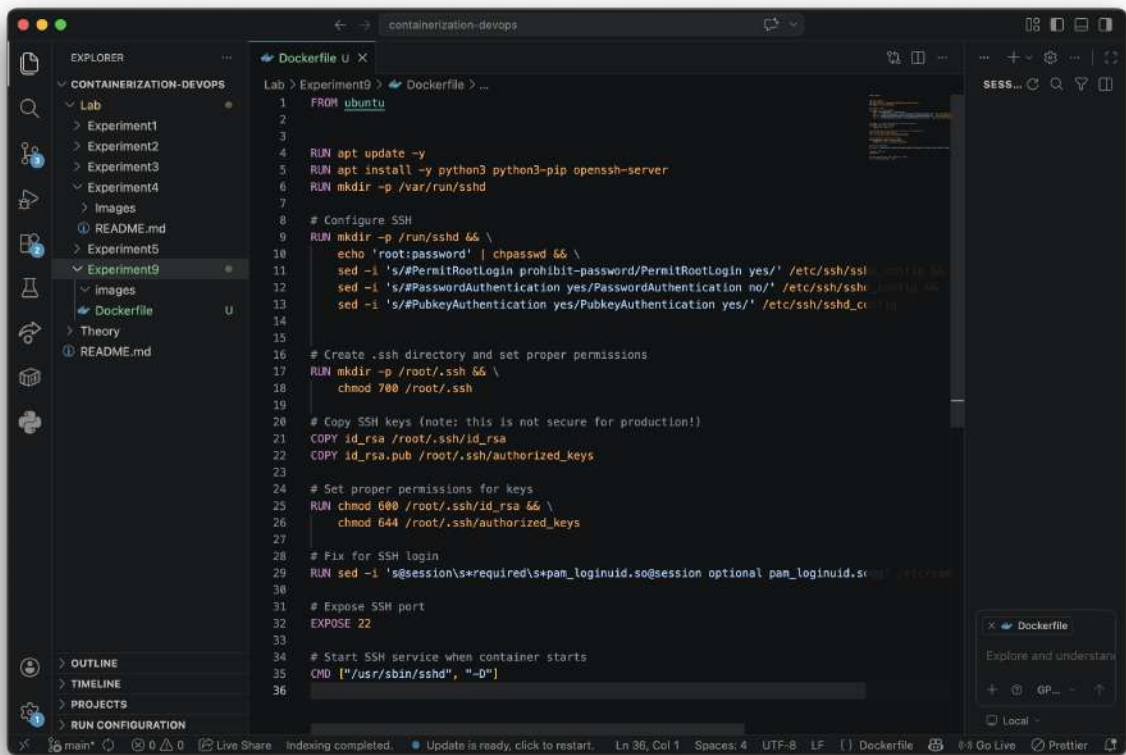
```
namansinghparihar — zsh — 80x26
namansinghparihar@NAMANs-MacBook-Air-3 ~ % ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/Users/namansinghparihar/.ssh/id_rsa):
/Users/namansinghparihar/.ssh/id_rsa already exists.
Overwrite (y/n)? y
[Enter passphrase for "/Users/namansinghparihar/.ssh/id_rsa" (empty for no passph
rase)]:
[Enter same passphrase again:
Your identification has been saved in /Users/namansinghparihar/.ssh/id_rsa
Your public key has been saved in /Users/namansinghparihar/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:RGAVY27g+/IF6yuTzvadTVS7y97XOUZy6RNM0RXtGLA namansinghparihar@NAMANs-MacB
ook-Air-3.local
The key's randomart image is:
+---[RSA 4096]-----+
|      +O* .  . . * |
|      o = .   . . o |
|      . +   E . + . |
|      +     . . o . |
|      . S   . + .   |
|      . o   . . B   |
|      ..o . . * +   |
|      . = + o + . 0o |
|      oo + = .o . . = |
+-----[SHA256]-----+
namansinghparihar@NAMANs-MacBook-Air-3 ~ %
```

Step 4: Create Dockerfile for Ubuntu SSH Server

A Dockerfile is used to define a custom image with an Ubuntu-based SSH server.

- Installs and configures SSH inside the container
- Allows remote access to the container
- Sets up required dependencies and configurations
- Helps create a reusable and consistent environment

Illustration



Step 5: Build the Docker Image

Building a Docker image converts the Dockerfile into a runnable image.

- Packages all configurations and dependencies
- Creates a reusable image for containers
- Ensures consistency across environments

Illustration

```
Experiment9 -- -zsh -- 120x44
namansinghparihar@NAMANS-MacBook-Air-3 Experiment9 % cp ~/.ssh/id_rsa .
namansinghparihar@NAMANS-MacBook-Air-3 Experiment9 % docker build -t ubuntu-server .

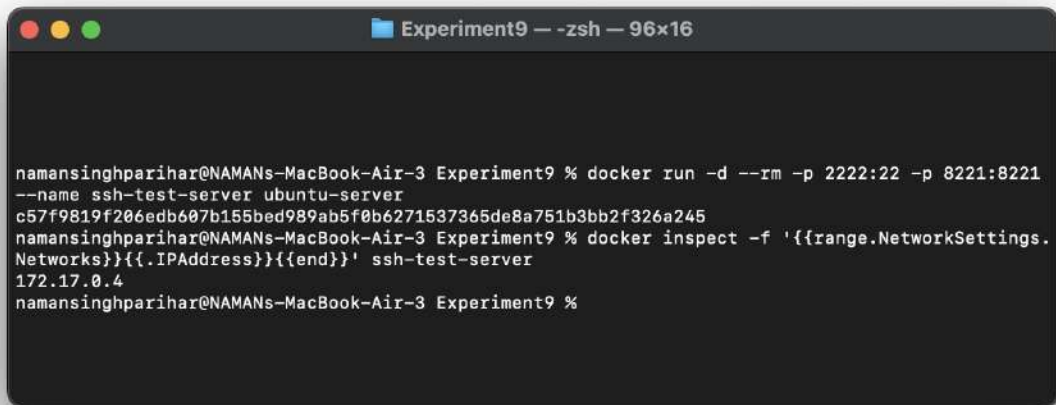
[+] Building 67.3s (14/15)                                docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 1.10kB                    0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                            0.0s
=> CACHED [ 1/10] FROM docker.io/library/ubuntu:latest@sha256:c4a8d5583dfb2a3eb8ab5f807da5bc69a85738fb49b5cfca2330194ebcc41c7b 0.0s
=> => resolve docker.io/library/ubuntu:latest@sha256:c4a8d5583dfb2a3eb8ab5f807da5bc69a85738fb49b5cfca2330194ebcc41c7b 0.0s
=> [internal] load build context                        0.0s
=> => transferring context: 3.49kB                        0.0s
=> [ 2/10] RUN apt update -y                          11.2s
=> [ 3/10] RUN apt install -y python3 python3-pip openssh-server 47.3s
=> [ 4/10] RUN mkdir -p /var/run/ssh                    0.1s
=> [ 5/10] RUN mkdir -p /run/ssh && echo 'root:password' | chpasswd && sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && sed -i 's/#PasswordAuthen 0.1s
[+] Building 67.4s (14/15)                                docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 1.10kB                    0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                            0.0s
=> CACHED [ 1/10] FROM docker.io/library/ubuntu:latest@sha256:c4a8d5583dfb2a3eb8ab5f807da5bc69a85738fb49b5cfca2330194eb
```

Step 6: Run the Docker Container

Running a container starts an instance of the built Docker image.

- Launches the application inside an isolated environment
- Allows interaction with the container (e.g., SSH access)
- Can be configured with ports, volumes, and settings

Illustration



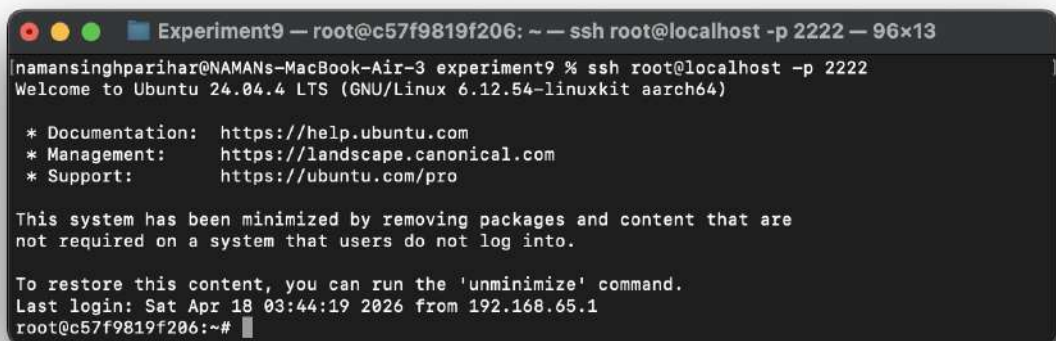
```
Experiment9 — -zsh — 96x16

namansinghparihar@NAMANS-MacBook-Air-3 Experiment9 % docker run -d --rm -p 2222:22 -p 8221:8221
--name ssh-test-server ubuntu-server
c57f9819f206edb607b155bed989ab5f0b6271537365de8a751b3bb2f326a245
namansinghparihar@NAMANS-MacBook-Air-3 Experiment9 % docker inspect -f '{{range.NetworkSettings.
Networks}}{{.IPAddress}}{{end}}' ssh-test-server
172.17.0.4
namansinghparihar@NAMANS-MacBook-Air-3 Experiment9 %
```

Step 7: Test SSH Connections

a) Test Password Authentication

- Verifies login using username and password
- Ensures SSH service is working correctly



```
Experiment9 — root@c57f9819f206: ~ — ssh root@localhost -p 2222 — 96x13

namansinghparihar@NAMANS-MacBook-Air-3 experiment9 % ssh root@localhost -p 2222
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.12.54-linuxkit aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Apr 18 03:44:19 2026 from 192.168.65.1
root@c57f9819f206:~#
```

b) Test Key-Based Authentication

- Verifies login using SSH key pair
- More secure and preferred over passwords

```
Experiment9 — root@c57f9819f206: ~ — ssh -i ~/.ssh/id_rsa root@localhost -p 2222 — 96...
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 % ssh -i ~/.ssh/id_rsa root@localhost -p 2222
The authenticity of host '[localhost]:2222 ([::1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:/S9ro9pmA0BtBDJB+iniyfpf09HuQuWoGPh0FPNwt5U.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.12.54-linuxkit aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@c57f9819f206:~# whoami
root
root@c57f9819f206:~#
```

Step 8: Start Multiple Containers (Ansible Targets)

Multiple containers are created to act as target servers for Ansible.

- Simulates a multi-server environment
- Each container acts as a managed node
- Useful for testing automation and configuration management

Illustration

```
Experiment9 — root@c57f9819f206: ~ — zsh — 96x28

namansinghparihar@NAMANs-MacBook-Air-3 experiment9 % for i in {1..4}; do
  echo -e "\n Creating server${i}\n"
  docker run -d --rm -p 220${i}:22 --name server${i} ubuntu-server
  echo -e "IP of server${i} is $(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' server${i})"
done

Creating server1

e5a2feabf82a5d6f388bc5cea6bdf9b53ccc15fbf704a8bbfeb6b6533df9b65
IP of server1 is 172.17.0.4

Creating server2

520290f6b35661dd75342123a0c9af37037c3a4feb99afc1b33fd52ed97e07c8
IP of server2 is 172.17.0.5

Creating server3

9310e27c52a2f69e1a71de12f7ba75ea32a67fa00b6fd58adfe990a1e5738b60
IP of server3 is 172.17.0.6

Creating server4

8ae6110580ffc9dbde94fabeb9f7e5fbe64b8910f6f3aee4c62076348dc9a278
IP of server4 is 172.17.0.8
namansinghparihar@NAMANs-MacBook-Air-3 experiment9 %
```

Step 9: Create Ansible Inventory

An Ansible inventory defines the list of target servers to be managed.

- Contains IP addresses or hostnames of servers
- Can group multiple servers together
- Helps Ansible know where to execute tasks

Illustration

```
Experiment9 — root@c57f9819f206: ~ — zsh — 96x21
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 % cat << EOF >> inventory.ini

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
EOF
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 % cat inventory.ini

[servers]
172.17.0.4
172.17.0.5
172.17.0.6
172.17.0.8

[servers:vars]
ansible_user=root
; ansible_ssh_private_key_file=./id_rsa
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 %
```

Step 10: Test Connectivity

a) SSH Test

- Verifies direct connection to target servers
- Ensures SSH access is working properly

```
Experiment9 — root@e5a2feabf82a: ~ — zsh — 96x23
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 % ssh root@localhost -p 2201
The authenticity of host '[localhost]:2201 ([::1]:2201)' can't be established.
ED25519 key fingerprint is SHA256:/S9ro9pmAOBtBDJB+iniyfpF09HuQuWoGPhOFPNwt5U.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:47: [localhost]:2222
  ~/.ssh/known_hosts:50: [localhost]:2202
  ~/.ssh/known_hosts:51: [localhost]:2203
  ~/.ssh/known_hosts:52: [localhost]:2204
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2201' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.12.54-linuxkit aarch64)

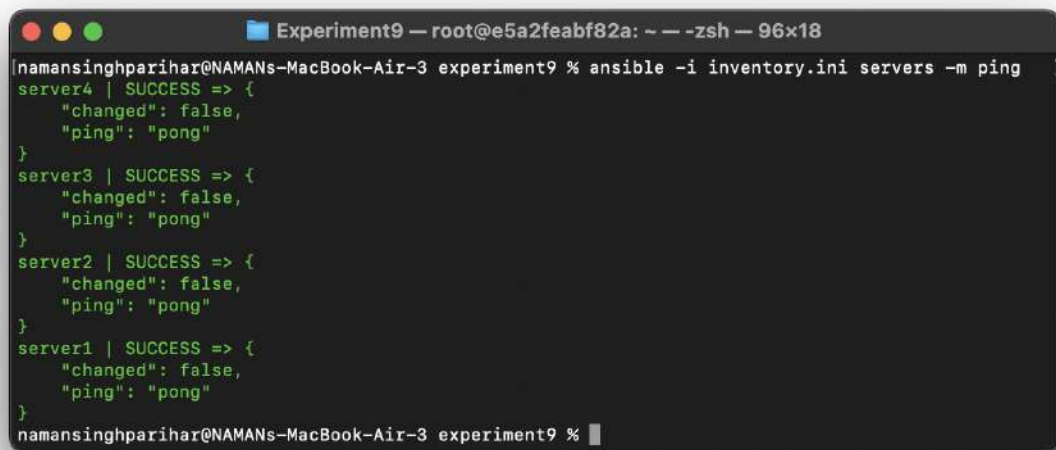
 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Apr 18 03:58:21 2026 from 192.168.65.1
root@e5a2feabf82a:~# exit
logout
```


b) Ansible Ping

- Checks connectivity using Ansible
- Confirms that managed nodes are reachable

A terminal window titled 'Experiment9' showing the execution of an Ansible command. The user 'namansinghparihar' is at a MacBook-Air-3. The command 'ansible -i inventory.ini servers -m ping' is executed, resulting in a JSON output for four servers (server1 to server4), all showing 'SUCCESS' and 'pong' response.

```
Experiment9 — root@e5a2feabf82a: ~ — zsh — 96x18
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 % ansible -i inventory.ini servers -m ping
server4 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server3 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server2 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
server1 | SUCCESS => {
  "changed": false,
  "ping": "pong"
}
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 %
```

Step 11: Create Playbook (update.yml)

An Ansible playbook is used to automate tasks on managed servers.

- Written in YAML format
- Defines tasks to be executed on target nodes
- Helps automate updates and configurations

Illustration

```
Experiment9 — root@e5a2feabf82a: ~ — zsh — 104x40

namansinghparihar@NAMANs-MacBook-Air-3 experiment9 % ansible-playbook -i inventory.ini playbook1.yml

PLAY [Update and configure servers] *****

TASK [Gathering Facts] *****
ok: [server4]
ok: [server3]
ok: [server2]
ok: [server1]

TASK [Update apt packages] *****
ok: [server4]
ok: [server2]
ok: [server1]
ok: [server3]

TASK [Install required packages] *****
changed: [server1]
changed: [server3]
changed: [server4]
changed: [server2]

TASK [Create test file] *****
changed: [server2]
changed: [server1]
changed: [server3]
changed: [server4]

PLAY RECAP *****
server1      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server2      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server3      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0
server4      : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0
  ignored=0

namansinghparihar@NAMANs-MacBook-Air-3 experiment9 %
```

Step 12: Verify Changes

a) Manually via Docker

- Check containers directly to confirm updates
- Ensures changes are applied correctly

```
Experiment9 — root@e5a2feabf82a: ~ — zsh — 104x12
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 % for i in {1..4}; do
  docker exec server${i} cat /root/ansible_test.txt
done
Configured by Ansible on server1Configured by Ansible on server2Configured by Ansible on server3Configur
ed by Ansible on server4%
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 %
```

b) Using Ansible

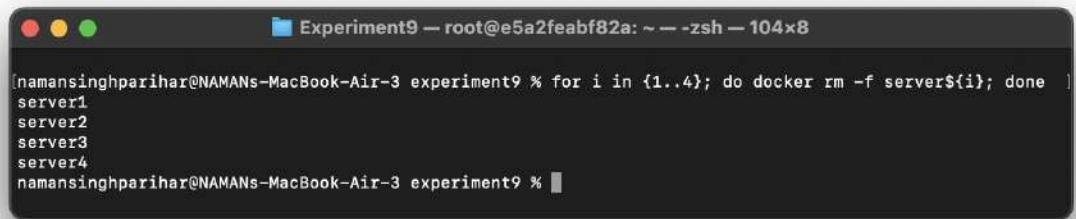
- Verify changes through automated tasks
- Confirms successful execution across all nodes

```
Experiment9 — root@e5a2feabf82a: ~ — zsh — 104x12
[namansinghparihar@NAMANS-MacBook-Air-3 experiment9 % ansible all -i inventory.ini -m command -a "cat /ro
ot/ansible_test.txt"
server4 | CHANGED | rc=0 >>
Configured by Ansible on server4
server1 | CHANGED | rc=0 >>
Configured by Ansible on server1
server2 | CHANGED | rc=0 >>
Configured by Ansible on server2
server3 | CHANGED | rc=0 >>
Configured by Ansible on server3
namansinghparihar@NAMANS-MacBook-Air-3 experiment9 %
```

Step 13: Clean Up (Remove Servers)

- Stop and remove containers after testing
- Frees system resources
- Keeps the environment clean and organized

Illustration



```
Experiment9 — root@e5a2feabf82a: ~ — zsh — 104x8
[namansinghparihar@NAMANs-MacBook-Air-3 experiment9 % for i in {1..4}; do docker rm -f server${i}; done ]
server1
server2
server3
server4
namansinghparihar@NAMANs-MacBook-Air-3 experiment9 %
```

[https://pariharnaman.github.io/containerization-devops/
Lab/Experiment10/](https://pariharnaman.github.io/containerization-devops/Lab/Experiment10/)

containerization-devops

Experiment 10: SonarQube — Static Code Analysis

SonarQube — Static Code Analysis

SonarQube is an open-source platform used for **static code analysis**, which means analyzing source code without executing it.

- Helps detect **bugs, vulnerabilities, and code smells**
- Improves **code quality and maintainability**
- Supports multiple languages (Java, Python, JavaScript, etc.)
- Integrates with CI/CD tools for continuous inspection

Key Features

- **Bug Detection:** Finds errors that may cause failures
- **Security Analysis:** Identifies vulnerabilities in code
- **Code Smells:** Highlights poor coding practices
- **Quality Gates:** Sets conditions to pass/fail code quality

Why Use SonarQube?

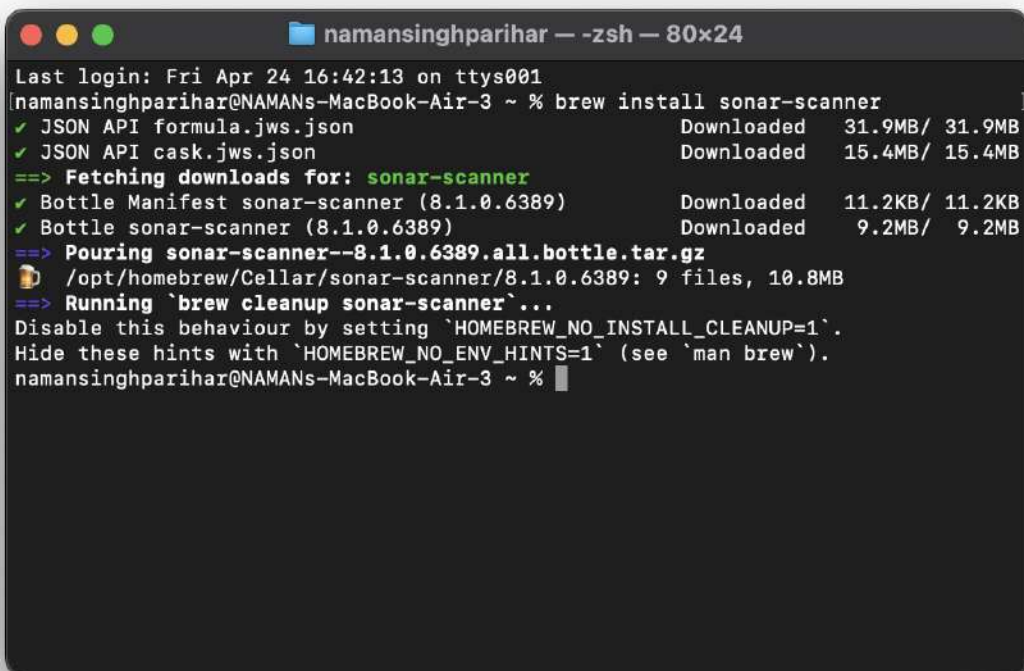
- Ensures **clean and secure code**
- Maintains **coding standards**
- Provides a **visual dashboard** for results
- Useful in **team-based development and DevOps pipelines**

Step 1: Installing SonarQube

Installed SonarQube using Homebrew (macOS):

- Simple and quick installation using package manager
- Automatically handles dependencies
- Easy to manage and update

Illustration



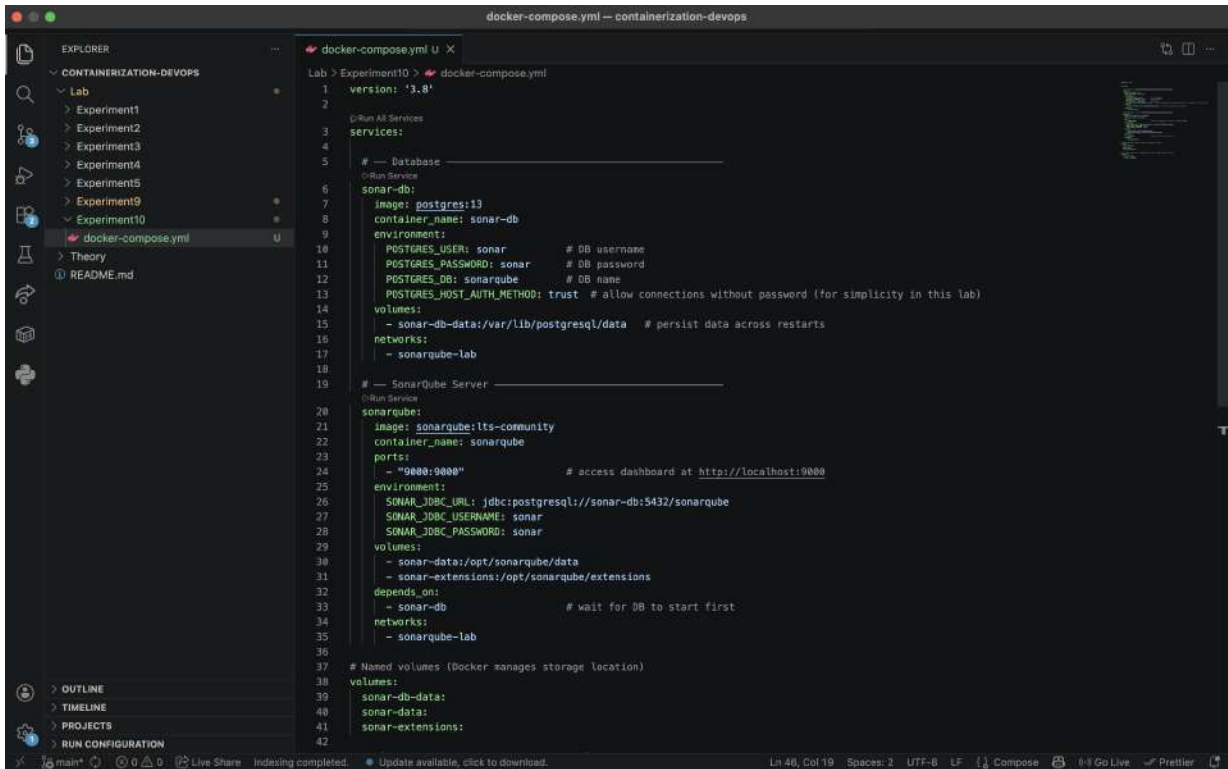
```
namansinghparihar — zsh — 80x24
Last login: Fri Apr 24 16:42:13 on ttys001
[namansinghparihar@NAMANS-MacBook-Air-3 ~ % brew install sonar-scanner ]
✓ JSON API formula.jws.json           Downloaded 31.9MB/ 31.9MB
✓ JSON API cask.jws.json              Downloaded 15.4MB/ 15.4MB
==> Fetching downloads for: sonar-scanner
✓ Bottle Manifest sonar-scanner (8.1.0.6389) Downloaded 11.2KB/ 11.2KB
✓ Bottle sonar-scanner (8.1.0.6389)      Downloaded 9.2MB/ 9.2MB
==> Pouring sonar-scanner--8.1.0.6389.all.bottle.tar.gz
📦 /opt/homebrew/Cellar/sonar-scanner/8.1.0.6389: 9 files, 10.8MB
==> Running `brew cleanup sonar-scanner`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
namansinghparihar@NAMANS-MacBook-Air-3 ~ %
```

Step 2: Start the SonarQube Server

The SonarQube server is started using Docker Compose.

- Runs SonarQube along with a PostgreSQL database
- Ensures all required services start together
- Simplifies setup and management

Illustration

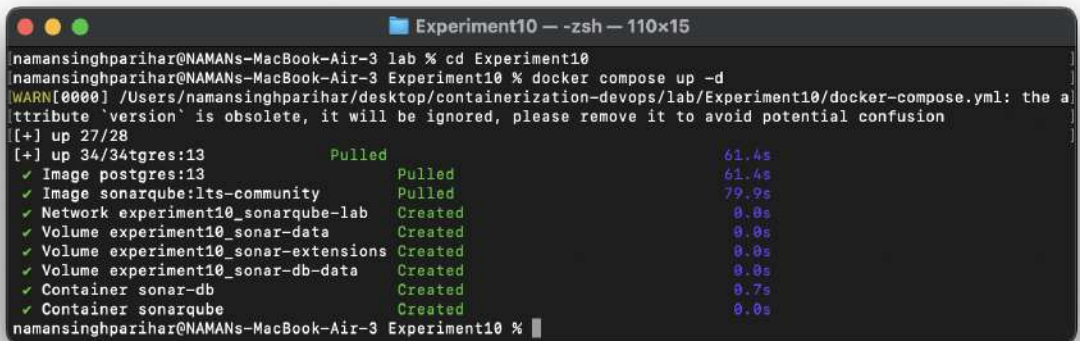


```
docker-compose.yml
version: '3.8'

services:
  # --- Database ---
  sonar-db:
    image: postgres:13
    container_name: sonar-db
    environment:
      POSTGRES_USER: sonar # DB username
      POSTGRES_PASSWORD: sonar # DB password
      POSTGRES_DB: sonarqube # DB name
      POSTGRES_HOST_AUTH_METHOD: trust # allow connections without password (for simplicity in this lab)
    volumes:
      - sonar-db-data:/var/lib/postgresql/data # persist data across restarts
    networks:
      - sonarqube-lab

  # --- SonarQube Server ---
  sonarqube:
    image: sonarqube:its-community
    container_name: sonarqube
    ports:
      - "9000:9000" # access dashboard at http://localhost:9000
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - sonar-data:/opt/sonarqube/data
      - sonar-extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db # wait for DB to start first
    networks:
      - sonarqube-lab

# Named volumes (Docker manages storage location)
volumes:
  sonar-db-data:
  sonar-data:
  sonar-extensions:
```

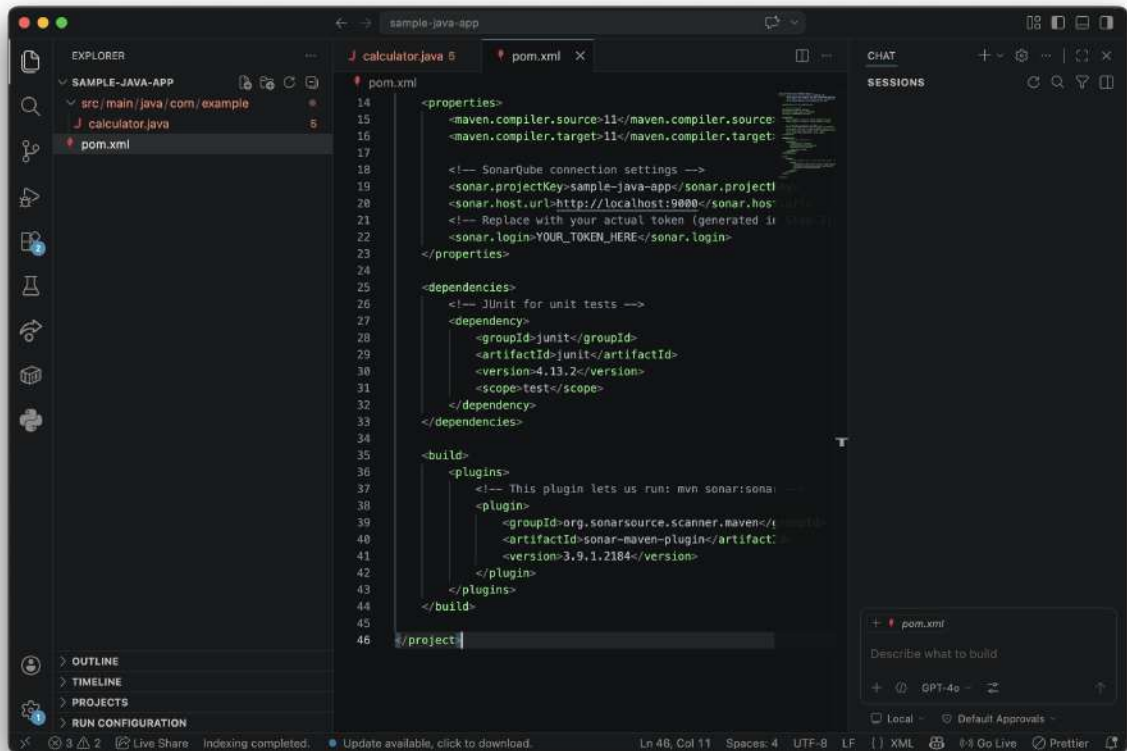


```
Experiment10 --zsh-- 110x15
namansinghparihar@NAMANS-MacBook-Air-3 lab % cd Experiment10
namansinghparihar@NAMANS-MacBook-Air-3 Experiment10 % docker compose up -d
[WARN[0000] /Users/namansinghparihar/desktop/containerization-devops/lab/Experiment10/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 27/28
[+] up 34/34
postgres:13 Pulled 61.4s
Image postgres:13 Pulled 61.4s
Image sonarqube:its-community Pulled 79.9s
Network experiment10_sonarqube-lab Created 0.0s
Volume experiment10_sonar-data Created 0.0s
Volume experiment10_sonar-extensions Created 0.0s
Volume experiment10_sonar-db-data Created 0.0s
Container sonar-db Created 0.7s
Container sonarqube Created 0.0s
namansinghparihar@NAMANS-MacBook-Air-3 Experiment10 %
```

Step 3: Create a Sample Java App with Code Issues

a) Sample App with Issues

- Create a simple Java class with intentional bugs
- Includes vulnerabilities and code smells
- Helps demonstrate SonarQube analysis



b) Create `pom.xml` (Maven Build File)

- Defines project dependencies and build configuration
- Required for building and analyzing the project
- Used by SonarQube during scanning

```
sample-java-app -- zsh -- 98x33
namansinghparihar@NAMANS-MacBook-Air-3 sample-java-app % mvn clean compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:sample-app >-----
[INFO] Building sample-app 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ sample-app ---
[INFO] Deleting /Users/namansinghparihar/sample-java-app/target
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ sample-app ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /Users/namansinghparihar/sample-java-app/src/main/resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ sample-app ---
[INFO] Recompiling the module because of changed source code.
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file with javac [debug target 11] to target/classes
[WARNING] location of system modules is not set in conjunction with -source 11
[ ] not setting the location of system modules may lead to class files that cannot run on JDK 11
[ ] --release 11 is recommended instead of -source 11 -target 11 because it sets the location of system modules automatically
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.366 s
[INFO] Finished at: 2026-04-24T17:02:17+05:30
[INFO]
namansinghparihar@NAMANS-MacBook-Air-3 sample-java-app %
```

Step 4: Generate Authentication Token

The SonarQube scanner requires a token to authenticate with the server.

- Generated manually from the SonarQube web UI
- Used for secure communication between scanner and server
- Acts as an alternative to username/password

Illustration

Generate Tokens

Name	Type	Expires in	
Enter Token Name	Select Token Type	30 days	Generate

 New token "scanner-token" has been created. Make sure you copy it now, you won't be able to see it again!

`squ_afd5a68bb553d0011f8a28740bf8c1ef3a4354d1`

Step 5: Run the Scanner

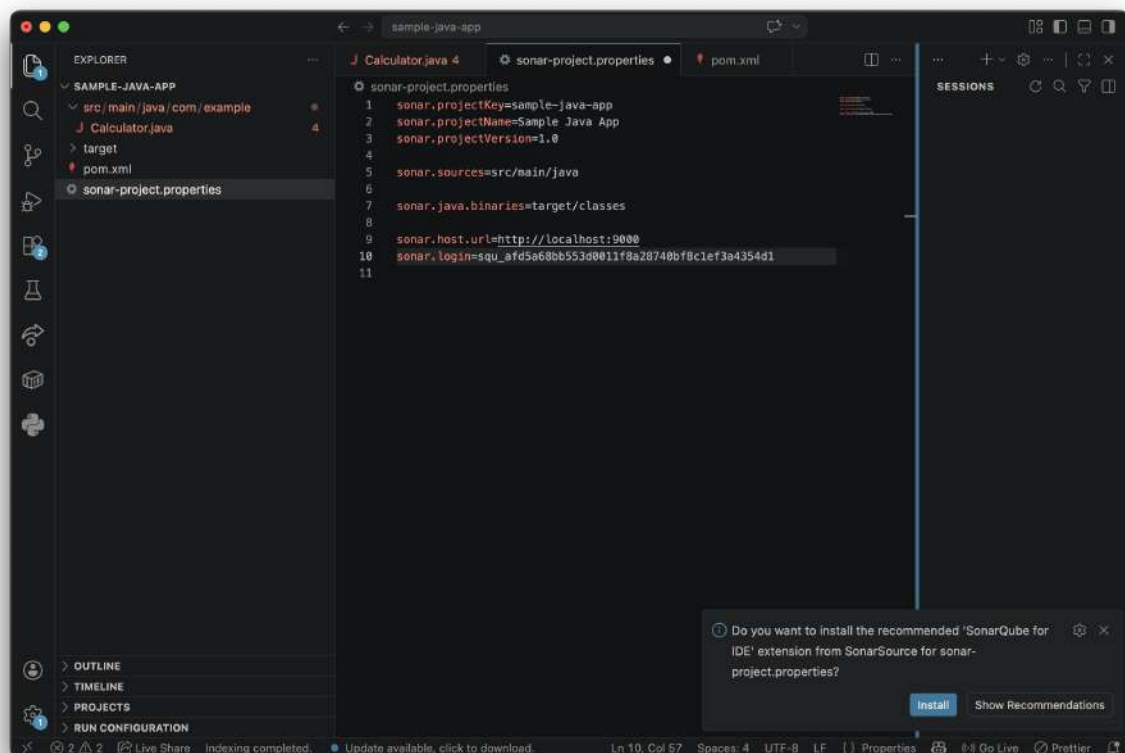
The Sonar Scanner is used to analyze the project code.

- Uses Sonar Scanner CLI (language independent)
- Requires a configuration file in the project root
- Sends analysis results to the SonarQube server

Configuration File: `sonar-project.properties`

- Contains project details and settings
- Helps the scanner understand what to analyze

Illustration



```

nanansinghparihar@NAMANS-MacBook-Air-3 sample-java-app % sonar-scanner
17:08:54.881 INFO Scanner configuration file: /opt/homebrew/Cellar/sonar-scanner/6.1.8.6389/libexec/conf/sonar-scanner.properties
17:08:54.882 INFO Project root configuration file: /Users/nanansinghparihar/sample-java-app/sonar-project.properties
17:08:54.887 INFO SonarScanner CLI 6.1.0.6389
17:08:54.888 INFO Mac OS X 15.7.2 aarch64
17:08:55.922 INFO Communicating with SonarQube Server 9.9.8.100196
17:08:56.242 INFO Load global settings
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe:arrayBaseOffset has been called by com.google.protobuf.UnsafeUtil$MemoryAccessor (file:/Users/nanansinghparihar/.sonar/cache/8386cbfda2785c8e493c8e645396dcfe/sonar-scanner-engine-sha
head-9.9.8.100196-all.jar)
WARNING: please consider reporting this to the maintainers of class com.google.protobuf.UnsafeUtil$MemoryAccessor
WARNING: sun.misc.Unsafe:arrayBaseOffset will be removed in a future release
17:08:56.291 INFO load global settings (done) | time=80ms
17:08:56.293 INFO Server id: 7AC553A8-A22-F0u4v0e7.2w97zz
17:08:56.293 INFO User cache: /Users/nanansinghparihar/.sonar/cache
17:08:56.295 INFO Load/download plugins
17:08:56.295 INFO Load plugins index
17:08:56.327 INFO Load plugins index (done) | time=32ms
17:08:57.020 INFO Load/download plugins (done) | time=725ms
17:08:57.156 INFO Process project properties
17:08:57.159 INFO Process project properties (done) | time=3ms
17:08:57.168 INFO Execute project builders
17:08:57.168 INFO Execute project builders (done) | time=0ms
17:08:57.162 INFO Project key: sample-java-app
17:08:57.162 INFO Base dir: /Users/nanansinghparihar/sample-java-app
17:08:57.162 INFO Working dir: /Users/nanansinghparihar/sample-java-app/.scannerwork
17:08:57.164 INFO Load project settings for component key: 'sample-java-app'
17:08:57.285 WARN SCM provider autodetection failed. Please use "sonar.scm.provider" to define SCM of your project, or disable the SCM Sensor in the project settings.
17:08:57.288 INFO Load quality profiles
17:08:57.246 INFO Load quality profiles (done) | time=38ms
17:08:57.256 INFO Load active rules
17:08:57.917 INFO Load active rules (done) | time=661ms
17:08:57.918 INFO Load analysis cache
17:08:57.921 INFO Load analysis cache (404) | time=3ms
17:08:57.937 INFO Load project repositories
17:08:57.940 INFO Load project repositories (done) | time=3ms
17:08:57.951 INFO Indexing files...
17:08:57.952 INFO Project configuration:
17:08:57.957 INFO 1 file indexed
17:08:57.957 INFO Quality profile for java: Sonar way
17:08:57.957 INFO ----- Run sensors on module Sample Java App
17:08:57.982 INFO Load metrics repository
17:08:57.995 INFO Load metrics repository (done) | time=13ms
17:08:58.058 INFO Sensor JavaSensor [java]
17:08:58.359 INFO Configured Java source version (sonar.java.source): none
17:08:58.361 INFO JavaClasspath initialization
17:08:58.361 INFO JavaClasspath initialization (done) | time=0ms
17:08:58.361 INFO JavaTestClasspath initialization
17:08:58.361 INFO JavaTestClasspath initialization (done) | time=0ms
17:08:58.363 INFO Server-side caching is enabled. The Java analyzer will not try to leverage data from a previous analysis.
17:08:58.364 INFO Using ECJ batch to parse 1 Main java source files with batch size 214 KB.
17:08:58.488 INFO Starting batch processing.
17:08:58.546 INFO The Java analyzer cannot skip unchanged files in this context. A full analysis is performed for all files.
17:08:58.648 INFO 100% analyzed
17:08:58.648 INFO Batch processing: Done.
17:08:58.648 INFO Did not optimize analysis for any files, performed a full analysis for all 1 files.
17:08:58.642 INFO No "Test" source files to scan.
17:08:58.642 INFO No "Generated" source files to scan.
17:08:58.641 INFO Sensor JavaSensor [java] (done) | time=283ms
17:08:58.641 INFO Sensor JaCoCo XML Report importer [jacoco]
17:08:58.641 INFO 'sonar.coverage.jacoco.xmlReportPaths' is not defined. Using default locations: target/site/jacoco/jacoco.xml,target/site/jacoco-it/jacoco.xml,build/reports/jacoco/test/jacocoTestReport.xml
17:08:58.641 INFO No report imported, no coverage information will be imported by JaCoCo XML Report Importer
17:08:58.641 INFO Sensor JaCoCo XML Report Importer [jacoco] (done) | time=0ms
17:08:58.641 INFO Sensor CSS Rules [javascript]

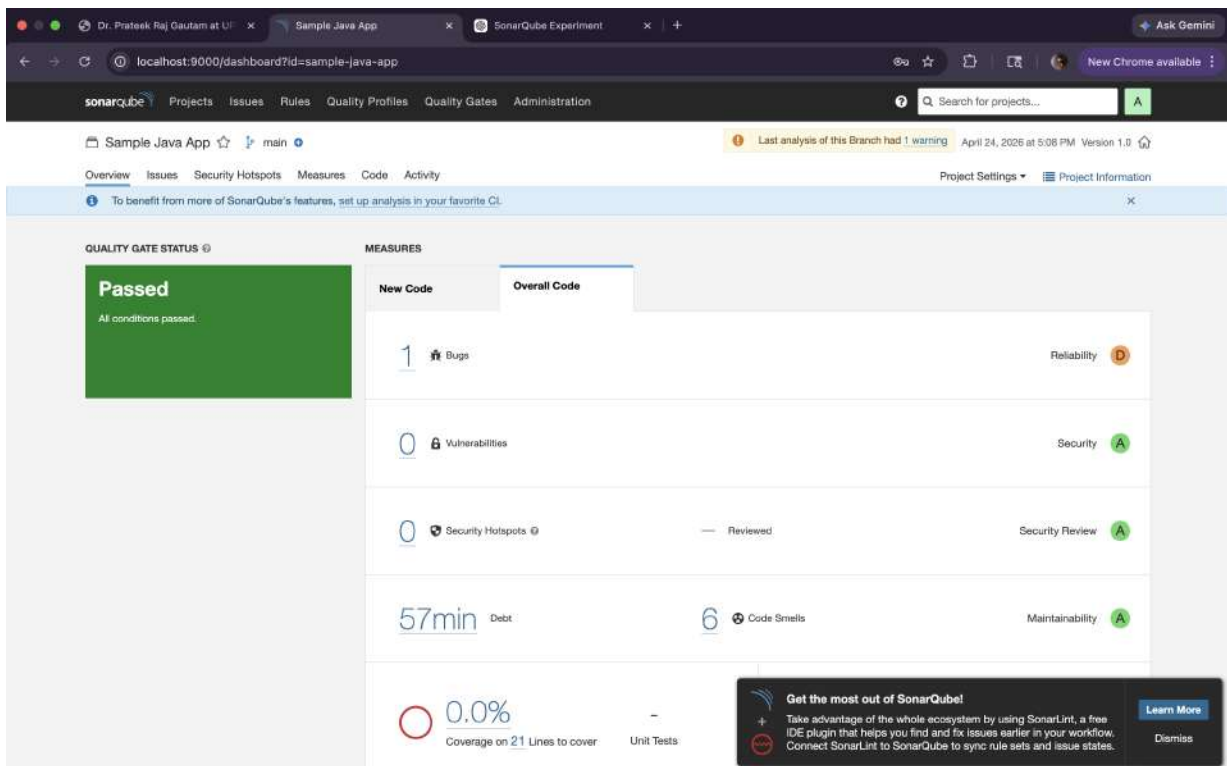
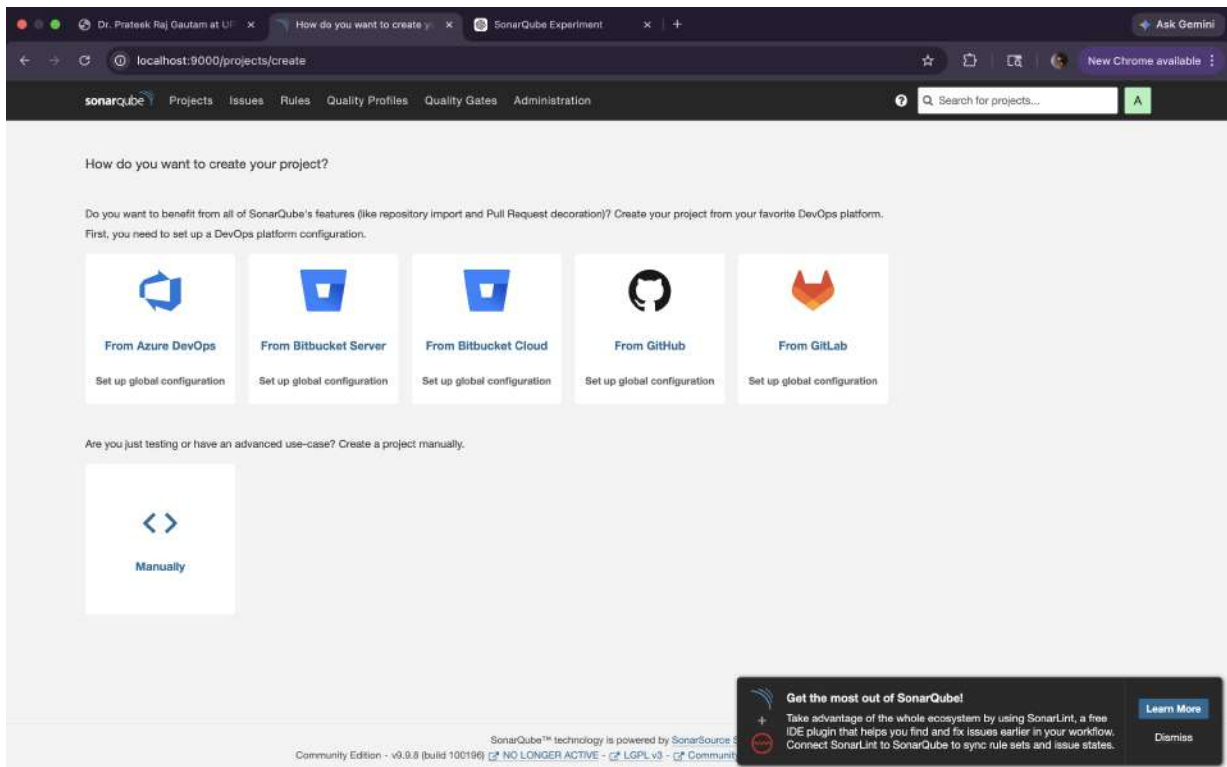
```

Step 6: View Results in the Dashboard

After the scan is complete, results can be viewed in the SonarQube dashboard.

- Displays bugs, vulnerabilities, and code smells
- Provides detailed code analysis reports
- Helps track code quality and improvements

Illustration

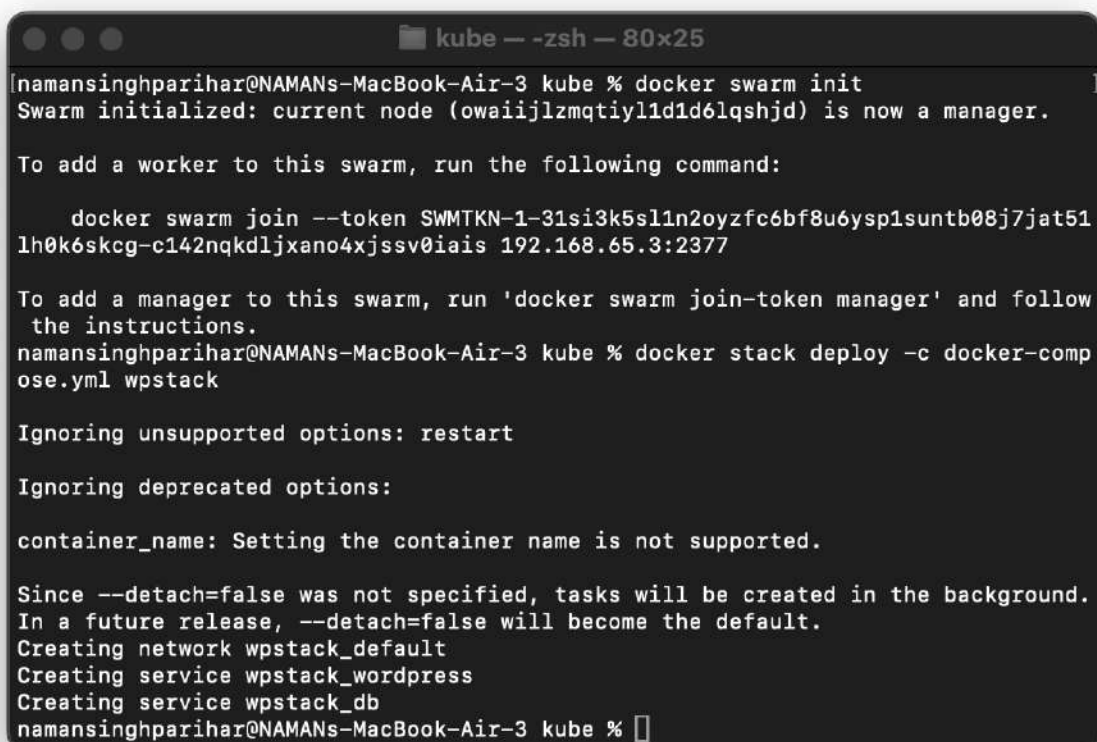


containerization-devops

Experiment 11: Orchestration using Docker Compose & Docker Swarm

Task 1: Initialize Swarm & Deploy Stack

- Initialized Docker Swarm mode
- Deployed application stack using a compose file
- Services started as swarm-managed containers



```
kube — -zsh — 80x25
namansinghparihar@NAMANS-MacBook-Air-3 kube % docker swarm init
Swarm initialized: current node (owaiijlzmqtiyl1d1d6lqshjd) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-31si3k5sl1n2oyzfc6bf8u6ysp1suntb08j7jat51
    lh0k6skcg-c142nqkdljxano4xjssv0iais 192.168.65.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow
the instructions.
namansinghparihar@NAMANS-MacBook-Air-3 kube % docker stack deploy -c docker-comp
ose.yml wpstack

Ignoring unsupported options: restart

Ignoring deprecated options:

container_name: Setting the container name is not supported.

Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network wpstack_default
Creating service wpstack_wordpress
Creating service wpstack_db
namansinghparihar@NAMANS-MacBook-Air-3 kube %
```

Task 2: Scale WordPress & Test Self-Healing

- Scaled WordPress service to multiple replicas

- Verified all replicas were running
- Manually removed one container
- Swarm automatically recreated it (**self-healing**)

```

namansinghparihar@NAMANS-MacBook-Air-3 kube % docker service scale wpstack_wordpress=3

wpstack_wordpress scaled to 3
overall progress: 3 out of 3 tasks
1/3: running
2/3: running
3/3: running
verify: Service wpstack_wordpress converged
namansinghparihar@NAMANS-MacBook-Air-3 kube % docker service ls

ID                NAME                MODE                REPLICAS    IMAGE                PORTS
gperg3hkp2n4      wpstack_db          replicated          1/1          mysql:8.0
pvq2411hbsgs      wpstack_wordpress   replicated          3/3          wordpress:latest     *:8080->80/
tcp
namansinghparihar@NAMANS-MacBook-Air-3 kube % docker ps | grep wordpress

8ccc45a2d040      wordpress:latest    "docker-entrypoint.s..." 35 seconds ago    Up 34 seconds
80/tcp            wpstack_wordpress.3.mb4wqadcie4u1ewfjzgfvveia
015efb25fc27      wordpress:latest    "docker-entrypoint.s..." 35 seconds ago    Up 34 seconds
80/tcp            wpstack_wordpress.2.ysqv7oole0koagmfjms24xm2e
71ad8627fd7c      wordpress:latest    "docker-entrypoint.s..." 2 minutes ago     Up 2 minutes
80/tcp            wpstack_wordpress.1.2hwajr0iijcrpjb16lhp2er0u
namansinghparihar@NAMANS-MacBook-Air-3 kube % docker kill 8ccc

[8ccc
namansinghparihar@NAMANS-MacBook-Air-3 kube % docker service ps wpstack_wordpress

ID                NAME                ERROR              IMAGE                NODE                DESIRED STATE
E CURRENT STATE    ERROR              IMAGE                NODE                DESIRED STATE
2hwajr0iijcr      wpstack_wordpress.1  Running 3 minutes ago    wordpress:latest     docker-desktop      Running
ysqv7oole0ko      wpstack_wordpress.2  Running about a minute ago    wordpress:latest     docker-desktop      Running
pg6k18g0wtpw      wpstack_wordpress.3  Running 11 seconds ago      wordpress:latest     docker-desktop      Running
mb4wqadcie4u      \_ wpstack_wordpress.3  Failed 17 seconds ago      "task: non-zero exit (137)"  docker-desktop      Shutdown

```

Task 3: Remove Stack & Verify

- Removed the deployed stack
- All services and containers were stopped
- Verified clean environment


```

    kube — -zsh — 89x13
namansinghparihar@NAMANs-MacBook-Air-3 kube % docker stack rm wpstack

Removing service wpstack_db
Removing service wpstack_wordpress
Removing network wpstack_default
namansinghparihar@NAMANs-MacBook-Air-3 kube % docker service ls

ID          NAME          MODE          REPLICAS    IMAGE          PORTS
namansinghparihar@NAMANs-MacBook-Air-3 kube %
```

<https://pariharnaman.github.io/containerization-devops/Lab/Experiment12/>

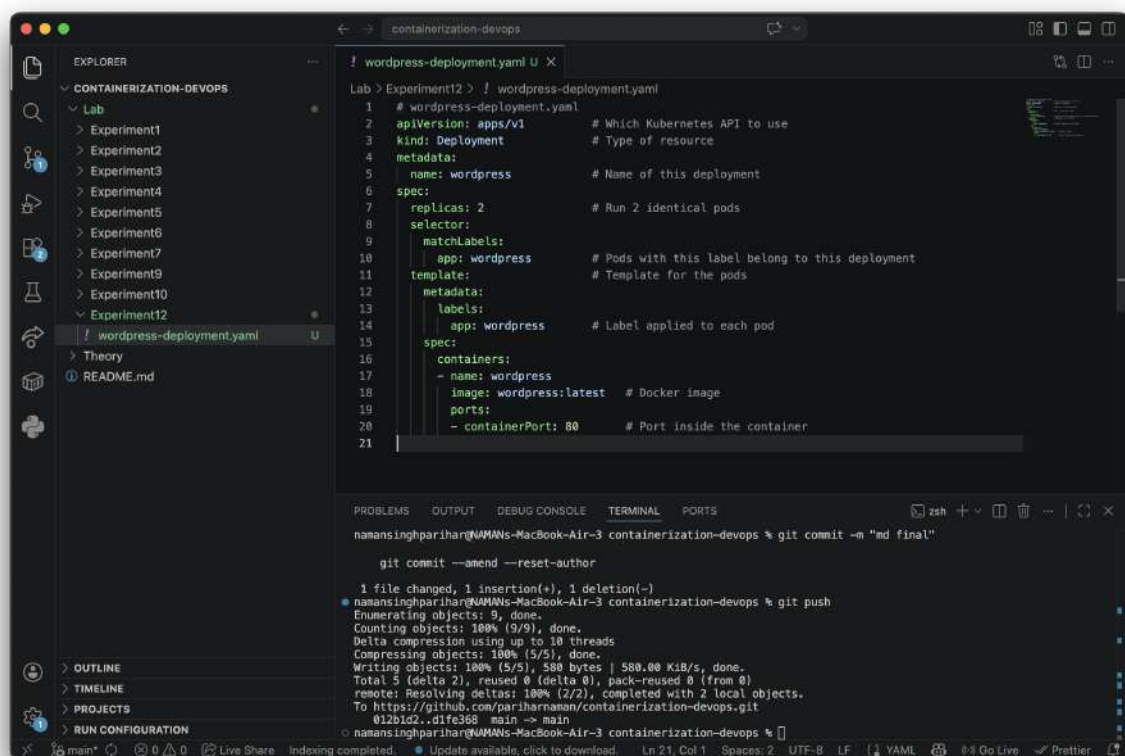
containerization-devops

Experiment 12: Study and Analyse Container Orchestration using Kubernetes

Task 1: Create a Deployment

Step 1: Create `wordpress-deployment.yaml`

- Defines Kubernetes deployment configuration
- Specifies pods, replicas, and container details



The screenshot shows a VS Code editor window with the following components:

- EXPLORER:** A sidebar on the left showing a file tree for 'CONTAINERIZATION-DEVOPS'. It includes a 'Lab' folder with subfolders 'Experiment1' through 'Experiment12'. The file 'wordpress-deployment.yaml' is selected under 'Experiment12'.
- Editor:** The main workspace displays the content of 'wordpress-deployment.yaml'. The file is a Kubernetes Deployment manifest for WordPress, configured with 2 replicas and the 'wordpress:latest' image.
- TERMINAL:** A terminal window at the bottom shows the execution of git commands to commit and push the file to a GitHub repository.

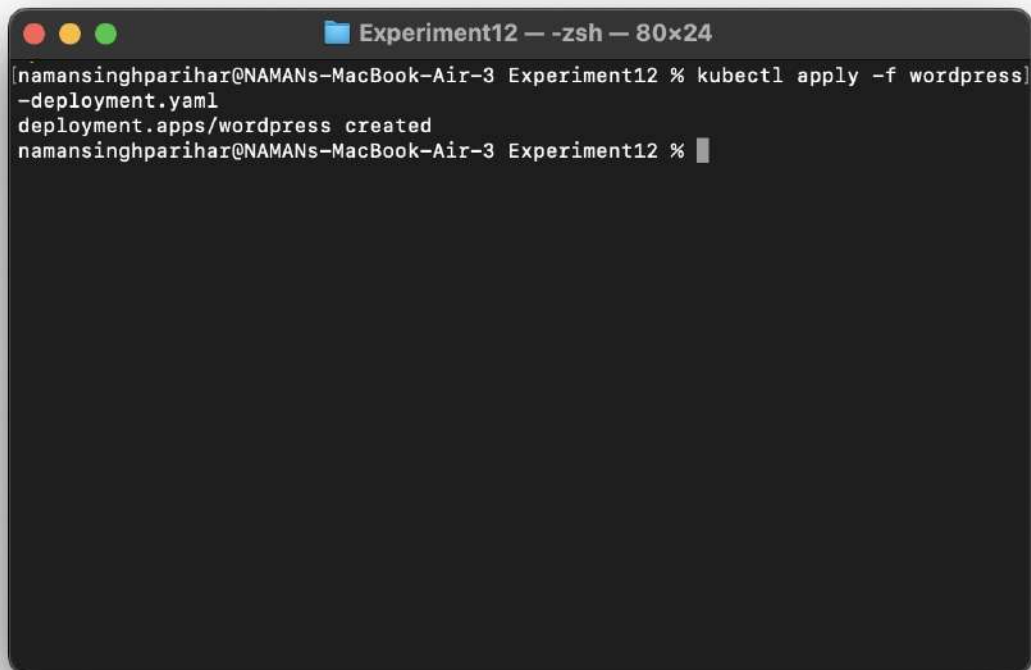
```
1 # wordpress-deployment.yaml
2 apiVersion: apps/v1 # Which Kubernetes API to use
3 kind: Deployment # Type of resource
4 metadata:
5   name: wordpress # Name of this deployment
6 spec:
7   replicas: 2 # Run 2 identical pods
8   selector:
9     matchLabels:
10      app: wordpress # Pods with this label belong to this deployment
11 template: # Template for the pods
12   metadata:
13     labels:
14      app: wordpress # Label applied to each pod
15   spec:
16     containers:
17       - name: wordpress
18         image: wordpress:latest # Docker image
19         ports:
20           - containerPort: 80 # Port inside the container
```

Terminal Output:

```
namansinghparihar@NAMANS-MacBook-Air-3 containerization-devops % git commit -m "md final"
git commit --amend --reset-author
1 file changed, 1 insertion(+), 1 deletion(-)
namansinghparihar@NAMANS-MacBook-Air-3 containerization-devops % git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 10 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 580 bytes | 580.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/pariharnaman/containerization-devops.git
012b1d2..d1fe368 main -> main
namansinghparihar@NAMANS-MacBook-Air-3 containerization-devops %
```

Step 2: Apply the Deployment

- Creates resources in the Kubernetes cluster
- Starts the defined application pods

A terminal window titled "Experiment12 - zsh - 80x24" is shown. The prompt is "namansinghparihar@NAMANS-MacBook-Air-3 Experiment12 %". The user has entered the command "kubectl apply -f wordpress-deployment.yaml". The output is "deployment.apps/wordpress created". The prompt is now "namansinghparihar@NAMANS-MacBook-Air-3 Experiment12 %".

```
Experiment12 - zsh - 80x24
namansinghparihar@NAMANS-MacBook-Air-3 Experiment12 % kubectl apply -f wordpress-
deployment.yaml
deployment.apps/wordpress created
namansinghparihar@NAMANS-MacBook-Air-3 Experiment12 %
```

Task 2: Pod Scaling and Self-Healing

- Initially deployed with **2 pods**
- Scaled the deployment to **4 pods** to handle increased load
- Verified that all 4 pods were running successfully
- Manually deleted one pod to test reliability
- Kubernetes automatically recreated the deleted pod (**self-healing**)
- Ensures high availability and maintains desired state

Illustration

```
Experiment12 -- zsh -- 103x24
namansinghparihar@NAMANs-MacBook-Air-3 Experiment12 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-cbglw         1/1     Running   0           12m
wordpress-6698dd7d66-m6qmb         1/1     Running   0           12m
namansinghparihar@NAMANs-MacBook-Air-3 Experiment12 % kubectl scale deployment wordpress --replicas=4

deployment.apps/wordpress scaled
namansinghparihar@NAMANs-MacBook-Air-3 Experiment12 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-cbglw         1/1     Running   0           13m
wordpress-6698dd7d66-gktfl         1/1     Running   0           15s
wordpress-6698dd7d66-m6qmb         1/1     Running   0           13m
wordpress-6698dd7d66-nphz9         1/1     Running   0           15s
namansinghparihar@NAMANs-MacBook-Air-3 Experiment12 % kubectl delete pod wordpress
Error from server (NotFound): pods "wordpress" not found
namansinghparihar@NAMANs-MacBook-Air-3 Experiment12 % kubectl delete pod wordpress-6698dd7d66-cbglw
pod "wordpress-6698dd7d66-cbglw" deleted from default namespace
namansinghparihar@NAMANs-MacBook-Air-3 Experiment12 % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
wordpress-6698dd7d66-59dzl         1/1     Running   0            9s
wordpress-6698dd7d66-gktfl         1/1     Running   0           95s
wordpress-6698dd7d66-m6qmb         1/1     Running   0           14m
wordpress-6698dd7d66-nphz9         1/1     Running   0           95s
namansinghparihar@NAMANs-MacBook-Air-3 Experiment12 %
```